Algorithms I

Mihir Rao

Contents

1 Introduction: Integer Multiplication and Merge Sort **6**
1.1 Integer multiplication and the cost of a step . . . 6
1.2 Recursive multiplication and Karatsuba's idea . . . 6
1.3 Divide and conquer through sorting . . . 7
1.4 Merging two sorted arrays . . . 8
1.5 Counting work in one merge . . . 9
1.6 The recursion-tree analysis . . . 9
1.7 Three principles for algorithm analysis . . . 10

2 Asymptotic Analysis **12**
2.1 Counting loops before simplifying . . . 12
2.2 Big-O: an eventual upper bound . . . 13
2.3 Polynomial examples and fixed witnesses . . . 14
2.4 Omega and Theta: lower and matching bounds . . . 14
2.5 Little-o and the order of the quantifiers . . . 16
2.6 Constants inside an exponent . . . 16
2.7 A sum and a maximum have the same growth . . . 17

3 Divide and Conquer Algorithms **18**
3.1 Counting inversions . . . 18
3.2 Piggybacking on merge sort . . . 19
3.3 Matrix multiplication and Strassen's algorithm . . . 20
3.4 The closest pair problem . . . 21
3.5 Correctness of the split-pair search . . . 22

4 The Master Method **25**
4.1 Recurrences for integer multiplication . . . 25
4.2 The precise statement . . . 25
4.3 Examples . . . 26
4.4 The recursion tree . . . 26
4.5 Intuition for the three cases . . . 27
4.6 Completing the proof . . . 27

5 QuickSort: Algorithm **29**
5.1 Partitioning around a pivot . . . 29
5.2 In-place partitioning . . . 30
5.3 Correctness . . . 31
5.4 The importance of the pivot . . . 32
5.5 Random pivots . . . 32

6 QuickSort: Analysis **33**
6.1 Reducing running time to comparisons . . . 33
6.2 Decomposing the comparison count . . . 33

6.3 The key insight . . . 34
6.4 Final calculation . . . 35

7 Probability for Randomized Algorithms **37**
7.1 Sample spaces and events . . . 37
7.2 Random variables and expectation . . . 37
7.3 Linearity of expectation . . . 38
7.4 Conditional probability . . . 38
7.5 Independence . . . 39

8 Linear-Time Selection **41**
8.1 The selection problem . . . 41
8.2 Partitioning and randomized selection . . . 41
8.3 Analysis of randomized selection . . . 42
8.4 Deterministic selection . . . 43
8.5 Analysis of deterministic selection . . . 44
8.6 A sorting lower bound . . . 45

9 Graphs and the Contraction Algorithm **47**
9.1 Graphs and cuts . . . 47
9.2 Representing graphs . . . 48
9.3 The random contraction algorithm . . . 48
9.4 Success probability of one run . . . 49
9.5 Repeated trials . . . 50
9.6 Counting minimum cuts . . . 51

10 Graph Search and Connectivity **52**
10.1 Generic graph search . . . 52
10.2 Breadth-first search . . . 53
10.3 Shortest paths via BFS . . . 54
10.4 Undirected connectivity . . . 54
10.5 Depth-first search . . . 55
10.6 Topological sort . . . 55
10.7 Strongly connected components . . . 57
10.8 Correctness of Kosaraju's algorithm . . . 58
10.9 Structure of the Web . . . 59

11 Dijkstra's Shortest-Path Algorithm **61**
11.1 Single-source shortest paths . . . 61
11.2 The algorithm . . . 61
11.3 Negative edge lengths . . . 62
11.4 Correctness . . . 63
11.5 A heap-based implementation . . . 64

12 Heaps **66**
12.1 Heap operations . . . 66
12.2 Applications . . . 66
12.3 The heap property . . . 67
12.4 Insert and bubble-up . . . 68
12.5 Extract-Min and bubble-down . . . 69

13 Balanced Binary Search Trees **71**
13.1 Sorted arrays and balanced search trees . . . 71
13.2 Binary search tree structure . . . 71
13.3 Search, insert, min, max, and predecessor . . . 72

13.4 In-order traversal and deletion . . . 73
13.5 Select and rank . . . 73
13.6 Red-black trees . . . 74
13.7 Rotations . . . 75
13.8 Insertion into a red-black tree . . . 76

14 Hashing: The Basics **78**
14.1 Applications . . . 78
14.2 Buckets and hash functions . . . 79
14.3 Resolving collisions . . . 80
14.4 What makes a good hash function . . . 80

15 Universal Hashing **82**
15.1 The load of a hash table . . . 82
15.2 Pathological data sets . . . 82
15.3 Universal families of hash functions . . . 83
15.4 A universal family for IP addresses . . . 83
15.5 Chaining with a universal family . . . 84
15.6 Open addressing: heuristic analysis . . . 85

16 Bloom Filters **86**
16.1 The data structure . . . 86
16.2 Heuristic analysis . . . 87

17 Greedy Algorithms **89**
17.1 The greedy paradigm . . . 89
17.2 Proving greedy algorithms correct . . . 89
17.3 Application: optimal caching . . . 89

18 A Scheduling Application **91**
18.1 Problem definition . . . 91
18.2 Designing the greedy algorithm . . . 91
18.3 Correctness with distinct ratios . . . 92
18.4 Handling ties . . . 93

19 Prim's Minimum Spanning Tree Algorithm **94**
19.1 The minimum spanning tree problem . . . 94
19.2 Prim's algorithm . . . 95
19.3 Cuts and three graph facts . . . 95
19.4 Part I: Prim's algorithm outputs a spanning tree . . . 96
19.5 Part II: the cut property . . . 96
19.6 Proof of the cut property . . . 97
19.7 A fast implementation with heaps . . . 98

20 Kruskal's Minimum Spanning Tree Algorithm **100**
20.1 Kruskal's algorithm . . . 100
20.2 Correctness . . . 101
20.3 Running time of the straightforward implementation . . . 102
20.4 The union-find data structure . . . 102
20.5 State of the art and open questions . . . 103

21 Clustering **105**
21.1 Max-spacing k-clusterings . . . 105
21.2 Single-link clustering . . . 105
21.3 Correctness . . . 106

22 Advanced Union-Find **108**
22.1 Lazy unions . . . 108
22.2 Union by rank . . . 109
22.3 Path compression . . . 111
22.4 The Hopcroft-Ullman bound . . . 112
22.5 The Ackermann function . . . 113
22.6 Tarjan's bound . . . 114

23 Huffman Codes **117**
23.1 Prefix-free codes . . . 117
23.2 Codes as trees . . . 117
23.3 Huffman's greedy algorithm . . . 118
23.4 Correctness . . . 120
23.5 Running time . . . 121

24 Introduction to Dynamic Programming **122**
24.1 Weighted independent sets in path graphs . . . 122
24.2 Optimal substructure . . . 123
24.3 Eliminating redundant recursion . . . 123
24.4 Reconstructing an optimal solution . . . 124
24.5 Principles of dynamic programming . . . 125

25 The Knapsack Problem **126**
25.1 Optimal substructure . . . 126
25.2 The recurrence and the subproblems . . . 126
25.3 An example . . . 127

26 Sequence Alignment **129**
26.1 Problem definition . . . 129
26.2 Optimal substructure . . . 129
26.3 Subproblems and recurrence . . . 130
26.4 The algorithm . . . 131
26.5 Reconstructing an optimal alignment . . . 131

27 Optimal Binary Search Trees **132**
27.1 Search trees and non-uniform access . . . 132
27.2 Problem definition . . . 132
27.3 Greedy does not work . . . 133
27.4 Optimal substructure . . . 134
27.5 Subproblems and recurrence . . . 134
27.6 The algorithm . . . 135

28 The Bellman-Ford Algorithm **137**
28.1 Negative cycles . . . 137
28.2 Optimal substructure . . . 138
28.3 The recurrence and the algorithm . . . 139
28.4 Detecting negative cycles . . . 140
28.5 Space and predecessor pointers . . . 140
28.6 From Bellman-Ford to Internet routing . . . 141

29 All-Pairs Shortest Paths **142**
29.1 Reduction to single-source shortest paths . . . 142
29.2 Optimal substructure . . . 142
29.3 The Floyd-Warshall algorithm . . . 143
29.4 Reweighting . . . 144

29.5 Johnson's algorithm . . . 145

30 NP-Complete Problems **147**
30.1 Polynomial-time solvability and the class P . . . 147
30.2 Reductions and completeness . . . 148
30.3 The class NP and NP-completeness . . . 148
30.4 The P versus NP question . . . 149
30.5 Algorithmic approaches to NP-complete problems . . . 150

31 Faster Exact Algorithms for NP-Complete Problems **151**
31.1 The vertex cover problem . . . 151
31.2 Smarter search for small vertex covers . . . 152
31.3 The traveling salesman problem . . . 153
31.4 A dynamic programming algorithm for TSP . . . 154

32 Approximation Algorithms for NP-Complete Problems **156**
32.1 Knapsack and a greedy heuristic . . . 156
32.2 Analysis of the greedy heuristic . . . 157
32.3 A dynamic programming heuristic . . . 158
32.4 Dynamic programming for knapsack, revisited . . . 159
32.5 Analysis of the dynamic programming heuristic . . . 160

33 Local Search Algorithms **161**
33.1 The maximum cut problem . . . 161
33.2 Principles of local search . . . 163
33.3 The 2-SAT problem . . . 164
33.4 Random walks on a line . . . 164
33.5 Analysis of Papadimitriou's algorithm . . . 166

34 Matching, Flows, and Linear Programming **167**
34.1 Stable matchings . . . 167
34.2 Bipartite matching and maximum flow . . . 168
34.3 Selfish flow and Braess's paradox . . . 169
34.4 Linear programming . . . 170
34.5 Other topics and models . . . 170

Lecture 1: Introduction: Integer Multiplication and Merge Sort

An algorithm turns an input into a required output. Design chooses the steps; analysis establishes correctness and cost.

Two familiar tasks, multiplying integers and sorting numbers, introduce a recurring question: can we improve on the obvious method?

1.1 Integer multiplication and the cost of a step

The input is two nonnegative integers x and y, each represented with n decimal digits; the required output is xy. Here a primitive operation adds or multiplies two single digits. Multiplying two arbitrarily long integers is therefore not one primitive operation.

The grade-school method forms one partial product for each digit of y, then adds the shifted rows. For the given example,

$$\begin{array}{r} 5678 \\ \times 1234 \\ \hline 22712 \\ 170340 \\ 1135600 \\ 5678000 \\ \hline 7006652 \end{array}$$

There are n rows, each involving work proportional to n, including digit multiplication and carrying. The final additions also take at most a constant times n^2 work. Thus the total grows quadratically with the number of digits. Doubling the digit count roughly quadruples this work.

1.2 Recursive multiplication and Karatsuba's idea

Split each input into a high half and a low half. With $m = n/2$ for an even digit count, write

$$x = 10^m a + b, \qquad y = 10^m c + d,$$

where each block uses at most m digits, allowing leading zeros. Expanding the product gives

$$xy = 10^{2m} ac + 10^m (ad + bc) + bd. \tag{1.1}$$

This identity reduces the original task to products of smaller integers. A direct recursive method computes ac, ad, bc, bd separately, then combines them using equation (1.1). Small inputs are

handled directly.

Karatsuba multiplication reuses information to remove one of these four recursive products. Compute

$$u = ac, \qquad v = bd, \qquad w = (a+b)(c+d).$$

Since $w = ac + ad + bc + bd$, the middle coefficient is $w - u - v = ad + bc$. Therefore

$$\boxed{xy = 10^{2m}u + 10^{m}(w - u - v) + v.}$$

There are three recursive multiplications, plus additions, subtractions, and decimal shifts. Sums such as $a + b$ can have an extra carry digit; a complete implementation must preserve it. The smaller number of recursive products is the reason to investigate this method further, rather than counting a recursive call as a single inexpensive step.

Example: For $x = 5678$ and $y = 1234$, take $a = 56$, $b = 78$, $c = 12$, $d = 34$. Then

$$u = 56 \cdot 12 = 672, \qquad v = 78 \cdot 34 = 2652,$$
$$w = 134 \cdot 46 = 6164, \quad w - u - v = 2840.$$

Recombining gives

$$10^4(672) + 10^2(2840) + 2652 = 7006652,$$

which agrees with the grade-school calculation.

1.3 Divide and conquer through sorting

A divide and conquer algorithm splits a task into smaller subproblems, solves them recursively, and combines their answers. The combination step matters: reducing the input size is useful only if the work of solving and combining the subproblems is controlled.

Definition 1.1 [Sorting problem]. Given an array of n distinct numbers, return the same numbers in increasing order. Correctness requires both ordering and preservation of the input elements.

Merge sort splits the array into two halves, sorts each half recursively, and merges the sorted halves. Arrays of length zero or one are already sorted.

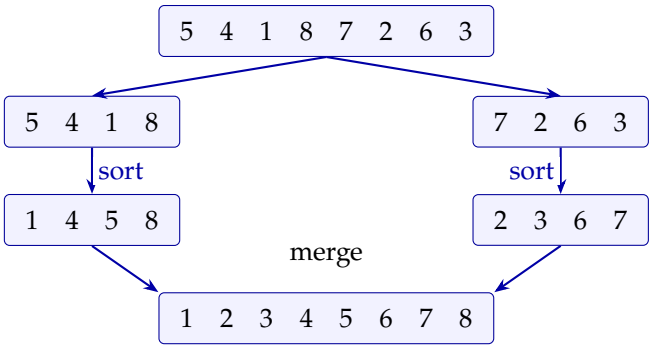


Figure 1: Merge sort on the supplied eight-element array. The two recursive calls return sorted halves; the final merge combines them in increasing order.

Algorithm 1.1: Merge sort

Input: Array X of length n, with distinct entries.
Output: The entries of X in increasing order.

```
if n ≤ 1 then
    return X
m ← ⌊n/2⌋
A ← MergeSort(X[1 : m])
B ← MergeSort(X[m + 1 : n])
return Merge(A, B)
```

Both recursive arrays have length smaller than n when $n \geq 2$, so the procedure eventually reaches a base case. The floor handles odd lengths without discarding an element.

1.4 Merging two sorted arrays

The smallest unused element of a sorted array is its first unused element. To find the next output element, it is therefore enough to compare the two current front elements. After choosing one, advance only its pointer. If an array is exhausted, copy the remaining elements of the other array.

Algorithm 1.2: Merge two sorted arrays

Input: Increasing arrays $A[1 : p]$ and $B[1 : q]$, with distinct entries overall.
Output: An increasing array C containing every input entry once.

```
i ← 1; j ← 1; C ← an empty array
while i ≤ p and j ≤ q do
    if A[i] < B[j] then
        Append A[i] to C; i ← i + 1
    else
        Append B[j] to C; j ← j + 1
Append the remaining suffix A[i : p], if nonempty, to C
Append the remaining suffix B[j : q], if nonempty, to C
return C
```

Lemma 1.2 [Correctness of merging]. *If A and B are sorted, the merge procedure returns their entries in increasing order, with no loss or repetition.*

Proof. The idea is to maintain that the output contains exactly the smallest entries processed so far. Initially it is empty. While both arrays have unused entries, the smaller front element is the smallest unused element of their union, because neither sorted suffix contains a smaller element behind its front. Appending that element preserves the ordering and moves exactly one pointer. When one array is exhausted, the remaining sorted suffix contains all remaining elements, each at least as large as the last element appended. Copying it finishes the output. Every element is copied once, so the result has the required order and contents. □

Proposition 1.3 [Correctness of merge sort]. *Merge sort returns the input entries in increasing order.*

Proof. Use induction on the array length. Lengths zero and one are base cases. For $n \geq 2$, each half is shorter than the input, so the induction hypothesis guarantees that the recursive calls sort both halves correctly. The merge lemma then orders their union and preserves all entries. This proves the claim for length n. □

1.5 Counting work in one merge

Let $m = p + q$ be the total number of input entries. Each iteration copies one entry, and the suffix copies account for exactly the remaining entries. Thus merging takes work proportional to m.

For a coarse operation count, use the simplified loop accounting: two pointer initializations and at most four elementary steps for each output position. This gives

$$\text{MergeCost}(m) \leq 4m + 2 \leq 6m \qquad (m \geq 1).$$

The constant depends on which bookkeeping steps are charged. Completing the end cases in actual code can change it, but does not change the linear growth. This bound is a mathematical accounting model, not a count of machine instructions.

1.6 The recursion-tree analysis

Each merge is linear in its own input size, but there are many merges. The useful observation is that the total input size across any one level of recursion is n.

Assume first that $n = 2^h$, so all splits are even. A recursion tree has one node per recursive call. At depth j there are 2^j subproblems, each of size $n/2^j$. The leaves have size one, so they occur at depth $h = \log_2 n$. There are $h + 1$ levels, counting the root and the leaves.

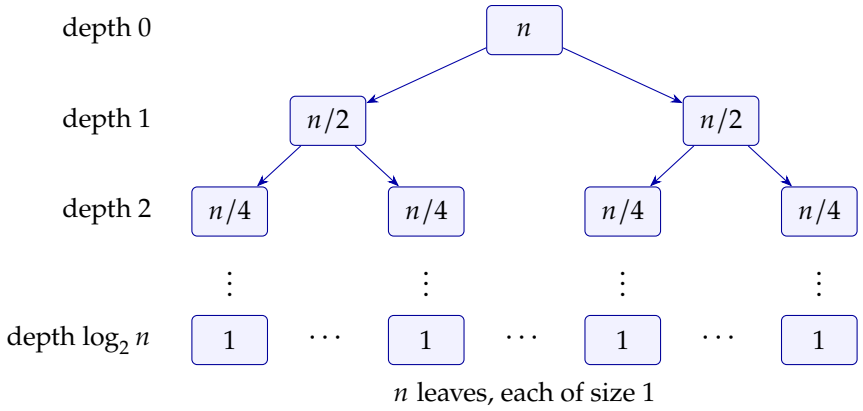


Figure 2: Schematic recursion tree for n a power of two. The number of subproblems doubles as their sizes halve. Intermediate levels are omitted.

Theorem 1.4 [Merge-sort running time]. *In the coarse operation model above, merge sort on $n = 2^h \geq 1$ entries uses at most $6n \log_2 n + 6n$ charged operations, including a constant charge for each leaf.*

Proof. Bound the work one level at a time. Charge at most $6m$ for a node of size m, with leaf work bounded by 6. The charge at depth j is at most

$$\underbrace{2^j}_{\text{number of nodes}} \quad \underbrace{6\left(\frac{n}{2^j}\right)}_{\text{work per node}} = 6n.$$

There are $\log_2 n + 1$ levels. Summing their charges gives

$$\sum_{j=0}^{\log_2 n} 6n = 6n(\log_2 n + 1) = 6n \log_2 n + 6n.$$

The leaf charge is included, even though a leaf performs no merge. □

For a general positive n, balanced splitting has at most $\lceil \log_2 n \rceil + 1$ levels. Nodes at any fixed depth cover disjoint portions of the input, whose total size is at most n. The same reasoning gives a bound proportional to $n \log n$. The power-of-two assumption makes the displayed level counts exact; it is not required by the algorithm.

1.7 Three principles for algorithm analysis

1. Worst-case analysis gives a running-time bound for every input of a given size. It is useful for a general-purpose routine whose future inputs are unknown. An average-case guarantee needs a model for which inputs are likely; a benchmark measures selected instances.

2. Suppress constant factors and lower-order terms when comparing growth rates. Constants

depend on implementation and computing hardware; retaining all of them can obscure the main design difference.

3. Asymptotic analysis studies how the cost grows as the input becomes large. A smaller growth rate need not give a smaller cost on every small input.

For example, compare the illustrative costs

$$f(n) = 6n \log_2 n + 6n, \qquad g(n) = \tfrac{1}{2}n^2.$$

The linearithmic expression eventually becomes much smaller, although the quadratic expression is smaller over part of the range of small n. These are cost formulas, not measured runtimes.

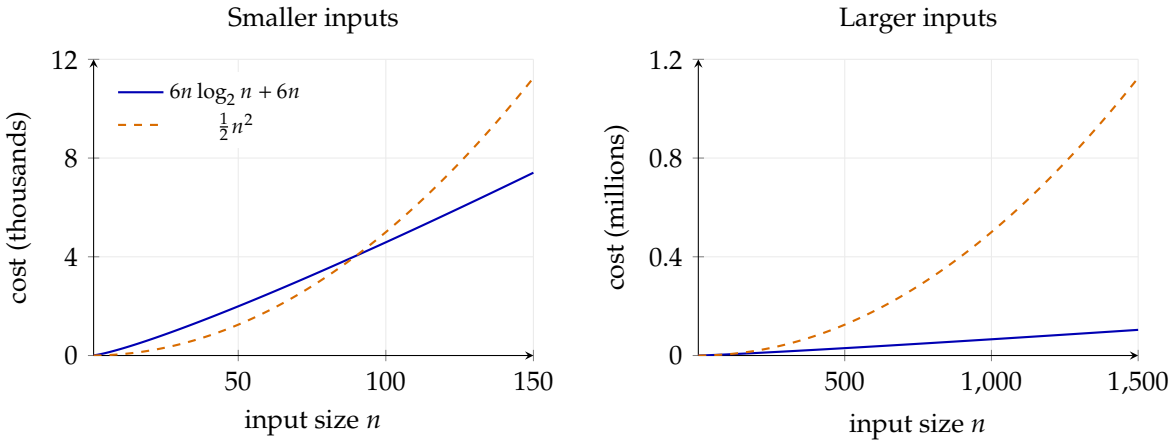


Figure 3: The supplied illustrative cost functions at two input scales. The relative advantage of $6n \log_2 n + 6n$ grows with the input size. The vertical axes use different units.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Introduction.

Lecture 2: Asymptotic Analysis

Asymptotic analysis compares operation counts as input size grows, suppressing constant factors and lower-order terms.

For example, a bound such as $6n \log_2 n + 6$ has the growth rate $n \log n$. The notation $O(n \log n)$ makes this comparison precise. Here n denotes the input size, such as the length of an array, and $T(n)$ usually denotes the worst-case running time over inputs of that size.

2.1 Counting loops before simplifying

Count each array comparison and loop update as constant work. First identify how many times that work is performed. A return statement can end an algorithm early, but a worst-case bound must also cover inputs on which no early return occurs.

Algorithm 2.1: Search one array

Input: Array $A[1 : n]$ and target t.
Output: Whether A contains t.

```
for i ← 1 to n do
    if A[i] = t then
        return true
return false
```

If t is absent, all n entries are compared with it. There is constant work per iteration, so the worst-case running time grows linearly with n. Finding t at the first position is a faster input, but does not reduce the worst-case bound.

Algorithm 2.2: Search two arrays in succession

Input: Arrays $A[1 : n]$, $B[1 : n]$, and target t.
Output: Whether either array contains t.

```
for i ← 1 to n do
    if A[i] = t then
        return true
for i ← 1 to n do
    if B[i] = t then
        return true
return false
```

When t is absent from both arrays, there are $n + n = 2n$ comparisons. The loops run in succession, so their work adds. Suppressing the factor 2 leaves the same linear growth rate as the first search.

Algorithm 2.3: Check two arrays for a common entry

```
Input: Arrays A[1 : n] and B[1 : n].
Output: Whether the arrays share an entry.
for i ← 1 to n do
    for j ← 1 to n do
        if A[i] = B[j] then
            return true
return false
```

If the arrays have no common entry, each of the n outer iterations executes all n inner iterations. There are n^2 comparisons, giving quadratic worst-case growth.

Algorithm 2.4: Check an array for duplicates

```
Input: Array A[1 : n].
Output: Whether two different positions have the same entry.
for i ← 1 to n do
    for j ← i + 1 to n do
        if A[i] = A[j] then
            return true
return false
```

Starting the inner loop at $i + 1$ avoids comparing an entry with itself or checking a pair twice. If all entries are distinct, the comparison count is

$$\sum_{i=1}^{n}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

The leading term is $n^2/2$, so the growth is still quadratic. A changing inner-loop length requires summing the iteration counts; it does not by itself imply a faster asymptotic bound.

2.2 Big-O: an eventual upper bound

Let $T(n) \geq 0$ be a cost function, and let $f(n) > 0$ for sufficiently large positive integers n. We want to express that T eventually stays below a fixed multiple of f, even if the two functions behave differently on small inputs.

Definition 2.1 [Big-O]. Write $T(n) = O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic upper bound.

The fixed multiplier c absorbs constant factors, and n_0 excludes small inputs. Neither depends on n; the bound holds for every integer $n \geq n_0$.

An upper bound need not be tight: a linear running time is also $O(n^2)$. When comparing the loop examples, $O(n)$ and $O(n^2)$ describe their respective linear and quadratic growth, rather than the only upper bounds they satisfy.

2.3 Polynomial examples and fixed witnesses

Proposition 2.2 [Polynomial upper bound]. *For fixed real coefficients $a_0, \dots, a_k$ and an integer $k \geq 0$, let $p(n) = a_k n^k + \cdots + a_1 n + a_0$. Then $|p(n)| = O(n^k)$.*

Proof. The idea is to bound every lower power by the highest power. For $n \geq 1$ and $0 \leq i \leq k$, we have $n^i \leq n^k$. Hence

$$|p(n)| \leq \sum_{i=0}^{k} |a_i| n^i \leq \left(\sum_{i=0}^{k} |a_i| \right) n^k.$$

Choose $n_0 = 1$ and $c = \max\{1, \sum_{i=0}^{k} |a_i|\}$. These are fixed witnesses for the big-O definition. In particular, the bound applies to a nonnegative polynomial running time. □

Proposition 2.3 [Different polynomial degrees]. *For every integer $k \geq 1$, n^k is not $O(n^{k-1})$.*

Proof. The idea is that one uncancelled factor of n eventually exceeds every fixed constant. If $n^k = O(n^{k-1})$, there would be constants $c, n_0 > 0$ with $n^k \leq c n^{k-1}$ for all integers $n \geq n_0$. Dividing by the positive quantity n^{k-1} gives $n \leq c$ for all such n. Choose an integer $n > \max\{n_0, c\}$. This contradicts the required inequality. □

Prove an upper bound with fixed witnesses; disprove one by defeating every proposed pair on a sufficiently large input.

2.4 Omega and Theta: lower and matching bounds

Big-O bounds a function from above. A corresponding lower bound says that the cost eventually remains at least a fixed multiple of another function.

Definition 2.4 [Big-Omega]. Write $T(n) = \Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$T(n) \geq c f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic lower bound.

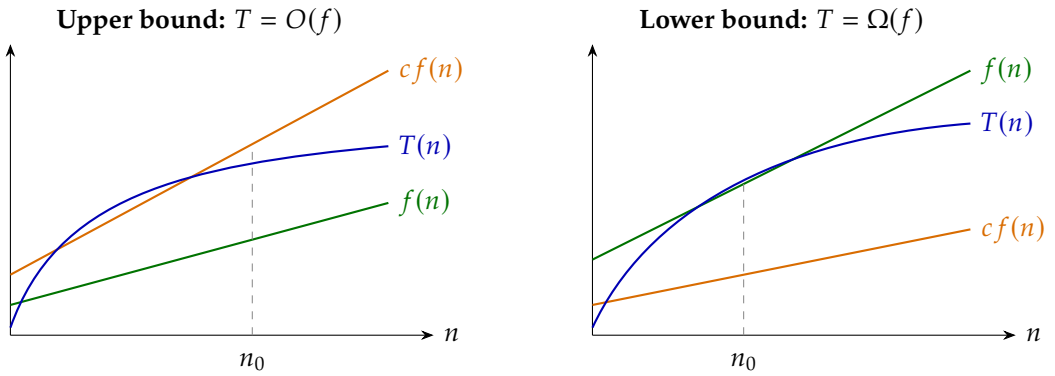


Figure 4: Schematic eventual bounds. For big-O, T lies below cf after n_0; for big-Omega, it lies above cf. The unscaled function f need not lie on the same side of T as its scaled version.

Definition 2.5 [Big-Theta]. Write $T(n) = \Theta(f(n))$ if both $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that

$$c_1 f(n) \leq T(n) \leq c_2 f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotically tight bound.

To obtain one threshold from separate upper and lower bounds, take the larger of their two thresholds. Then both inequalities hold simultaneously. Conversely, the two-sided inequality immediately supplies both bounds.

These symbols describe inequalities between functions. They do not select which inputs are considered: if $T(n)$ is a worst-case running time, then $\Omega(f(n))$ is a lower bound on that same worst-case function.

Example: Consider $T(n) = \frac{1}{2}n^2 + 3n$. For every integer $n \geq 1$,

$$\tfrac{1}{2}n^2 \leq T(n) \leq \tfrac{1}{2}n^2 + 3n^2 \leq 4n^2.$$

Thus $T(n) = \Theta(n^2)$, with $c_1 = 1/2$, $c_2 = 4$, and $n_0 = 1$. The other comparisons follow directly:

Claim	True?	Reason for $n \geq 1$
$T(n) = O(n)$	No	$T(n)/n = \frac{1}{2}n + 3$ is unbounded.
$T(n) = \Omega(n)$	Yes	$T(n) \geq \frac{1}{2}n^2 \geq \frac{1}{2}n$.
$T(n) = \Theta(n^2)$	Yes	$\frac{1}{2}n^2 \leq T(n) \leq 4n^2$.
$T(n) = O(n^3)$	Yes	$T(n) \leq 4n^2 \leq 4n^3$.

The cubic upper bound is valid, but the quadratic Theta bound describes the growth more precisely. Similarly, the four loop examples have tight worst-case bounds $\Theta(n)$, $\Theta(n)$, $\Theta(n^2)$, and $\Theta(n^2)$, respectively.

2.5 Little-o and the order of the quantifiers

A fixed upper multiplier allows two functions to grow at the same rate. To say that T eventually becomes smaller than *every* positive multiple of f, use a stronger comparison.

Definition 2.6 [Little-o]. Write $T(n) = o(f(n))$ if for every constant $c > 0$, there exists a threshold $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

For big-O, one suitable multiplier is enough. For little-o, the inequality must eventually work no matter how small the positive multiplier is. Here n_0 may depend on the chosen c, but not on the running variable n.

Proposition 2.7. *For every integer $k \geq 1$, $n^{k-1} = o(n^k)$.*

Proof. The idea is to make the ratio $1/n$ smaller than any given constant. Fix $c > 0$, and choose $n_0 = \max\{1, \lceil 1/c \rceil\}$. Then every integer $n \geq n_0$ satisfies $1/n \leq c$, so

$$n^{k-1} \leq cn^k.$$

Since this works for every positive c, the little-o definition holds. □

2.6 Constants inside an exponent

A constant added to an exponent behaves differently from a constant multiplied by the input size in that exponent.

Proposition 2.8. *$2^{n+10} = O(2^n)$, but 2^{10n} is not $O(2^n)$.*

Proof. Compare each expression with 2^n by division. First,

$$2^{n+10} = 2^{10}2^n = 1024 \cdot 2^n,$$

so $c = 1024$ and $n_0 = 1$ give the required upper bound.
If $2^{10n} = O(2^n)$, there would be fixed constants $c, n_0 > 0$ with $2^{10n} \leq c2^n$ for all integers $n \geq n_0$. Dividing by 2^n yields $2^{9n} \leq c$. Choose an integer $n > \max\{n_0, (\log_2 c)/9\}$. Then $2^{9n} > c$, a contradiction. □

The factor 10 in $10n$ cannot be discarded: after division, it leaves a ratio that grows without bound. The fixed additive shift +10 leaves only a constant ratio.

2.7 A sum and a maximum have the same growth

The larger of two nonnegative quantities controls their sum up to a factor of two. The identity is useful even when the two functions cross repeatedly, so neither is always the larger one.

Proposition 2.9 [Maximum and sum]. *For positive functions f and g on the positive integers,*

$$\max\{f(n), g(n)\} = \Theta(f(n) + g(n)).$$

Proof. Use pointwise inequalities, so no assumption about which function grows faster is needed. Each function is at most the maximum, and both are nonnegative. Therefore, for every $n \geq 1$,

$$\max\{f(n), g(n)\} \leq f(n) + g(n) \leq 2\max\{f(n), g(n)\}.$$

Rearranging gives

$$\tfrac{1}{2}\big(f(n) + g(n)\big) \leq \max\{f(n), g(n)\} \leq f(n) + g(n).$$

These are the Theta inequalities with $c_1 = 1/2$, $c_2 = 1$, and $n_0 = 1$. □

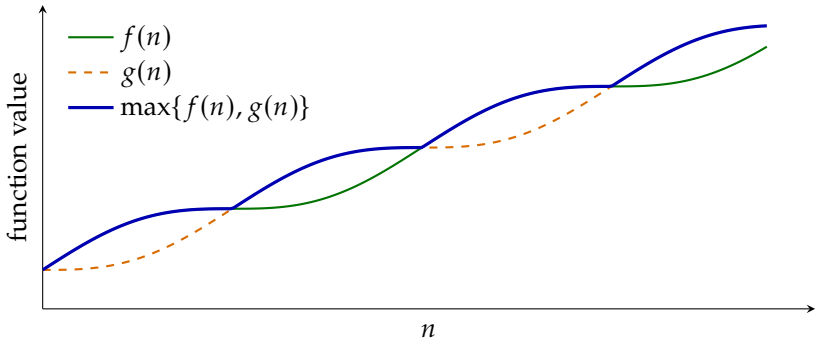


Figure 5: Schematic crossing functions. Their pointwise maximum follows whichever curve is higher. Their sum remains between this maximum and twice the maximum, including where the curves switch order.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Asymptotic Analysis.

Lecture 3: Divide and Conquer Algorithms

The divide and conquer paradigm has three steps: *divide* the input into smaller subproblems, *conquer* the subproblems recursively, and *combine* their solutions into one for the original problem. Three applications show that the combine step is usually where the cleverness lies.

3.1 Counting inversions

Definition 3.1 [Inversion]. Let $A[1:n]$ contain the numbers $1, 2, \ldots, n$ in some order. An inversion is a pair of indices (i, j) with $i < j$ and $A[i] > A[j]$. The task is to count the inversions of A.

For $A = (1, 3, 5, 2, 4, 6)$ the inversions, written as pairs of values, are $(3, 2)$, $(5, 2)$, and $(5, 4)$. Connect each value in sorted order to its position in A: two segments cross exactly when their values form an inversion. The count is a numerical measure of how similar two rankings are, which is useful, for example, in collaborative filtering.

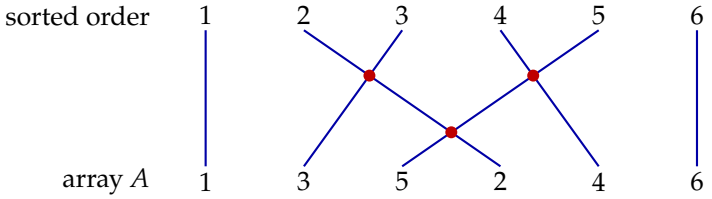


Figure 6: The inversions of $A = (1, 3, 5, 2, 4, 6)$ as crossings between sorted order and array order. The three marked crossings are the pairs $(3, 2)$, $(5, 2)$, and $(5, 4)$.

There are $\binom{n}{2} = n(n-1)/2$ pairs of indices, and the reversed array makes every pair an inversion. So a 6-element array has at most 15 inversions, and checking every pair takes $\Theta(n^2)$ time.

To do better, classify an inversion (i, j) with $i < j$ as

- left if $i, j \le n/2$ and right if $i, j > n/2$; these are inversions of one half and can be counted recursively;
- split if $i \le n/2 < j$; these need a separate subroutine.

If split inversions can be counted in $O(n)$ time, the recursion has the shape of merge sort, $T(n) \le 2T(n/2) + O(n)$, and runs in $O(n \log n)$ time.

3.2 Piggybacking on merge sort

The second key idea is to make each recursive call both count the inversions of its subarray *and* sort it. The merge step then uncovers the split inversions.

If A had no split inversions, every element of the sorted left half B would be smaller than every element of the sorted right half C, and merging would copy all of B before any of C. In general, consider merging $B = (1, 3, 5)$ and $C = (2, 4, 6)$. When 2 is copied to the output, the elements 3 and 5 remain in B; these give the split inversions $(3, 2)$ and $(5, 2)$. When 4 is copied, 5 remains, giving $(5, 4)$.

Lemma 3.2 [Split inversions during merging]. *Let B and C be the sorted first and second halves of A. The split inversions involving an element y of C are exactly the pairs (x, y) with x still remaining in B when y is copied to the output.*

Proof. Let x be an element of B. Since x lies in the first half and y in the second, (x, y) is a split inversion exactly when $x > y$. Merging always copies the smaller front element. If x is copied before y, then $x < y$, so they form no inversion. If y is copied while x remains in B, then $y < x$, so they form a split inversion. □

So the merge keeps a running total: whenever an element of C is copied, add the number of elements remaining in B. The extra work is $O(1)$ per copied element.

Algorithm 3.1: Sort and count inversions

Input: Array $A[1 : n]$ of distinct numbers.
Output: The sorted array and the number of inversions of A.

```
if n ≤ 1 then
    return (A, 0)
m ← ⌊n/2⌋
(B, X) ← SortAndCount(A[1 : m])
(C, Y) ← SortAndCount(A[m + 1 : n])
(D, Z) ← MergeAndCountSplitInv(B, C)
return (D, X + Y + Z)
```

Algorithm 3.2: Merge and count split inversions

Input: Sorted arrays $B[1 : p]$ and $C[1 : q]$.
Output: Their merge D and the number Z of pairs (x, y), $x \in B$, $y \in C$, with $x > y$.

```
i ← 1; j ← 1; Z ← 0; D ← an empty array
for k ← 1 to p + q do
    if j > q or (i ≤ p and B[i] < C[j]) then
        Append B[i] to D; i ← i + 1
    else
        Append C[j] to D; Z ← Z + (p − i + 1); j ← j + 1
return (D, Z)
```

Here $p - i + 1$ is the number of elements left in B, so by lemma 3.2, Z is the number of split inversions. Merging and maintaining the total each take $O(n)$ time, so SortAndCount satisfies the merge-sort recurrence and runs in $O(n \log n)$ time.

3.3 Matrix multiplication and Strassen's algorithm

For $n \times n$ matrices X and Y, the product $Z = XY$ has entries

$$z_{ij} = (\text{row } i \text{ of } X) \cdot (\text{column } j \text{ of } Y) = \sum_{k=1}^{n} X_{ik} Y_{kj}.$$

The input size is $\Theta(n^2)$. The straightforward iterative algorithm computes n^2 entries, each in $\Theta(n)$ time, for $\Theta(n^3)$ in total. For $n = 2$,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}.$$

The same formula holds for blocks. Assume n is a power of 2 and write

$$X = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \qquad Y = \begin{pmatrix} E & F \\ G & H \end{pmatrix}, \qquad XY = \begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix},$$

where A through H are $n/2 \times n/2$ matrices. The first recursive algorithm computes the eight block products recursively, then adds them in $\Theta(n^2)$ time. Its running time is still $\Theta(n^3)$, which follows from the master method of Lecture 4.

Strassen's algorithm (1969) recursively computes only seven cleverly chosen products,

$$\begin{aligned} P_1 &= A(F - H), & P_2 &= (A + B)H, & P_3 &= (C + D)E, & P_4 &= D(G - E), \\ P_5 &= (A + D)(E + H), & P_6 &= (B - D)(G + H), & P_7 &= (A - C)(E + F), \end{aligned}$$

and recovers the product with $\Theta(n^2)$ additions and subtractions. Replacing eight recursive products with seven makes the running time better than cubic; Lecture 4 shows it is $O(n^{\log_2 7})$. How one would discover these seven products is left open.

Proposition 3.3 [Strassen's identities].

$$XY = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{pmatrix}.$$

Proof. Expand each block, keeping the order of every matrix product, since matrix multiplication is not commutative. For the top-left block,

$$P_5 + P_4 - P_2 + P_6 = (AE + AH + DE + DH) + (DG - DE) - (AH + BH) \\ + (BG + BH - DG - DH) = AE + BG.$$

Similarly, $P_1 + P_2 = AF - AH + AH + BH = AF + BH$, $P_3 + P_4 = CE + DE + DG - DE = CE + DG$, and

$$P_1 + P_5 - P_3 - P_7 = (AF - AH) + (AE + AH + DE + DH) \\ - (CE + DE) - (AE + AF - CE - CF) = CF + DH.$$

□

3.4 The closest pair problem

Definition 3.4 [Closest pair]. Given a set $P = \{p_1, \ldots, p_n\}$ of $n \geq 2$ points in the plane, with Euclidean distance

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad \text{for } p_i = (x_i, y_i),\ p_j = (x_j, y_j),$$

find a pair of distinct points $p^*, q^* \in P$ minimizing $d(p, q)$.

For convenience, assume that all points have distinct x-coordinates and distinct y-coordinates. Brute-force search takes $\Theta(n^2)$ time. In one dimension, sorting in $O(n \log n)$ time and comparing adjacent points in $O(n)$ time suffices. The goal is an $O(n \log n)$ algorithm in two dimensions.

Sorting is still the first step: make copies P_x and P_y of the points sorted by x- and by y-coordinate, in $O(n \log n)$ time. This is not enough, because the closest pair need not be adjacent in either order. Divide and conquer provides the rest.

Algorithm 3.3: Closest pair

Input: Copies P_x, P_y of a point set P, $|P| = n \geq 2$, sorted by x- and by y-coordinate.
Output: A closest pair of points of P.

```
if n ≤ 3 then
    return a closest pair found by brute force
Q ← first ⌈n/2⌉ points of P_x (left half); R ← the remaining points
Form Q_x, R_x by splitting P_x, and Q_y, R_y by one scan of P_y
(p_1, q_1) ← ClosestPair(Q_x, Q_y)
(p_2, q_2) ← ClosestPair(R_x, R_y)
δ ← min{d(p_1, q_1), d(p_2, q_2)}
(p_3, q_3) ← ClosestSplitPair(P_x, P_y, δ)
return the best of (p_1, q_1), (p_2, q_2), and (p_3, q_3), if found
```

The scan of P_y sends each point to Q_y or R_y according to its x-coordinate, so all four lists take $O(n)$ time. A split pair has one point in Q and one in R. The key idea is that a split pair matters only

in the "unlucky case" where it is closer than the best pairs (p_1, q_1) and (p_2, q_2) returned by the recursive calls. Therefore ClosestSplitPair must only

1. run in $O(n)$ time, and
2. be correct whenever the closest pair of P is a split pair at distance less than δ.

Then $T(n) \le 2T(n/2) + O(n)$, as for merge sort, and the whole algorithm runs in $O(n \log n)$ time, including the initial sorting.

Algorithm 3.4: Closest split pair

Input: P_x, P_y, and $\delta > 0$ as above.
Output: The closest split pair at distance less than δ, if one exists.

$\bar{x} \leftarrow$ largest x-coordinate in the left half Q (the last point of Q_x)
$S_y \leftarrow$ points of P_y with x-coordinate in $[\bar{x} - \delta, \bar{x} + \delta]$, sorted by y-coordinate
best $\leftarrow \delta$; *bestpair* $\leftarrow$ none
for $i \leftarrow 1$ **to** $|S_y| - 1$ **do**
    for $j \leftarrow 1$ **to** $\min\{7, |S_y| - i\}$ **do**
        $p \leftarrow S_y[i]$; $q \leftarrow S_y[i + j]$
        if $d(p, q) <$ *best* **then**
            bestpair $\leftarrow (p, q)$; *best* $\leftarrow d(p, q)$
return *bestpair*

Finding $\bar{x}$ takes $O(1)$ time and filtering P_y keeps its y-order in $O(n)$ time. Each point is compared with at most the next seven points of S_y, so the loops take $O(n)$ time.

3.5 Correctness of the split-pair search

Theorem 3.5 [Correctness claim]. *Let $p \in Q$ and $q \in R$ be a split pair with $d(p, q) < \delta$. Then*

(A) p and q are members of S_y;

(B) p and q are at most 7 positions apart in S_y.

Proof of (A). Write $p = (x_1, y_1)$ and $q = (x_2, y_2)$. Since $d(p, q) < \delta$, both $|x_1 - x_2| < \delta$ and $|y_1 - y_2| < \delta$. Because $p \in Q$ and $q \in R$, we have $x_1 \le \bar{x} \le x_2$. Hence

$$\bar{x} - \delta \le x_2 - \delta < x_1 \le \bar{x} \qquad \text{and} \qquad \bar{x} \le x_2 < x_1 + \delta \le \bar{x} + \delta,$$

so both x-coordinates lie in $[\bar{x} - \delta, \bar{x} + \delta]$. □

For (B), draw eight $\delta/2 \times \delta/2$ boxes in two rows of four, centered horizontally at $\bar{x}$ and with bottom edge at height $\min\{y_1, y_2\}$. Assign shared box boundaries so that each point lies in one box, and each box lies on one side of the line $x = \bar{x}$.

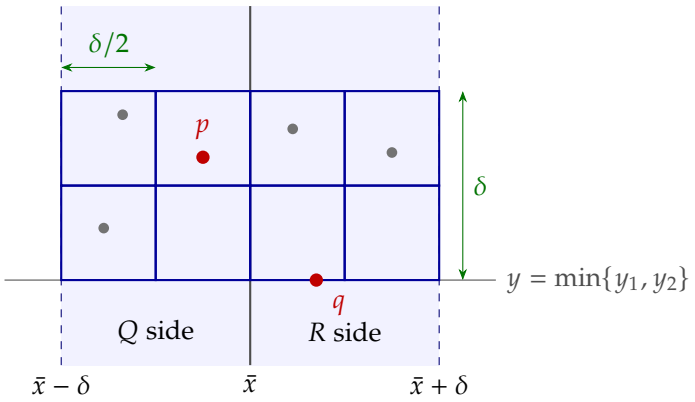


Figure 7: Schematic of the eight boxes in the strip that defines S_y. Every point of S_y with y-coordinate between those of p and q lies in a box, and no box contains two points.

Lemma 3.6. *Every point of S_y whose y-coordinate is between those of p and q, inclusive, lies in one of the eight boxes.*

Proof. The y-coordinates of p and q differ by less than δ, so such a point has height between $\min\{y_1, y_2\}$ and $\min\{y_1, y_2\} + \delta$. By the definition of S_y, its x-coordinate is between $\bar{x} - \delta$ and $\bar{x} + \delta$. These two ranges are exactly covered by the boxes. □

Lemma 3.7. *Each box contains at most one point of P.*

Proof. Suppose distinct points a, b lie in the same box. The box is on one side of $x = \bar{x}$, so a and b are both in Q or both in R. Its diagonal has length $\sqrt{2}\,\delta/2$, so

$$d(a, b) \leq \frac{\delta}{2}\sqrt{2} < \delta.$$

This contradicts the definition of δ as the smallest distance between a pair of points both in Q or both in R. □

Proof of (B). By the two lemmas, at most eight points of S_y, including p and q, have y-coordinates between those of p and q. Since S_y is sorted by y-coordinate, these are exactly the points from p to q in S_y. Hence the positions of p and q differ by at most 7. □

Corollary 3.8. *If the closest pair of P is a split pair,* ClosestSplitPair *finds it.*

Proof. If its distance is less than δ, theorem 3.5 places both points in S_y at most 7 positions apart, and the loops compare every such pair. If its distance equals δ, a recursive call already returned a pair at the minimum distance. □

Hence ClosestPair is correct: if the closest pair lies within Q or within R, a recursive call finds it;

otherwise it is a split pair, found by the split-pair search. Since every step outside the recursive calls takes $O(n)$ time, it runs in $O(n \log n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Divide and Conquer Algorithms.

Lecture 4: The Master Method

Promising algorithmic ideas often need a mathematical analysis before we can tell whether they help. The master method is a "black box" that bounds the running times of many divide and conquer algorithms directly from their recurrences.

4.1 Recurrences for integer multiplication

The grade-school algorithm uses $\Theta(n^2)$ operations to multiply two n-digit numbers. For a recursive approach, write

$$x = 10^{n/2}a + b, \qquad y = 10^{n/2}c + d,$$

where a, b, c, d are $n/2$-digit numbers. Then

$$xy = 10^n ac + 10^{n/2}(ad + bc) + bd. \tag{4.1}$$

Let $T(n)$ be the maximum number of operations an algorithm needs to multiply two n-digit numbers. A recurrence expresses $T(n)$ in terms of the running times of the recursive calls.

- *Algorithm 1* recursively computes ac, ad, bc, bd, then evaluates equation (4.1) directly. With $T(1)$ at most a constant, for all $n > 1$,

$$T(n) \leq \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{O(n)}_{\text{work done here}}.$$

- *Algorithm 2* (Gauss's trick, the method of Lecture 1) recursively computes only ac, bd, and $(a + b)(c + d)$, and recovers $ad + bc = (a + b)(c + d) - ac - bd$. Its recurrence is

$$T(n) \leq 3T(n/2) + O(n).$$

4.2 The precise statement

Assume that all subproblems have equal size, and that the recurrence has the following form.

1. Base case: $T(n)$ is at most a constant for all sufficiently small n.
2. For all larger n,

$$T(n) \leq aT(n/b) + O(n^d),$$

where $a \geq 1$ is the number of recursive calls, $b > 1$ is the factor by which the input size shrinks,

and $d \geq 0$ is the exponent in the running time of the combine step. The parameters a, b, d do not depend on n.

Theorem 4.1 [Master method]. *For a recurrence of this form,*

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad \text{(Case 1)}, \\ O(n^d) & \text{if } a < b^d \quad \text{(Case 2)}, \\ O(n^{\log_b a}) & \text{if } a > b^d \quad \text{(Case 3)}. \end{cases}$$

In Case 1 the base of the logarithm does not matter, since changing it only changes the leading constant. In Case 3 the base b appears in the exponent and does matter.

4.3 Examples

Recurrence	a	b	d	Case	Bound
Merge sort	2	2	1	1	$O(n \log n)$
Binary search of a sorted array	1	2	0	1	$O(\log n)$
Integer multiplication, Algorithm 1	4	2	1	3	$O(n^{\log_2 4}) = O(n^2)$
Integer multiplication, Gauss	3	2	1	3	$O(n^{\log_2 3}) = O(n^{1.59})$
Strassen's matrix multiplication	7	2	2	3	$O(n^{\log_2 7}) = O(n^{2.81})$
$T(n) \leq 2T(n/2) + O(n^2)$	2	2	2	2	$O(n^2)$

For merge sort, $b^d = 2 = a$. Binary search makes one recursive call on half the array and does constant extra work, so $b^d = 1 = a$. The first integer multiplication algorithm has $b^d = 2 < 4 = a$ and is no better than the grade-school algorithm. Gauss's trick has $b^d = 2 < 3 = a$ and beats it. Strassen's algorithm has $b^d = 4 < 7 = a$ and beats the naive iterative algorithm for matrix multiplication. In the last, fictitious recurrence, $b^d = 4 > 2 = a$. The exponents 1.59 and 2.81 are rounded values of $\log_2 3$ and $\log_2 7$.

4.4 The recursion tree

To prove the theorem, generalize the recursion-tree analysis of merge sort. Assume $T(1) \leq c$ and $T(n) \leq aT(n/b) + cn^d$ for one constant c, and that n is a power of b. The general case is similar but more tedious.

The tree has levels $j = 0, 1, \ldots, \log_b n$, where $\log_b n$ is the number of times n can be divided by b before reaching 1. Each node has a children, so level j has a^j subproblems, each of size n/b^j. The leaves are the base cases of size 1.

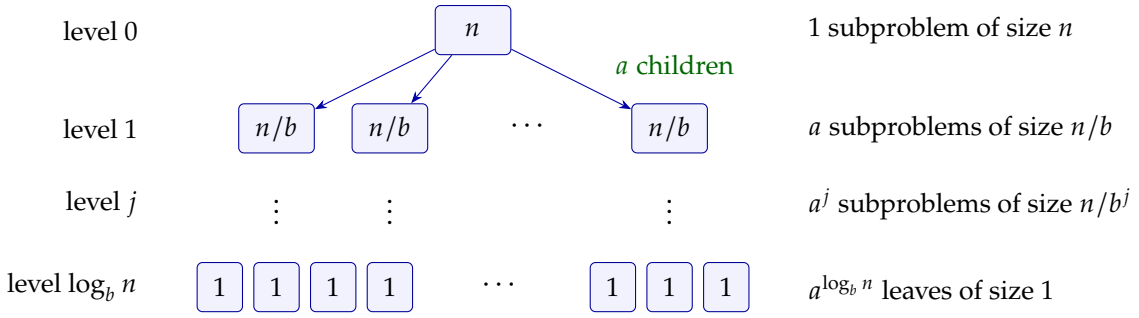


Figure 8: Schematic recursion tree for $T(n) \le aT(n/b) + cn^d$ with n a power of b. Only three of the a children of the root are drawn.

Ignoring work done inside recursive calls, the work at level j is at most

$$\underbrace{a^j}_{\substack{\text{number of} \\ \text{subproblems}}} \cdot c\left(\frac{n}{b^j}\right)^d = cn^d\left(\frac{a}{b^d}\right)^j.$$

At the leaves this is c per subproblem, matching the base case. Summing over all levels,

$$\text{total work} \le cn^d \sum_{j=0}^{\log_b n} \left(\frac{a}{b^d}\right)^j. \tag{4.2}$$

4.5 Intuition for the three cases

Read equation (4.2) as a contest between two forces: a is the RATE OF SUBPROBLEM PROLIFERATION (RSP), and b^d is the RATE OF WORK SHRINKAGE (RWS) per subproblem.

1. If RSP = RWS, every level does the same amount of work, as in merge sort. Expect $O(n^d \log n)$.
2. If RSP < RWS, the work decreases with the level, so most work is at the root. Expect $O(n^d)$.
3. If RSP > RWS, the work increases with the level, so most work is at the leaves. Expect $O(\#\text{leaves})$.

4.6 Completing the proof

Lemma 4.2 [BASIC SUMS FACT]. *For $r \neq 1$ and every integer $k \ge 0$,*

$$1 + r + r^2 + \cdots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

Proof. Induct on k. The case $k = 0$ is $1 = (r-1)/(r-1)$. If the identity holds for k, adding r^{k+1} gives $\left(r^{k+1} - 1 + r^{k+2} - r^{k+1}\right)/(r-1) = (r^{k+2} - 1)/(r-1)$. □

Two consequences hold for a constant r, uniformly in k:

- if $r < 1$, the sum is at most $1/(1-r)$, a constant, so the first term dominates;
- if $r > 1$, the sum is at most $r^{k+1}/(r-1) = r^k\left(1 + \frac{1}{r-1}\right)$, a constant times the last term, which dominates.

Proof of Theorem 4.1. Let $r = a/b^d$ in equation (4.2).
Case 1 ($a = b^d$). Every term of the sum equals 1, so the total work is at most $cn^d(\log_b n + 1) = O(n^d \log n)$.
Case 2 ($a < b^d$). Then $r < 1$, and by lemma 4.2 the sum is at most a constant independent of n. The total work is $O(n^d)$, dominated by the top level.
Case 3 ($a > b^d$). Then $r > 1$, and the sum is at most a constant times its largest term. So the total work is

$$O\left(n^d \left(\frac{a}{b^d}\right)^{\log_b n}\right) = O\left(a^{\log_b n}\right), \qquad \text{since } b^{-d \log_b n} = \left(b^{\log_b n}\right)^{-d} = n^{-d}.$$

Here $a^{\log_b n}$ is the number of leaves of the recursion tree. Finally, $a^{\log_b n} = n^{\log_b a}$, since both sides have logarithm base b equal to $(\log_b n)(\log_b a)$. The left form is more intuitive; the right form is simpler to apply. Thus $T(n) = O(n^{\log_b a})$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), The Master Method.

Lecture 5: QuickSort: Algorithm

QuickSort sorts by partitioning an array around one of its elements and recursing on the two sides. With random pivots it runs in $O(n \log n)$ time on average and works in place, using only a constant amount of extra memory beyond the input array.

The input is an array of n numbers, and the output is the same numbers in increasing order. Assume throughout that all entries are distinct.

5.1 Partitioning around a pivot

Definition 5.1 [Partition]. Given an array and one of its elements p, the pivot, a partition around p rearranges the array so that every entry left of p is less than p and every entry right of p is greater than p.

The two sides need not be sorted. For example, partitioning $3, 8, 2, 5, 1, 4, 7, 6$ around 3 may produce $2, 1, 3, 6, 7, 4, 5, 8$. Exactly the entries smaller than p precede it, so the pivot ends in its rightful position: the position it occupies in the sorted output.

Partitioning has two useful properties. It takes linear time with no extra memory, as shown below, and it reduces the problem size: the entries on each side can be sorted independently, and neither side contains the pivot.

With $O(n)$ extra memory, partitioning in $O(n)$ time is easy. Scan the input once, writing each entry less than p into a new array from the left end and each entry greater than p from the right end, then place p in the one remaining slot.

Algorithm 5.1: QuickSort, high-level description

Input: Array A of length n, with distinct entries.
Output: A rearranged into increasing order.

```
if n ≤ 1 then
    return
p ← ChoosePivot(A, n)
Partition A around p
Recursively sort the first part, the entries left of p
Recursively sort the second part, the entries right of p
```

The rule ChoosePivot is left open for now. Either part may be empty, so the base case includes $n = 0$.

5.2 In-place partitioning

Assume the pivot is the first entry of the subarray. Otherwise, first swap the pivot with the first entry. A single left-to-right scan then keeps the scanned entries partitioned: the pivot, then entries less than p, then entries greater than p, followed by the unscanned entries.

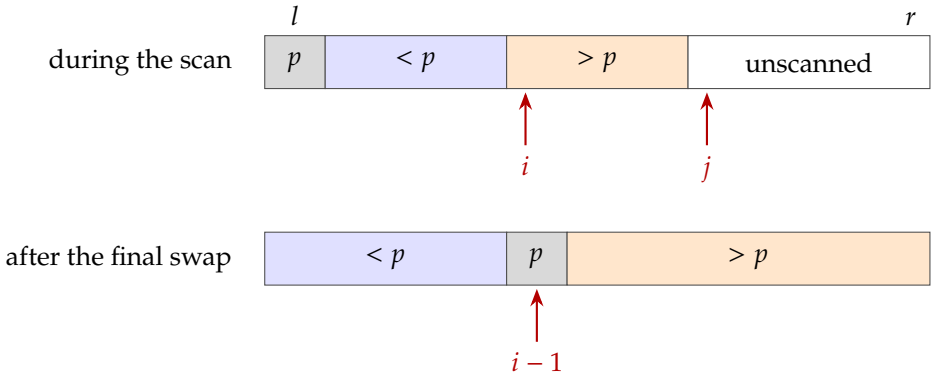


Figure 9: The subarray $A[l : r]$ during the scan and after the final swap. The index i marks the first entry greater than p, and j marks the next entry to examine.

Algorithm 5.2: Partition

Input: Subarray $A[l : r]$ with distinct entries; the pivot is $A[l]$.
Output: $A[l : r]$ partitioned around the pivot, and the pivot's final index.

```
1 p ← A[l]
2 i ← l + 1
3 for j ← l + 1 to r do
4 |   if A[j] < p then // if A[j] > p, do nothing
5 |   |   swap A[j] and A[i]
6 |   |   i ← i + 1
7 swap A[l] and A[i − 1]
8 return i − 1
```

The loop examines each of the $n - 1$ non-pivot entries once, with $O(1)$ work per entry, where $n = r - l + 1$. Thus Partition runs in $O(n)$ time. It only swaps entries, so it works in place.

Lemma 5.2 [Partition invariant]. *At the start of the iteration for each j, and when the loop ends with $j = r + 1$,*

$$A[l + 1], \ldots, A[i - 1] < p \qquad \text{and} \qquad A[i], \ldots, A[j - 1] > p.$$

Consequently Partition correctly partitions $A[l : r]$, and returns the final index of the pivot.

Proof. Use induction on j. Initially $i = j = l + 1$, so both ranges are empty. Suppose the invariant holds before examining $A[j]$. If $A[j] > p$, the second range simply grows to include position j. If $A[j] < p$, then $A[i]$ is either the first entry greater than p, which moves to position

j at the end of the second range, or $i = j$ and the swap does nothing. Either way, the entry less than p moves to position i, extending the first range, and incrementing i restores both conditions. When the loop ends, $A[l+1 : i-1]$ holds the entries less than p and $A[i : r]$ holds those greater than p. The final swap moves p to position $i-1$ and moves $A[i-1]$, which is either less than p or the pivot itself, to position l. Now everything left of position $i-1$ is less than p and everything right of it is greater. □

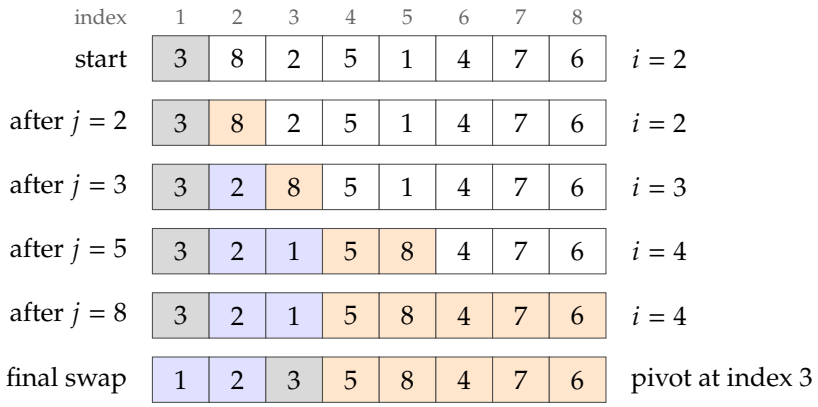


Figure 10: Partition on $3, 8, 2, 5, 1, 4, 7, 6$ with pivot 3 (gray). Blue entries are less than the pivot, orange entries are greater, and white entries are unscanned. Only $j = 3$ and $j = 5$ trigger swaps; the other iterations only extend the orange range.

In the in-place implementation, a call on $A[l : r]$ partitions it, obtains the pivot index q, and recurses on $A[l : q-1]$ and $A[q+1 : r]$. No combining step is needed.

5.3 Correctness

Let $P(n)$ be the assertion that QuickSort correctly sorts every input array of length n. Strong induction proves $P(n)$ for all $n \geq 1$: prove $P(1)$ directly, and for each $n \geq 2$ prove that $P(k)$ for all $k < n$ implies $P(n)$. The INDUCTIVE HYPOTHESIS is the assumption that $P(k)$ holds for every $k < n$.

Theorem 5.3 [CORRECTNESS OF QUICKSORT]. *For every $n \geq 1$, QuickSort correctly sorts every input array of length n, no matter how the pivots are chosen.*

Proof. An array of length one is already sorted, and QuickSort returns it unchanged, so $P(1)$ holds; an empty part is likewise returned unchanged. For the inductive step, fix $n \geq 2$ and an input array of length n. QuickSort first partitions around some pivot p. Let k_1 and k_2 be the lengths of the first and second parts. Since p belongs to neither part, $k_1, k_2 < n$. By the inductive hypothesis $P(k_1)$ and $P(k_2)$, the recursive calls sort both parts correctly. Every entry of the first part is less than p, which is less than every entry of the second part. Hence the whole array is sorted. □

5.4 The importance of the pivot

The running time of QuickSort depends on the quality of the pivot.

Example: Suppose ChoosePivot always selects the first entry, and the input is already sorted. Each pivot is the minimum, so the first part is empty and the second part has length $n-1$ and is still sorted. Partitioning a subarray of length m takes time proportional to m, so the running time is at least proportional to

$$n + (n-1) + (n-2) + \cdots + 1 \;\geq\; \frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{4},$$

because the first $n/2$ terms are each at least $n/2$. With the matching upper bound $n(n+1)/2$, the running time is $\Theta(n^2)$.

Example: Suppose instead that every recursive call magically chooses the median of its subarray as its pivot. Let $T(n)$ be the running time on arrays of length n. Both parts then have length about $n/2$, and choosing the pivot and partitioning take $\Theta(n)$ time, so

$$T(n) \leq 2T(n/2) + \Theta(n).$$

This is the merge-sort recurrence, so $T(n) = \Theta(n \log n)$.

5.5 Random pivots

The key question is how to choose pivots. The answer is to choose them randomly: in every recursive call, choose the pivot uniformly at random, with each entry of the subarray equally likely. The hope is that a random pivot is good enough sufficiently often.

Two observations support this. First, a 25–75 SPLIT, where each part has at most 3/4 of the entries, would already suffice. If every call achieved such a split, a subproblem at depth d of the recursion tree would have size at most $(3/4)^d n$, so the depth would be at most $\log_{4/3} n + 1 = O(\log n)$. The subarrays at any one depth are disjoint, so partitioning them all takes $O(n)$ total time per level, giving $O(n \log n)$ overall. Second, half of the entries give a 25–75 split or better: for n divisible by 4, any pivot whose rank lies between $n/4 + 1$ and $3n/4$.

Theorem 5.4 [QUICKSORT THEOREM]. *For every input array of length n, the average running time of QuickSort with random pivots is $O(n \log n)$.*

The bound holds for every input, with no assumptions on the data. The average is over the random choices made by the algorithm, namely its pivot choices, not over a distribution of inputs. Lecture 6 proves the theorem.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), QuickSort: Algorithm.

Lecture 6: QuickSort: Analysis

This lecture proves the QuickSort theorem: for every input array of length n, the average running time of QuickSort with uniformly random pivots is $O(n \log n)$. The average is over the pivot choices made by the algorithm; the input is arbitrary. The proof uses finite sample spaces, random variables, expectation, and linearity of expectation (Lecture 7).

6.1 Reducing running time to comparisons

Fix an input array A of length n with distinct entries. The sample space Ω is the set of all possible outcomes of the random choices in QuickSort, that is, all pivot sequences. For $\sigma \in \Omega$, let $C(\sigma)$ be the number of comparisons between two input entries made by QuickSort with pivot sequence σ, and let $RT(\sigma)$ be its running time.

Lemma 6.1 [Comparisons dominate]. *For $n \geq 2$, there is a constant c such that $RT(\sigma) \leq c\, C(\sigma)$ for every $\sigma \in \Omega$.*

Proof. A call on a subarray of length $m \geq 2$ chooses a pivot, swaps, and runs Partition, which makes $m - 1 \geq 1$ comparisons, each with $O(1)$ surrounding work. Hence, for some constant c', such a call does at most $c'm \leq 2c'(m-1)$ work beyond its recursive calls. A call on length 0 or 1 does $O(1)$ work and makes no comparisons, but it is one of at most two children of a call on length $m \geq 2$, so its cost can be charged to that parent's comparisons. Summing over all calls gives the bound. □

It therefore remains to prove $\mathbb{E}[C] = O(n \log n)$. The master method does not apply, because the subproblems are random and can be unbalanced.

6.2 Decomposing the comparison count

Let z_i denote the ith smallest entry of A; these labels refer to values, not positions. For indices $i < j$ and $\sigma \in \Omega$, let $X_{ij}(\sigma)$ be the number of times z_i and z_j are compared when QuickSort uses pivot sequence σ.

Any two entries are compared at most once. Every comparison involves the current pivot, and the pivot is excluded from both recursive calls, so it is never compared again. Thus each X_{ij} is an indicator random variable, taking only the values 0 and 1.

Every comparison is between some pair z_i, z_j with $i < j$, so

$$C(\sigma) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}(\sigma) \qquad \text{for every } \sigma \in \Omega.$$

By linearity of expectation, and since $\mathbb{E}[X_{ij}] = 0 \cdot \mathbb{P}[X_{ij} = 0] + 1 \cdot \mathbb{P}[X_{ij} = 1] = \mathbb{P}[X_{ij} = 1]$,

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{P}[z_i, z_j \text{ get compared}]. \tag{6.1}$$

The random variable C is complicated, but each X_{ij} is simple.

This is an instance of a general DECOMPOSITION PRINCIPLE:

1. identify the random variable Y of interest;
2. express Y as a sum of indicator random variables, $Y = \sum_{e=1}^{m} X_e$;
3. apply linearity of expectation, $\mathbb{E}[Y] = \sum_{e=1}^{m} \mathbb{P}[X_e = 1]$, and analyze each of these probabilities.

6.3 The key insight

Proposition 6.2 [COMPARISON PROBABILITY]. *For all $i < j$,*

$$\mathbb{P}[z_i, z_j \textit{ get compared}] = \frac{2}{j - i + 1}.$$

Proof. Consider the $j - i + 1$ entries $z_i, z_{i+1}, \ldots, z_j$. As long as none of them has been chosen as a pivot, they all lie in the same recursive call: any pivot used so far is either smaller than all of them or larger than all of them, so partitioning sends all of them to the same side. Consider the first among $z_i, \ldots, z_j$ chosen as a pivot.

1. If it is z_i or z_j, then the other one is in the same subarray and is compared with the pivot during partitioning.
2. If it is some z_k with $i < k < j$, then $z_i < z_k < z_j$ are split into different recursive calls, so z_i and z_j are never compared.

When the first of them is chosen, the pivot is uniform over a subarray containing all $j - i + 1$ of them, so each is equally likely to be the first. Two of these $j - i + 1$ choices lead to case 1. □

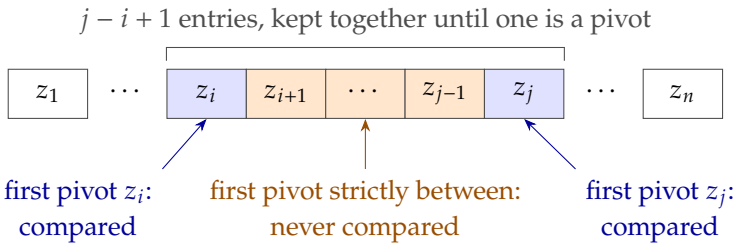


Figure 11: Entries in sorted order. Until one of $z_i, \ldots, z_j$ becomes a pivot, the whole group stays together. The comparison happens exactly when the first pivot from the group is one of its two endpoints.

Substituting into equation (6.1) gives

$$\mathbb{E}[C] = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{1}{j-i+1}. \tag{6.2}$$

6.4 Final calculation

The double sum in equation (6.2) has $\Theta(n^2)$ terms, so it must be bounded more carefully than term by term. For each fixed i, the inner sum is

$$\sum_{j=i+1}^{n} \frac{1}{j-i+1} = \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-i+1} \leq \sum_{k=2}^{n} \frac{1}{k}.$$

There are fewer than n choices of i, so

$$\mathbb{E}[C] \leq 2n \sum_{k=2}^{n} \frac{1}{k}.$$

Lemma 6.3 [Harmonic bound]. $\displaystyle\sum_{k=2}^{n} \frac{1}{k} \leq \ln n.$

Proof. Since $1/x$ is decreasing, $1/k \leq 1/x$ for $x \in [k-1, k]$, so the term $1/k$ is the area of a width-one rectangle lying under the curve $1/x$ on $[k-1, k]$. Therefore

$$\sum_{k=2}^{n} \frac{1}{k} \leq \int_1^n \frac{dx}{x} = \ln x \Big|_1^n = \ln n - \ln 1 = \ln n. \qquad \square$$

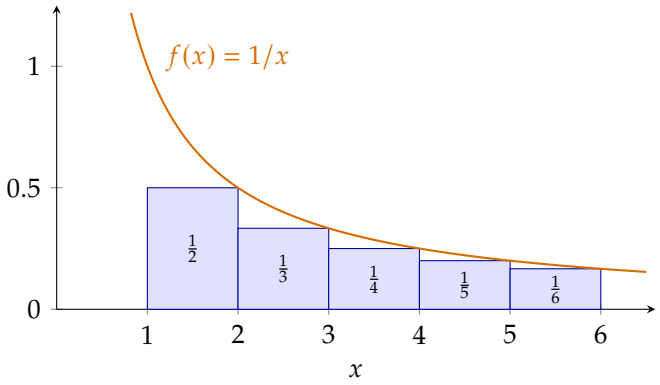


Figure 12: The rectangle of height $1/k$ on $[k-1, k]$ lies under $f(x) = 1/x$, shown here for $n = 6$. The total rectangle area $\sum_{k=2}^{n} 1/k$ is at most the area under the curve from 1 to n.

Theorem 6.4 [QuickSort theorem]. *For every input array of length n, the expected number of comparisons of QuickSort with random pivots satisfies $\mathbb{E}[C] \leq 2n \ln n$. Hence its average running time is $O(n \log n)$.*

Proof. Combine the bound $\mathbb{E}[C] \leq 2n \sum_{k=2}^{n} 1/k$ with the harmonic bound. By the comparison lemma, $\mathbb{E}[RT] \leq c\,\mathbb{E}[C] \leq 2cn \ln n$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), QuickSort: Analysis.

Lecture 7: Probability for Randomized Algorithms

The analysis of randomized algorithms such as QuickSort uses a small amount of discrete probability. Two running examples recur: rolling two dice, and choosing a uniformly random pivot in the outermost call of QuickSort on an array of length n with distinct entries.

7.1 Sample spaces and events

Definition 7.1 [Sample space]. A sample space Ω is the set of all possible outcomes. In algorithms Ω is usually finite. Each outcome $i \in \Omega$ has a probability $p(i) \geq 0$, and

$$\sum_{i \in \Omega} p(i) = 1.$$

For two dice, $\Omega = \{(1,1), (2,1), \ldots, (6,6)\}$ consists of the 36 ordered pairs, each with probability $1/36$. For a random pivot, $\Omega = \{1, 2, \ldots, n\}$ is the index of the pivot, and $p(i) = 1/n$ for every $i \in \Omega$.

Definition 7.2 [Event]. An event is a subset $S \subseteq \Omega$. Its probability is

$$\mathbb{P}[S] = \sum_{i \in S} p(i).$$

Example: The event that the two dice sum to 7 is $S = \{(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)\}$, so $\mathbb{P}[S] = 6/36 = 1/6$.

Example: Let n be divisible by 4. The event that the pivot gives a 25–75 split or better contains the pivots from the $(n/4 + 1)$th smallest entry to the $(3n/4)$th smallest entry. It has $n/2$ outcomes, so $\mathbb{P}[S] = (n/2)/n = 1/2$.

7.2 Random variables and expectation

Definition 7.3 [Random variable]. A random variable is a real-valued function $X : \Omega \to \mathbb{R}$.

Examples are the sum of the two dice, and the size of the subarray passed to the first recursive call of QuickSort.

Definition 7.4 [Expectation]. The expectation of a random variable X is its average value, weighted by probability:

$$\mathbb{E}[X] = \sum_{i \in \Omega} X(i)\, p(i).$$

Example: Let X be the size of the subarray passed to the first recursive call. If the pivot is the ith smallest entry, which happens with probability $1/n$, the first part has $i-1$ entries. Hence

$$\mathbb{E}[X] = \frac{1}{n}\cdot 0 + \frac{1}{n}\cdot 1 + \cdots + \frac{1}{n}\cdot (n-1) = \frac{1}{n}\cdot\frac{(n-1)n}{2} = \frac{n-1}{2}.$$

7.3 Linearity of expectation

Theorem 7.5 [Linearity of expectation]. *Let $X_1, \ldots, X_n$ be random variables defined on the same sample space Ω. Then*

$$\mathbb{E}\Big[\sum_{j=1}^{n} X_j\Big] = \sum_{j=1}^{n} \mathbb{E}[X_j].$$

Proof. Expand each expectation and reverse the order of the two finite sums:

$$\sum_{j=1}^{n}\mathbb{E}[X_j] = \sum_{j=1}^{n}\sum_{i\in\Omega} X_j(i)p(i) = \sum_{i\in\Omega} p(i)\sum_{j=1}^{n} X_j(i) = \mathbb{E}\Big[\sum_{j=1}^{n} X_j\Big]. \qquad \square$$

Crucially, the identity holds even when the X_j are not independent. It would fail in general if sums were replaced by products.

Example: Let X_1, X_2 be the values of the two dice. Each has $\mathbb{E}[X_j] = \frac{1}{6}(1+2+\cdots+6) = 3.5$, so the expected sum is $\mathbb{E}[X_1 + X_2] = 3.5 + 3.5 = 7$.

Example: *Load balancing.* Assign each of n processes to one of n servers, chosen uniformly at random. The sample space consists of all n^n assignments, each equally likely. Let Y be the number of processes assigned to the first server, and let $X_j = 1$ if process j is assigned to the first server and $X_j = 0$ otherwise. Then $Y = \sum_{j=1}^{n} X_j$, and each X_j is an indicator random variable with $\mathbb{P}[X_j = 1] = 1/n$. Thus

$$\mathbb{E}[Y] = \sum_{j=1}^{n}\mathbb{E}[X_j] = \sum_{j=1}^{n}\big(0\cdot\mathbb{P}[X_j=0] + 1\cdot\mathbb{P}[X_j=1]\big) = \sum_{j=1}^{n}\frac{1}{n} = 1.$$

7.4 Conditional probability

Definition 7.6 [Conditional probability]. For events $X, Y \subseteq \Omega$ with $\mathbb{P}[Y] > 0$, the conditional probability of X given Y is

$$\mathbb{P}[X \mid Y] = \frac{\mathbb{P}[X \cap Y]}{\mathbb{P}[Y]}.$$

Conditioning on Y restricts attention to the outcomes in Y, rescaled to total probability one.

Example: Roll two fair dice. Let X be the event that at least one die shows 1, and Y the event that the sum is 7. Then $X \cap Y = \{(1,6),(6,1)\}$, so

$$\mathbb{P}[X \mid Y] = \frac{2/36}{6/36} = \frac{1}{3}.$$

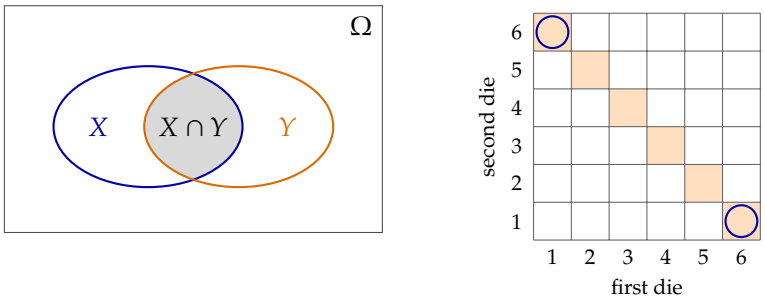


Figure 13: Left: events X and Y in a sample space Ω. Right: the 36 outcomes for two dice. The six outcomes with sum 7 form Y; the two circled outcomes form $X \cap Y$.

7.5 Independence

Definition 7.7 [Independent events]. Events $X, Y \subseteq \Omega$ are INDEPENDENT if and only if

$$\mathbb{P}[X \cap Y] = \mathbb{P}[X] \cdot \mathbb{P}[Y].$$

When $\mathbb{P}[Y] > 0$, dividing by $\mathbb{P}[Y]$ shows that this holds if and only if $\mathbb{P}[X \mid Y] = \mathbb{P}[X]$. Similarly, when $\mathbb{P}[X] > 0$, it holds if and only if $\mathbb{P}[Y \mid X] = \mathbb{P}[Y]$. Independence can be a very subtle concept, and intuition about it is often incorrect.

Definition 7.8 [Independent random variables]. Random variables A, B defined on Ω are independent if and only if the events $\{A = a\}$ and $\{B = b\}$ are independent for all values a, b, that is,

$$\mathbb{P}[A = a \text{ and } B = b] = \mathbb{P}[A = a] \cdot \mathbb{P}[B = b].$$

Proposition 7.9 [Expectation of a product]. *If A and B are independent, then $\mathbb{E}[AB] = \mathbb{E}[A] \cdot \mathbb{E}[B]$.*

Proof. Group outcomes by the values a of A and b of B, then use independence to factor each probability:

$$\begin{aligned}\mathbb{E}[AB] &= \sum_{a,b} ab\, \mathbb{P}[A = a \text{ and } B = b] = \sum_{a,b} ab\, \mathbb{P}[A = a]\, \mathbb{P}[B = b] \\ &= \Big(\sum_{a} a\, \mathbb{P}[A = a]\Big)\Big(\sum_{b} b\, \mathbb{P}[B = b]\Big) = \mathbb{E}[A] \cdot \mathbb{E}[B].\end{aligned}$$

□

Example: Let $X_1, X_2 \in \{0, 1\}$ be random bits and $X_3 = X_1 \oplus X_2$ their exclusive or. Formally, writing outcomes as $X_1X_2X_3$,

$$\Omega = \{000,\ 101,\ 011,\ 110\},$$

each equally likely. The pair (X_1, X_3) takes each of the four values in $\{0, 1\}^2$ on exactly one outcome, so each joint probability is $1/4 = \frac{1}{2} \cdot \frac{1}{2}$, and X_1, X_3 are independent.

However, X_1X_3 and X_2 are not independent. If they were, the product rule would give $\mathbb{E}[X_1X_2X_3] = \mathbb{E}[X_1X_3] \cdot \mathbb{E}[X_2]$. But $X_1X_2X_3 = 0$ on every outcome, so the left side is 0, while

$$\mathbb{E}[X_1X_3] \cdot \mathbb{E}[X_2] = \mathbb{E}[X_1]\,\mathbb{E}[X_3] \cdot \mathbb{E}[X_2] = \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8},$$

using the independence of X_1 and X_3.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Probability Review.

Lecture 8: Linear-Time Selection

Selection asks for one element of a given rank, rather than the whole sorted order. Sorting solves it in $O(n \log n)$ time; adapting QuickSort's partitioning solves it in linear time, first in expectation with random pivots and then deterministically.

8.1 The selection problem

Definition 8.1 [Selection]. Given an array A of n distinct numbers and an integer i with $1 \le i \le n$, return the ith order statistic: the ith smallest element of A.

Distinctness is assumed for simplicity. The median is the case $i = (n+1)/2$ for odd n and $i = n/2$ for even n. For example, the array $(10, 8, 2, 4)$ has third order statistic 8.

Merge sort followed by reading position i of the sorted array solves the problem in $O(n \log n)$ time. The last subsection shows that no comparison-based sorting algorithm is asymptotically faster, so a linear algorithm must avoid sorting.

8.2 Partitioning and randomized selection

To partition an array around a chosen pivot element p, rearrange it so that every element left of p is smaller than p and every element right of p is larger. The two sides need not be sorted. Partitioning takes linear time and puts p in its "rightful position": if it lands at position j, then p is the jth order statistic.

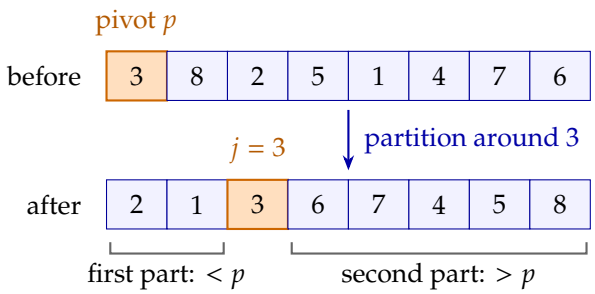


Figure 14: Partitioning $(3, 8, 2, 5, 1, 4, 7, 6)$ around the pivot 3. The pivot ends at position $j = 3$, its rank; the elements before it are smaller and the elements after it are larger, in no particular order.

Only one side of the pivot can contain the ith order statistic. If $j > i$, it is the ith smallest element of the first part. If $j < i$, then the first j positions hold the j smallest elements, so the target is the

$(i-j)$th smallest element of the second part. For instance, when seeking the 5th order statistic of 10 elements and the pivot lands in position 3, recurse on the right side for its 2nd order statistic.

Algorithm 8.1: Randomized selection, RSelect(A, n, i)

Input: Array $A[1:n]$ of distinct numbers and an integer $1 \le i \le n$.
Output: The ith order statistic of A.

```
if n = 1 then
    return A[1]
Choose a pivot p from A uniformly at random
Partition A around p; let j be the new position of p
if j = i then
    return p
if j > i then
    return RSelect(A[1 : j − 1], j − 1, i)
else
    return RSelect(A[j + 1 : n], n − j, i − j)
```

Proposition 8.2 [Correctness of RSelect]. *RSelect always returns the ith order statistic.*

Proof. Induct on n. The case $n = 1$ is immediate. For $n \ge 2$, the pivot is the jth order statistic. If $j = i$ it is the answer. Otherwise the recursive call receives a shorter array that, by the observation above, contains the answer at the stated rank; the induction hypothesis finishes the proof. □

The running time depends on the quality of the pivots. With the worst possible pivots it is $\Theta(n^2)$: if $i = n/2$ and the pivot is always the minimum, each of the first $n/2$ calls does $\Omega(n)$ partitioning work on an array that shrinks by only one element.

The ideal pivot is the median, which halves the array. That would give $T(n) \le T(n/2) + O(n)$, so $T(n) = O(n)$ by the master method ($a = 1 < b^d = 2$). Computing the median is the original problem, so this is circular. The hope is that a random pivot is good enough often enough.

Theorem 8.3 [RSelect]. *For every input array of length n, the expected running time of RSelect is $O(n)$.*

No assumption is made on the data. The expectation is over the random pivot choices made by the algorithm.

8.3 Analysis of randomized selection

Outside its recursive call, RSelect performs at most cn operations on an array of length n, for a constant $c > 0$, mostly from partitioning.

Track progress in phases. RSelect is in phase j when the current array has length between $(3/4)^{j+1}n$ and $(3/4)^j n$. Let X_j be the number of recursive calls made during phase j. Each such call works on

an array of length at most $(3/4)^j n$, so

$$\text{running time of RSelect} \le \sum_{\text{phases } j} X_j \cdot c \left(\tfrac{3}{4}\right)^j n. \tag{8.1}$$

Lemma 8.4 [Phases are short]. *For every phase j, $\mathbb{E}[X_j] \le 2$.*

Proof. If a call chooses a pivot giving a 25–75 split or better, the next array has at most 75% of the current length, so the current phase ends. Such a split occurs when the pivot's rank lies in the middle half of the current array, which has probability $1/2$ (ignoring rounding) independently of earlier choices. Call a good pivot "heads" and a bad one "tails". Then X_j is at most the number N of fair coin flips needed to see the first heads. This N is a geometric random variable. The first flip is always made; with probability $1/2$ it is tails and the process starts afresh. Hence

$$\mathbb{E}[N] = 1 + \tfrac{1}{2}\mathbb{E}[N], \qquad \text{so} \qquad \mathbb{E}[N] = 2.$$

Therefore $\mathbb{E}[X_j] \le \mathbb{E}[N] = 2$. □

Proof of the RSelect theorem. Take expectations in (8.1), use linearity of expectation, and bound the geometric series:

$$\mathbb{E}[\text{running time}] \le cn \sum_j \left(\tfrac{3}{4}\right)^j \mathbb{E}[X_j] \le 2cn \sum_{j \ge 0} \left(\tfrac{3}{4}\right)^j \le 2cn \cdot \frac{1}{1 - 3/4} = 8cn = O(n).$$ □

8.4 Deterministic selection

To guarantee a good pivot without randomness, use the median of medians. Break A into $n/5$ groups of 5 elements, sort each group, copy the $n/5$ group medians (middle elements) into a new array C, and recursively compute the median of C. The rest of the algorithm is unchanged. DSelect therefore makes two recursive calls: one to choose the pivot and one on a side of the partition.

Sorting the groups takes linear time. By the merge-sort bound $6m(\log_2 m + 1)$ with $m = 5$ and $\log_2 5 \le 3$, one group costs at most $6 \cdot 5 \cdot 4 = 120$ operations, so all $n/5$ groups cost at most $24n = O(n)$.

Algorithm 8.2: Deterministic selection, DSelect(A, n, i)

Input: Array $A[1:n]$ of distinct numbers and an integer $1 \le i \le n$.
Output: The ith order statistic of A.

```
if n = 1 then
    return A[1]
Break A into groups of 5 and sort each group
C ← the n/5 middle elements of the groups
p ← DSelect(C, n/5, n/10)                    // median of C
Partition A around p; let j be the new position of p
if j = i then
    return p
if j > i then
    return DSelect(A[1 : j − 1], j − 1, i)
else
    return DSelect(A[j + 1 : n], n − j, i − j)
```

Theorem 8.5 [DSelect]. *For every input array of length n, DSelect runs in $O(n)$ time.*

In practice DSelect is not as good as RSelect: its constants are worse and it does not work in place. The algorithm is due to Blum, Floyd, Pratt, Rivest, and Tarjan (1973).

8.5 Analysis of deterministic selection

Let $T(n)$ be the maximum running time of DSelect on an array of length n. There is a constant $c \ge 1$ such that $T(1) = 1$ and

$$T(n) \le cn + T(n/5) + T(?).$$

Here cn covers sorting the groups and partitioning, $T(n/5)$ is the call that chooses the pivot, and $T(?)$ is the call on one side of the partition, whose size depends on how good the pivot is.

Lemma 8.6 [Key lemma]. *The second recursive call of DSelect is on an array of length at most roughly $7n/10$.*

Proof. Let $k = n/5$ be the number of groups and x_i the ith smallest group median, so the pivot is $x_{k/2}$. The goal is to show that at least 30% of the input is smaller than $x_{k/2}$ and at least 30% is larger. Imagine the elements laid out in a grid: each column is a group, sorted increasing from bottom to top, and the columns are ordered left to right by their medians $x_1 < x_2 < \cdots < x_k$. The pivot $x_{k/2}$ is larger than the medians $x_1, \ldots, x_{k/2-1}$, and each median is larger than the two elements below it in its column. So $x_{k/2}$ exceeds 3 of the 5 elements (60%) in roughly half of the groups, hence about $\frac{3}{5} \cdot \frac{1}{2} = 30\%$ of A. Symmetrically, $x_{k/2}$ is smaller than the medians $x_{k/2+1}, \ldots, x_k$ and the two elements above each of them, again about 30% of A. Whichever side the second call recurses on, it excludes at least 30% of the input. □

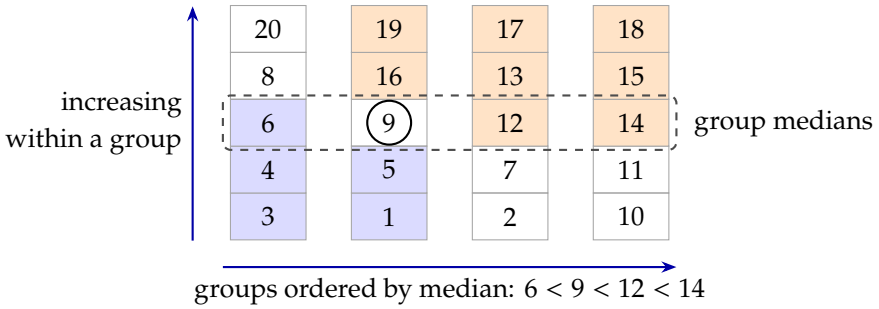


Figure 15: The grid for the input $(7, 2, 17, 12, 13,\ 8, 20, 4, 6, 3,\ 19, 1, 9, 5, 16,\ 10, 15, 18, 14, 11)$. The group medians are $12, 6, 9, 14$, and the pivot is their median 9 (circled). Blue cells are known to be smaller than the pivot; orange cells are known to be larger.

The recurrence becomes $T(1) = 1$ and

$$T(n) \le cn + T(n/5) + T(7n/10).$$

The subproblems have different sizes, so the master method does not apply. Instead, guess that $T(n) \le an$ for a constant a independent of n, and check it.

Proposition 8.7 [Solving the recurrence]. *With $a = 10c$, $T(n) \le an$ for all $n \ge 1$. Hence DSelect runs in $O(n)$ time.*

Proof. Induct on n. Base case: $T(1) = 1 \le a$, since $a \ge 1$. For $n > 1$, assume $T(k) \le ak$ for all $k < n$. Then

$$T(n) \le cn + T(n/5) + T(7n/10) \le cn + a\frac{n}{5} + a\frac{7n}{10} = n\left(c + \frac{9a}{10}\right) = an,$$

where the last step uses $c = a/10$. The argument works because the two subproblem sizes total $\frac{1}{5} + \frac{7}{10} = \frac{9}{10}$ of the input, strictly less than all of it. □

8.6 A sorting lower bound

A comparison-based sorting algorithm accesses the input elements only through comparisons; it is a general-purpose method. Merge sort, QuickSort, and heap sort are comparison-based. Bucket sort (suited to data drawn from known distributions), counting sort (small integers), and radix sort (medium-size integers) are not.

Theorem 8.8 [Sorting lower bound]. *Every comparison-based sorting algorithm has worst-case running time $\Omega(n \log n)$.*

The proof below assumes a deterministic algorithm; the lower bound extends to randomized

algorithms.

Proof. Fix a comparison-based algorithm and a length n. Consider the $n!$ input arrays that contain $\{1, 2, \ldots, n\}$ in some order. Suppose the algorithm always correctly sorts these inputs using at most k comparisons. The execution of a comparison-based algorithm is determined by the outcomes of its comparisons. With at most k binary outcomes, there are at most 2^k distinct executions across all $n!$ inputs. If $2^k < n!$, then by the pigeonhole principle two distinct inputs produce identical executions. Both are then rearranged in exactly the same way, so at most one of them comes out sorted, a contradiction. Hence

$$2^k \geq n! \geq \left(\frac{n}{2}\right)^{n/2},$$

where the second inequality keeps only the largest $n/2$ factors of $n!$, each at least $n/2$. Taking logarithms,

$$k \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n).$$

□

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Linear-Time Selection.

Lecture 9: Graphs and the Contraction Algorithm

Graphs give a new domain for randomized algorithms. After the basic vocabulary and two representations, a simple random contraction algorithm finds a minimum cut with probability at least $1/n^2$; repetition makes failure unlikely, and the same analysis bounds how many minimum cuts a graph can have.

9.1 Graphs and cuts

A graph $G = (V, E)$ has two ingredients: vertices (or nodes) V, and edges E, which are pairs of vertices. An edge is undirected if the pair is unordered and directed (an arc) if it is ordered, from its tail to its head. Graphs model road networks, the Web, social networks, precedence constraints, and more.

Definition 9.1 [Cut]. A cut of a graph (V, E) is a partition of V into two nonempty sets A and B. The crossing edges of a cut (A, B) are the edges with one endpoint in each of A and B (undirected case), or with tail in A and head in B (directed case).

Each vertex can go to either side, and both sides must be nonempty, so a graph with n vertices has $2^n - 2$ ordered cuts (A, B): roughly 2^n.

Definition 9.2 [Minimum cut problem]. Given an undirected graph $G = (V, E)$, possibly with parallel edges, compute a cut with the fewest crossing edges, a minimum cut.

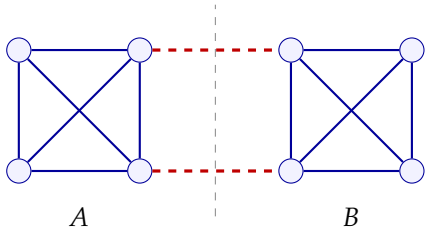


Figure 16: Two complete graphs on four vertices joined by two edges. The cut separating them has 2 crossing edges (dashed), which is the minimum.

Minimum cuts identify network bottlenecks and weaknesses, and help detect communities in social networks. In image segmentation, the input is a graph of pixels with edge weights, where a large weight on (u, v) means u and v are expected to come from the same object; repeated minimum cuts can identify the primary objects in the picture.

9.2 Representing graphs

Throughout, n is the number of vertices and m the number of edges. A connected undirected graph with no parallel edges has between $n-1$ and $n(n-1)/2$ edges. In most (but not all) applications, m is $\Omega(n)$ and $O(n^2)$. In a SPARSE graph m is $O(n)$ or close to it; in a DENSE graph m is closer to $\Theta(n^2)$.

The ADJACENCY MATRIX represents G by an $n \times n$ 0-1 matrix with $A_{ij} = 1$ exactly when G has an i-j edge. Variants store the number of i-j edges (for parallel edges), the weight of the i-j edge, or, for directed graphs, $A_{ij} = +1$ for an arc from i to j and -1 for an arc from j to i. It needs $\Theta(n^2)$ space regardless of m.

ADJACENCY LISTS store the following ingredients:

Ingredient	Space
array (or list) of vertices	$\Theta(n)$
array (or list) of edges	$\Theta(m)$
each edge points to its endpoints	$\Theta(m)$
each vertex points to its incident edges	$\Theta(m)$
total	$\Theta(m+n)$

The last two rows are in one-to-one correspondence: each edge-endpoint incidence is recorded once from each side. The total $\Theta(m+n)$ is the same as $\Theta(\max\{m,n\})$. Which representation is better depends on the graph's density and the operations needed; this course focuses on adjacency lists.

9.3 The random contraction algorithm

CONTRACTING an edge (u,v) merges u and v into a single vertex, which keeps all edges of both; edges between u and v become self-loops. Parallel edges can appear, which is why the problem allows them.

Algorithm 9.1: Random contraction (Karger, early 1990s)

Input: Undirected graph $G = (V, E)$, possibly with parallel edges.
Output: A cut of G.

```
while more than 2 vertices remain do
    Pick a remaining edge (u, v) uniformly at random
    Contract u and v into a single vertex
    Remove self-loops
return the cut whose sides are the vertex sets merged into the final two vertices
```

Each remaining vertex stands for the set of original vertices merged into it. The output depends on the random choices, and it need not be a minimum cut.

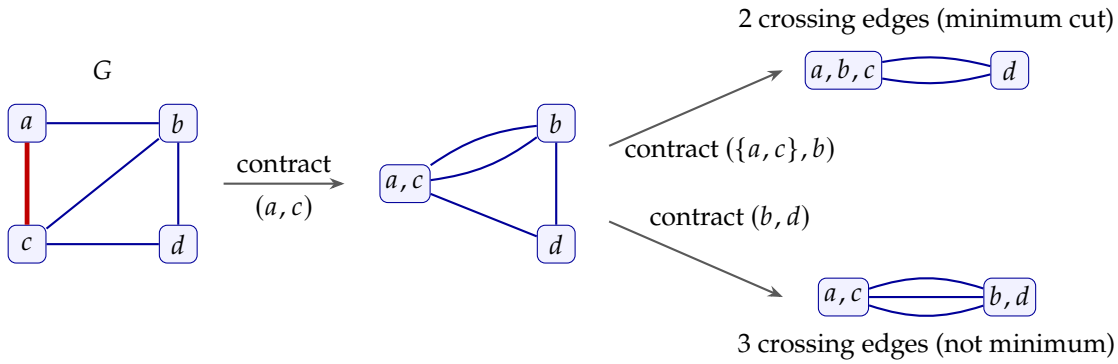


Figure 17: Two runs on the same graph, both contracting (a, c) first (thick edge). Contracting one of the two edges between $\{a, c\}$ and b next yields the cut $(\{a, b, c\}, \{d\})$ with 2 crossing edges, a minimum cut. Contracting (b, d) instead yields $(\{a, c\}, \{b, d\})$ with 3 crossing edges, which is not a minimum cut.

9.4 Success probability of one run

Fix a graph $G = (V, E)$ with n vertices and m edges, and fix a minimum cut (A, B). Let k be its number of crossing edges and F the set of these edges.

If an edge of F is ever contracted, an endpoint in A and an endpoint in B are merged, so the output cannot be (A, B). If only edges inside A or inside B are contracted, every merged vertex lies entirely in A or entirely in B, and the final two vertices must therefore be A and B. Thus

$$\mathbb{P}[\text{output is } (A, B)] = \mathbb{P}[\text{no edge of } F \text{ is ever contracted}].$$

Each iteration reduces the number of vertices by one, so there are $n - 2$ iterations. Let S_i be the event that an edge of F is contracted in iteration i. The goal is to bound $\mathbb{P}[\neg S_1 \wedge \neg S_2 \wedge \cdots \wedge \neg S_{n-2}]$.

Lemma 9.3 [First iteration]. $\mathbb{P}[S_1] = k/m \leq 2/n$.

Proof. The first edge is uniform among m edges, k of which cross (A, B), so $\mathbb{P}[S_1] = k/m$. Every vertex v defines the cut $(\{v\}, V - \{v\})$, whose crossing edges are the $\deg(v)$ edges incident to v. Since (A, B) is a minimum cut, $\deg(v) \geq k$ for every v. Summing degrees counts each edge twice, so $2m = \sum_v \deg(v) \geq kn$, that is, $m \geq kn/2$. Hence $k/m \leq 2/n$. □

The same argument applies to the contracted graphs. Each vertex of a contracted graph corresponds to a set of original vertices, which defines a cut of G with at least k crossing edges, so every degree in the contracted graph is at least k. Given that no edge of F has yet been contracted, all k edges of F remain.

Theorem 9.4 [Success probability]. *The contraction algorithm outputs the fixed minimum cut* (A, B) *with probability at least* $2/(n(n-1)) \geq 1/n^2$.

Proof. Before iteration i, the contracted graph has $n - i + 1$ vertices, each of degree at least k, so at least $k(n - i + 1)/2$ edges remain. Given $\neg S_1 \wedge \cdots \wedge \neg S_{i-1}$, all k edges of F remain, so

$$\mathbb{P}[\neg S_i \mid \neg S_1 \wedge \cdots \wedge \neg S_{i-1}] = 1 - \frac{k}{\#\text{ remaining edges}} \geq 1 - \frac{2}{n-i+1}.$$

For $i = 2$ this gives $1 - 2/(n-1)$. Multiplying the conditional probabilities,

$$\begin{aligned}\mathbb{P}[\neg S_1 \wedge \cdots \wedge \neg S_{n-2}] &= \mathbb{P}[\neg S_1]\,\mathbb{P}[\neg S_2 \mid \neg S_1] \cdots \mathbb{P}[\neg S_{n-2} \mid \neg S_1 \wedge \cdots \wedge \neg S_{n-3}] \\ &\geq \left(1 - \frac{2}{n}\right)\left(1 - \frac{2}{n-1}\right) \cdots \left(1 - \frac{2}{4}\right)\left(1 - \frac{2}{3}\right) \\ &= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} \geq \frac{1}{n^2}.\end{aligned}$$

The product telescopes: each numerator cancels the denominator two factors later, leaving $2 \cdot 1$ over $n(n-1)$. □

The success probability is low, but nontrivial, since a graph has roughly 2^n cuts.

9.5 Repeated trials

Run the basic algorithm N times independently and remember the smallest cut found. Let T_i be the event that the cut (A, B) is found on the ith try. The T_i are independent by construction, so

$$\mathbb{P}[\text{all } N \text{ trials fail}] = \mathbb{P}[\neg T_1 \wedge \cdots \wedge \neg T_N] = \prod_{i=1}^{N} \mathbb{P}[\neg T_i] \leq \left(1 - \frac{1}{n^2}\right)^N.$$

The calculus fact $1 + x \leq e^x$ for every real x bounds this quantity.

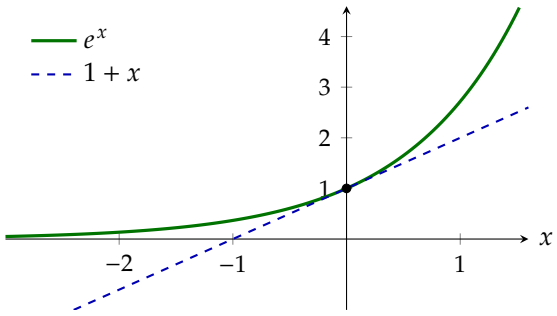


Figure 18: The line $1 + x$ lies below e^x, touching it only at $x = 0$.

With $x = -1/n^2$,

$$\mathbb{P}[\text{all fail}] \leq \left(e^{-1/n^2}\right)^N.$$

Taking $N = n^2$ trials gives failure probability at most $1/e$; taking $N = n^2 \ln n$ gives at most

$(1/e)^{\ln n} = 1/n$. The resulting running time is polynomial in n and m but slow, $\Omega(n^2 m)$. More ideas give large speedups, to roughly $O(n^2)$.

9.6 Counting minimum cuts

A graph can have several minimum cuts. For example, a tree with n vertices has $n - 1$ minimum cuts: removing any one edge splits the tree into two pieces, giving a cut with one crossing edge.

Theorem 9.5 [Number of minimum cuts]. *The largest number of minimum cuts that a graph with n vertices can have is $\binom{n}{2} = n(n-1)/2$.*

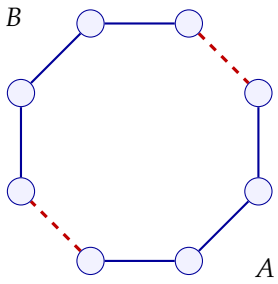


Figure 19: The 8-cycle. Removing any two edges, such as the dashed ones, splits it into two paths A and B, giving a minimum cut with 2 crossing edges.

Proof. Lower bound. Consider the n-cycle. Every cut crosses the cycle at least twice, since traversing the cycle from a vertex in A must leave A and return. Each pair of the n edges defines a cut with exactly these two crossing edges: removing them splits the cycle into two paths, which form the two sides. Different pairs give different cuts, so the n-cycle has at least $\binom{n}{2}$ minimum cuts.

Upper bound. Let $(A_1, B_1), \ldots, (A_t, B_t)$ be the minimum cuts of a graph with n vertices. The analysis of a single run of the contraction algorithm applies to each fixed minimum cut, so for every $i = 1, \ldots, t$,

$$\mathbb{P}[\text{output} = (A_i, B_i)] \geq \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}.$$

These events are disjoint, since one run outputs only one cut. Their probabilities therefore sum to at most 1:

$$\frac{t}{\binom{n}{2}} \leq 1, \qquad \text{so} \qquad t \leq \binom{n}{2}. \qquad \square$$

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Graphs and the Contraction Algorithm.

Lecture 10: Graph Search and Connectivity

Graph search answers questions such as whether a network is connected, so that every vertex can reach every other. In a graph of actors joined when they share a film, the path Jon Hamm, Colin Firth, Kevin Bacon gives Jon Hamm a Bacon number of 2. Search also underlies driving directions, planning (for a Sudoku puzzle, nodes are partially completed puzzles and arcs fill in one new square), and computing the pieces of a graph, used in clustering and in studying the structure of the Web. Throughout, $G = (V, E)$ has $n = |V|$ vertices and $m = |E|$ edges, stored as adjacency lists.

10.1 Generic graph search

The goals are to find everything findable from a given start vertex s, and to explore nothing twice, which suggests aiming for $O(m + n)$ time.

Algorithm 10.1: Generic graph search

Input: Graph $G = (V, E)$, undirected or directed, and a start vertex s.
Output: The vertices marked explored.

```
Mark s explored and every other vertex unexplored
while some edge (u, v) has u explored and v unexplored do
    Choose such an edge (u, v)
    Mark v explored
```

Proposition 10.1 [Search finds exactly the reachable vertices]. *For G undirected or directed, at the end of generic search a vertex v is explored if and only if G has a path from s to v.*

Proof. ($\Rightarrow$) Induct on the number of iterations. The vertex s is reachable, and a vertex v explored through an edge (u, v) with u already explored is reachable by extending an s–u path by that edge.
($\Leftarrow$) Suppose G has an s–v path P, but v is unexplored at the end. Since P starts at an explored vertex and ends at an unexplored one, some edge (u, x) of P has u explored and x unexplored. Then the loop condition still holds, so the algorithm would not have terminated, a contradiction. □

The generic procedure leaves open which frontier edge, crossing from explored to unexplored vertices, to choose. Two choices give the main search strategies.

- Breadth-first search (BFS) explores nodes in layers, using a queue (first in, first out), in $O(m + n)$ time. It computes shortest paths and the connected components of an undirected graph.

- Depth-first search (DFS) explores aggressively, like a maze, backtracking only when necessary. It uses a stack (last in, first out) or recursion, in $O(m + n)$ time. It computes a topological ordering of a directed acyclic graph and the strongly connected components of a directed graph.

10.2 Breadth-first search

Algorithm 10.2: Breadth-first search, BFS(G, s)

Input: Graph G with all vertices unexplored, and a start vertex s.
Output: The vertices reachable from s, marked explored.

```
1 Mark s explored
2 Q ← a queue containing only s
3 while Q ≠ ∅ do
4     Remove the first vertex v of Q
5     for each edge (v, w) do
6         if w is unexplored then
7             Mark w explored
8             Add w to the end of Q
```

The vertices at distance i from s form the layer L_i. BFS removes all of $L_0 = \{s\}$, then all of L_1, then all of L_2, and so on.

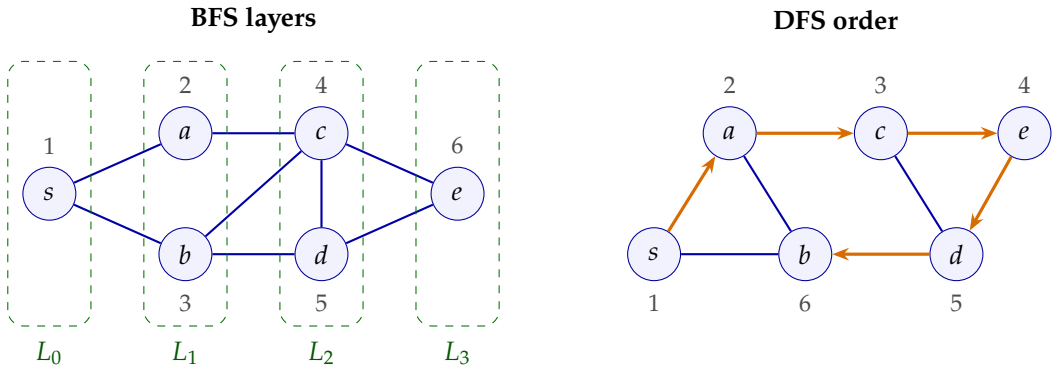


Figure 20: One undirected graph explored two ways. Left: BFS layers $L_0, \ldots, L_3$, with numbers giving the order in which vertices are explored. Right: DFS on a similar graph; the highlighted edges are the recursive calls, and the search backtracks only after reaching b.

Proposition 10.2 [Basic BFS properties]. *At the end of* BFS(G, s)*, a vertex v is explored if and only if G has an s–v path. The main loop runs in time $O(n_s + m_s)$, where n_s and m_s are the numbers of vertices and edges reachable from s.*

Proof. BFS is a special case of generic search: each newly explored w is reached by an edge from an explored v, and the loop stops only when no frontier edge remains. For the time bound, each reachable vertex enters Q at most once, because it is marked when added. Its

adjacency list is then scanned once, and each scanned edge costs $O(1)$. An edge is scanned at most twice (once from each end) in an undirected graph, and once in a directed graph. □

10.3 Shortest paths via BFS

Let $\text{dist}(v)$ be the fewest edges on an s–v path. Initialize $\text{dist}(s) = 0$ and $\text{dist}(v) = \infty$ for $v \neq s$. When BFS considers an edge (v, w) with w unexplored, it also sets $\text{dist}(w) \leftarrow \text{dist}(v) + 1$.

Proposition 10.3 [BFS computes shortest paths]. *At termination,* $\text{dist}(v) = i$ *if and only if* $v \in L_i$*, that is, a shortest* s–v *path has* i *edges.*

Proof. The idea is that every layer-i vertex w is added to Q by a layer-$(i-1)$ vertex v, via the edge (v, w). Prove by induction on i that the vertices of L_i enter Q after all vertices of earlier layers, before any vertex of a later layer, and with dist $= i$. This holds for $L_0 = \{s\}$. Assume it for layers up to $i - 1$. A vertex $w \in L_i$ has an edge from some vertex of L_{i-1}, and every vertex with an edge to w lies in L_{i-1} or a later layer. Hence w is still unexplored when the first such vertex of L_{i-1} is removed from Q, since vertices of later layers enter Q only after L_{i-1} does. At that moment w is added with $\text{dist}(w) = (i-1) + 1 = i$. Vertices of L_{i+1} and beyond have no edge from L_{i-1}, so they are added only after this. □

10.4 Undirected connectivity

Let G be undirected. Its connected components, the pieces of G, are the equivalence classes of the relation $u \sim v$ if and only if G has a u–v path. This is an equivalence relation: a single vertex is a path, reversing a path shows symmetry, and concatenating paths shows transitivity. Computing the components checks whether a network is disconnected and supports graph visualization and clustering.

Algorithm 10.3: Connected components via BFS

Input: Undirected graph G with vertices labeled $1, \ldots, n$.
Output: The connected components of G.

```
1 Mark every vertex unexplored
2 for i ← 1 to n do
3     if i is not yet explored (in some previous BFS) then
4         BFS(G, i)                    // discovers exactly the component of i
```

Each call discovers precisely the component of i, by Proposition 10.2, so every component is found once. Initialization and the outer loop take $O(n)$ time. Each BFS uses $O(1)$ time per vertex and per edge of its own component, and the components partition V and E. The total running time is therefore $O(m + n)$. For example, on a graph with vertex set $\{1, \ldots, 10\}$ whose components are

$\{1, 3, 5, 7, 9\}$, $\{2, 4\}$, and $\{6, 8, 10\}$, the loop calls BFS from 1, 2, and 6.

10.5 Depth-first search

DFS can mimic the BFS code with a stack in place of the queue, plus minor modifications. The recursive version is shorter.

Algorithm 10.4: Depth-first search, DFS(G, s)

Input: Graph G and a start vertex s.
Output: The vertices reachable from s, marked explored.

```
Mark s explored
for each edge (s, v) do
    if v is unexplored then
        DFS(G, v)
```

Proposition 10.4 [Basic DFS properties]. *At the end of* DFS(G, s)*, a vertex v is explored if and only if G has an s–v path, and the running time is $O(n_s + m_s)$.*

Proof. DFS is a particular instantiation of generic search. It makes at most one call per reachable vertex, and each call scans that vertex's edges once, so each edge is examined at most twice. □

10.6 Topological sort

Definition 10.5 [Topological ordering]. A topological ordering of a directed graph G is a labeling f of its vertices such that

1. the labels $f(v)$ are exactly $\{1, 2, \ldots, n\}$, and
2. $(u, v) \in E$ implies $f(u) < f(v)$.

The motivation is to sequence tasks while respecting all precedence constraints. If G has a directed cycle, it has no topological ordering: the labels would strictly increase around the cycle and return to their starting value. Conversely, if G is a directed acyclic graph (DAG), a topological ordering can be computed in $O(m + n)$ time. A DAG may have several topological orderings.

Lemma 10.6 [Sinks exist]. *Every directed acyclic graph has a sink vertex, one with no outgoing arcs.*

Proof. If no vertex were a sink, we could keep following outgoing arcs from any vertex. After n steps some vertex would repeat, giving a directed cycle. □

A straightforward algorithm follows: let v be a sink of G, set $f(v) = n$, and recurse on $G - \{v\}$, which is still acyclic. When v is assigned position i, all its outgoing arcs have already been deleted,

so they all lead to vertices later in the ordering.

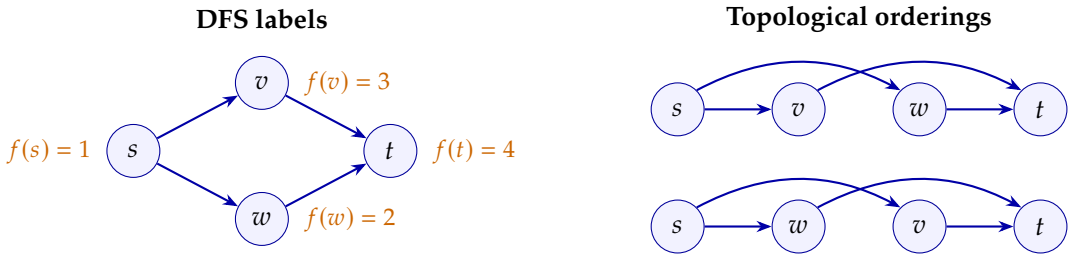


Figure 21: A DAG with edges $(s,v),(s,w),(v,t),(w,t)$. Left: the labels produced by the DFS method below, starting at s and exploring v before w. Right: its two topological orderings, with every arc pointing forward.

DFS gives a slicker linear-time method. Each vertex receives its label when its recursive call finishes, so labels are assigned in decreasing order.

Algorithm 10.5: Topological sort via DFS

```
Input: Directed acyclic graph G.
Output: A topological ordering f.
Mark every vertex unexplored
current_label ← n                                   // to keep track of the ordering
for each vertex v do
    if v is not yet explored (in a previous DFS call) then
        DFS(G, v)
procedure DFS(G, s):
    Mark s explored
    for each edge (s, v) do
        if v is not yet explored then
            DFS(G, v)
    f(s) ← current_label
    current_label ← current_label − 1
```

Theorem 10.7 [Topological sort via DFS]. *On a directed acyclic graph, this algorithm computes a topological ordering in $O(m+n)$ time.*

Proof. The running time is $O(1)$ per vertex and $O(1)$ per edge. Each vertex is labeled once, with labels $n, n-1, \ldots, 1$, so condition 1 holds. For condition 2, let (u,v) be an edge; we show $f(u) < f(v)$. Since labels decrease over time, it suffices that v's call finishes before u's.
Case 1: u is visited before v. When DFS(G,u) starts, v is unexplored, and the call scans the edge (u,v). So v is explored, and its recursive call finishes, before the call for u does. Hence $f(v) > f(u)$.
Case 2: v is visited before u. There is no path from v to u, since with the edge (u,v) it would close a directed cycle. So u is not explored during v's call, which finishes before u's call even starts. Hence $f(v) > f(u)$. □

10.7 Strongly connected components

Definition 10.8 [Strongly connected components]. The strongly connected components (SCCs) of a directed graph G are the equivalence classes of the relation $u \sim v$ if and only if G has both a path from u to v and a path from v to u.

The relation is reflexive and symmetric by definition and transitive by concatenating paths. A single DFS finds everything reachable from its start. Started in an SCC with no arcs leaving it, it discovers exactly that SCC; started elsewhere, it may discover the union of several SCCs. The order in which DFS calls are started therefore matters.

Theorem 10.9 [Kosaraju]. *The SCCs of a directed graph can be computed in $O(m + n)$ time.*

Kosaraju's two-pass algorithm uses one subroutine twice. With the vertices labeled $1, \ldots, n$, a global counter t (the number of vertices finished so far) records finishing times for the first pass, and a global vertex s (the current source) records leaders for the second.

Algorithm 10.6: DFS-Loop

```
Input: Directed graph G with vertices labeled 1, ..., n.
Output: Finishing times f(i) and leaders leader(i).
Mark every vertex unexplored; t ← 0; s ← NULL
for i ← n down to 1 do
    if i is not yet explored then
        s ← i
        DFS(G, i)
procedure DFS(G, i):
    Mark i explored; leader(i) ← s
    for each arc (i, j) ∈ E do
        if j is not yet explored then
            DFS(G, j)
    t ← t + 1; f(i) ← t                                  // i's finishing time
```

Algorithm 10.7: Kosaraju's two-pass algorithm

```
Input: Directed graph G.
Output: The SCCs of G.
G^rev ← G with every arc reversed
Run DFS-Loop on G^rev                                    // computes a good ordering f
Run DFS-Loop on G in decreasing order of f               // discovers the SCCs
return the groups of vertices with a common leader
```

The running time is that of two depth-first searches, $O(m + n)$. Building G^{rev} takes $O(m + n)$, and since the finishing times are a permutation of $1, \ldots, n$, relabeling each vertex by f orders the second pass without sorting.

In this example the finishing times of vertices $1, 2, \ldots, 9$ are $7, 3, 1, 8, 2, 5, 9, 4, 6$. The second pass

starts at vertex 7, the vertex with $f = 9$, and discovers exactly $\{1, 4, 7\}$.

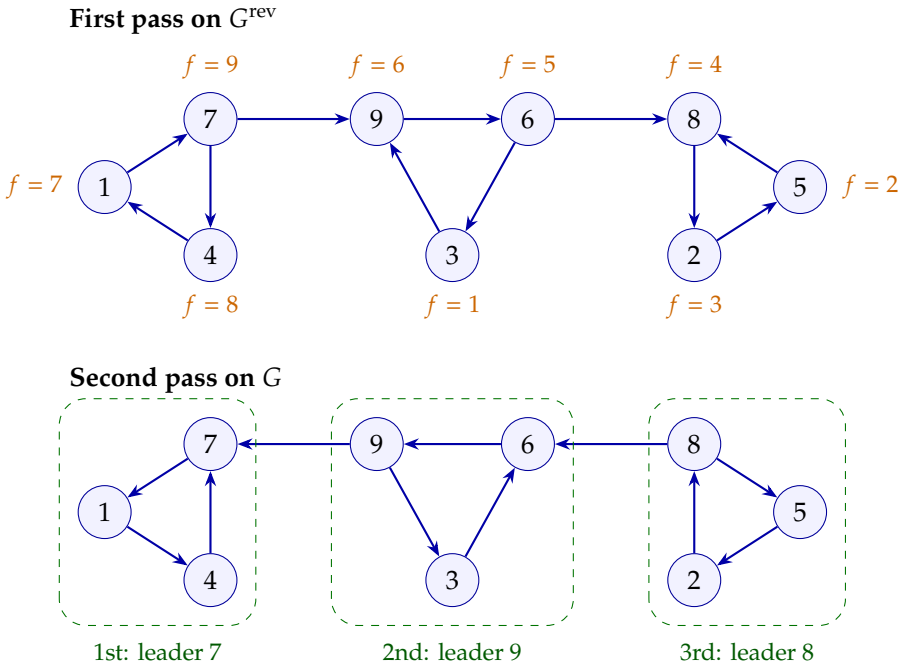


Figure 22: Kosaraju's algorithm on a nine-vertex graph. Top: the first pass on G^{rev}, scanning from vertex 9 down and, from 6, exploring 3 before 8; each vertex shows its finishing time. Bottom: the second pass on G in decreasing order of f discovers the three SCCs in turn, with leaders 7, 9, and 8 (finishing times 9, 6, 4).

10.8 Correctness of Kosaraju's algorithm

The SCCs induce an acyclic META-GRAPH: its nodes are the SCCs $C_1, \ldots, C_k$ of G, with an arc $C \to \hat{C}$ if and only if G has an arc (i, j) with $i \in C$ and $j \in \hat{C}$. It is acyclic because a cycle of SCCs would collapse into a single SCC. Moreover, G and G^{rev} have exactly the same SCCs, since a u–v path in G is a v–u path in G^{rev}.

Lemma 10.10 [KEY LEMMA]. *Let C_1 and C_2 be SCCs of G joined by an arc of G from C_1 to C_2, and let f be the finishing times of DFS-Loop on G^{rev}. Then*

$$\max_{v \in C_1} f(v) < \max_{v \in C_2} f(v).$$

Proof. In G^{rev}, C_1 and C_2 are still SCCs, and the arc points from C_2 to C_1. Let v be the first vertex of $C_1 \cup C_2$ reached by the first pass.
Case 1: $v \in C_1$. There is no path from C_1 to C_2 in G^{rev}, since with the arc from C_2 to C_1 it would create a cycle in the meta-graph. So all of C_1 is explored and finished before C_2 is ever reached, and every f-value in C_1 is less than every f-value in C_2.
Case 2: $v \in C_2$. All of $C_1 \cup C_2$ is reachable from v and unexplored when v is reached, so

DFS(G^{rev}, v) does not finish until all of $C_1 \cup C_2$ has been completely explored. Thus $f(v) > f(w)$ for every $w \in C_1$. □

Corollary 10.11. *The maximum f-value lies in a* SINK SCC *of G, one with no arcs to other SCCs.*

Indeed, if the SCC containing the maximum had an arc to another SCC, that SCC would contain a larger f-value by Lemma 10.10.

Theorem 10.12 [CORRECTNESS OF KOSARAJU'S ALGORITHM]. *In the second pass, each call* DFS(G, i) *from the outer loop explores exactly one SCC, so vertices share a leader if and only if they lie in the same SCC.*

Proof. By Corollary 10.11, the second pass begins in a sink SCC C^*. The first call explores all of C^* and nothing else, since no arc leaves it. The rest of DFS-Loop then behaves like a run on G with C^* deleted.
In general, suppose the SCCs explored so far are each completely explored, and the next call starts at the unexplored vertex i with the largest f-value, lying in SCC C. If C had an arc to an unexplored SCC C', Lemma 10.10 would give a vertex of C' with f-value larger than $\max_{v \in C} f(v) \geq f(i)$, contradicting the choice of i. So every arc leaving C enters an explored SCC, and DFS(G, i) explores exactly C. By induction, successive calls peel off the SCCs one by one, in reverse topological order of the meta-graph. □

10.9 Structure of the Web

In the WEB GRAPH, vertices are Web pages and directed edges are hyperlinks. To ask what this graph looks like, assume it has already been crawled. Broder et al. (WWW 2000) computed the SCCs of a Web graph with roughly 200 million nodes and 1 billion edges, before MapReduce and Hadoop. They found a BOW TIE made of a giant SCC, the core of the Web; IN, pages that can reach the giant SCC but cannot be reached from it, such as new Web pages; OUT, pages reachable from the giant SCC that cannot reach back, such as corporate websites; tubes from IN to OUT that bypass the core; tendrils leaving IN or entering OUT; and disconnected pieces. The main findings were:

1. The four parts (giant SCC, IN, OUT, and tubes plus tendrils) have roughly the same size.
2. Within the core, the graph is very well connected: it has the SMALL WORLD property, in the sense of Milgram.
3. Outside the core, it is surprisingly poorly connected.

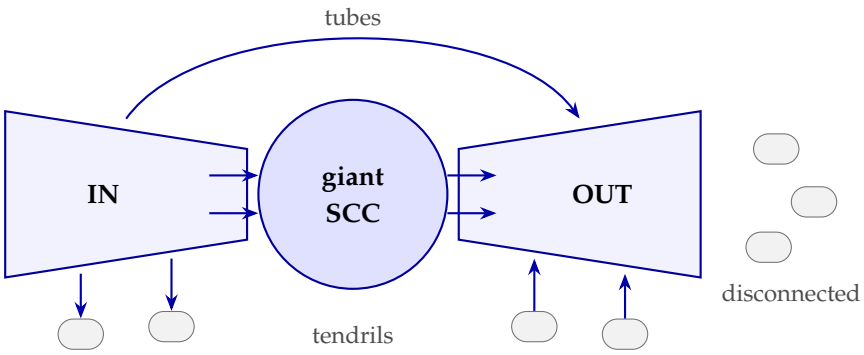


Figure 23: Schematic bow tie of the Web graph. Arrows show the direction in which each part can reach the others; sizes are not to scale.

Later research studies how the Web graph evolves over time, how new information propagates through the Web (or blogs, or Twitter), and how to define and compute communities in information and social networks.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Graph Search and Connectivity.

Lecture 11: Dijkstra's Shortest-Path Algorithm

11.1 Single-source shortest paths

Definition 11.1 [Single-source shortest paths]. The input is a directed graph $G = (V, E)$, with $m = |E|$ and $n = |V|$, a nonnegative length ℓ_e for each edge e, and a source vertex s. The length of a path is the sum of its edge lengths. The output is, for each $v \in V$, the length $L(v)$ of a shortest s–v path in G.

For convenience, assume that every $v \in V$ is reachable from s. The important assumption is $\ell_e \geq 0$ for every $e \in E$.

Breadth-first search already computes shortest paths in linear time, but only if $\ell_e = 1$ for every edge e. One could replace each edge e by a directed path of ℓ_e unit-length edges, but this blows up the graph too much. Dijkstra's algorithm handles general nonnegative lengths directly.

11.2 The algorithm

The algorithm grows a set X of processed vertices by one vertex per iteration. It records computed distances in A and computed shortest paths in B; the array B only helps the explanation and is not needed in an implementation.

Algorithm 11.1: Dijkstra's algorithm

Input: Directed graph $G = (V, E)$, lengths $\ell_e \geq 0$, source s.
Output: Distances $A[v]$ and paths $B[v]$ for every $v \in V$.

```
X ← {s}                                   // vertices processed so far
A[s] ← 0                                  // computed shortest-path distances
B[s] ← the empty path                     // computed shortest paths
while X ≠ V do
    Among all edges (v, w) ∈ E with v ∈ X and w ∉ X, choose the edge (v*, w*) minimizing
      A[v] + ℓ_vw
    Add w* to X
    A[w*] ← A[v*] + ℓ_v*w*
    B[w*] ← B[v*] ∪ (v*, w*)
```

The quantity $A[v] + \ell_{vw}$ is Dijkstra's greedy score of the edge (v, w); $A[v]$ was already computed in an earlier iteration. By the reachability assumption, some edge crosses from X to $V \setminus X$ while $X \neq V$, so each iteration can proceed.

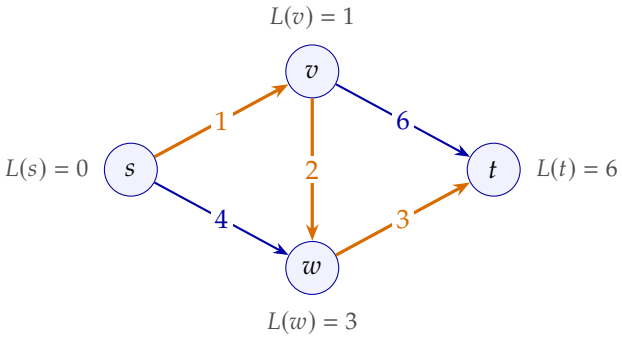


Figure 24: Running example. The shortest-path distances of s, v, w, t are $0, 1, 3, 6$; the highlighted edges form the shortest paths. Dijkstra's algorithm adds s, v, w, t to X in that order.

Example: In the running example, the first iteration compares the scores 1 of (s, v) and 4 of (s, w), and sets $A[v] = 1$ with $B[v] = s \to v$. The second compares 4 for (s, w), 3 for (v, w), and 7 for (v, t), and sets $A[w] = 3$ with $B[w] = s \to v \to w$. The third compares 7 for (v, t) and 6 for (w, t), and sets $A[t] = 6$ with $B[t] = s \to v \to w \to t$.

11.3 Negative edge lengths

One might try to reduce shortest paths with negative edge lengths to the nonnegative case by adding a large constant to every edge length. This does not preserve shortest paths, because paths with more edges are penalized more. Dijkstra's algorithm is also incorrect when lengths can be negative.

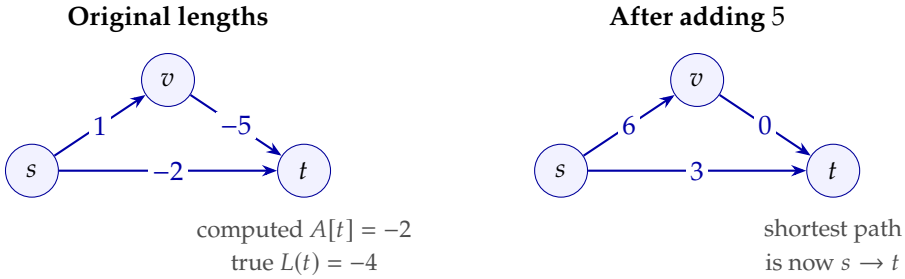


Figure 25: A graph with negative edge lengths. Dijkstra's algorithm computes the s–t distance as -2; the true distance is -4. Adding 5 to every length makes the direct edge, of length 3, shorter than the two-edge path, of length 6.

With $X = \{s\}$, the edge (s, t) has the smaller score -2, so t joins X with $A[t] = -2$ and $B[t] = s \to t$, although the path $s \to v \to t$ has length $1 - 5 = -4$.

11.4 Correctness

Theorem 11.2 [Dijkstra]. *For every directed graph with nonnegative edge lengths, Dijkstra's algorithm correctly computes all shortest-path distances: $A[v] = L(v)$ for every $v \in V$, where $A[v]$ is what the algorithm computes and $L(v)$ is the true shortest distance from s to v.*

Proof. Induct on the number of iterations. In the base case, $A[s] = L(s) = 0$ and $B[s]$ is the empty path.
Inductive hypothesis: all previous iterations were correct, that is, $A[v] = L(v)$ and $B[v]$ is a true shortest s–v path in G for every v already in X.
In the current iteration, the algorithm picks an edge (v^*, w^*) with $v^* \in X$ and $w^* \notin X$, and adds w^* to X. By the inductive hypothesis, $B[v^*]$ has length $L(v^*)$, so

$$A[w^*] = A[v^*] + \ell_{v^*w^*} = L(v^*) + \ell_{v^*w^*},$$

and $B[w^*] = B[v^*] \cup (v^*, w^*)$ is an s–w^* path of exactly this length. It remains to show that every s–w^* path has length at least $L(v^*) + \ell_{v^*w^*}$; then our path is a shortest one.
Let P be any s–w^* path. Since $s \in X$ and $w^* \notin X$, P must cross the frontier: it has a first edge (y, z) with $y \in X$ and $z \notin X$. So P consists of an s–y path, the edge (y, z), and a z–w^* path. Bound the three pieces:

- the s–y part has length at least $L(y) = A[y]$, by the inductive hypothesis since $y \in X$;
- the edge (y, z) has length ℓ_{yz};
- the z–w^* part has length at least 0, since no edge length is negative.

Thus P has length at least $A[y] + \ell_{yz}$. The edge (y, z) was a candidate in this iteration, so by Dijkstra's greedy criterion,

$$A[v^*] + \ell_{v^*w^*} \le A[y] + \ell_{yz} \le \text{length of } P.$$

The left side is the length of our path, which completes the induction. □

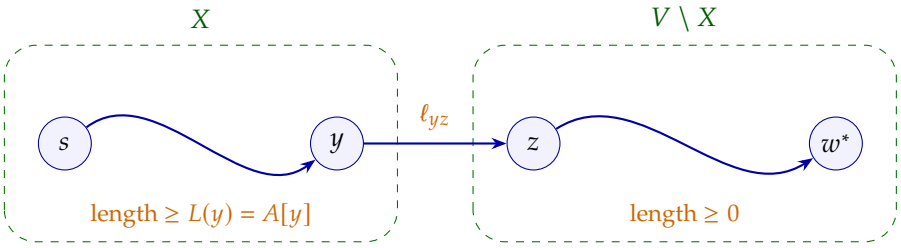


Figure 26: Any s–w^* path P leaves X through some first edge (y, z). Its three pieces have lengths at least $A[y]$, exactly ℓ_{yz}, and at least 0.

11.5 A heap-based implementation

A naive implementation runs $n - 1$ iterations of the main loop, each scanning all edges with $\Theta(1)$ work per edge, so $\Theta(m)$ work per iteration. Its running time is $\Theta(mn)$.

A HEAP performs Insert and Extract-Min in $O(\log n)$ time. Conceptually it is a perfectly balanced binary tree, of height about $\log_2 n$, satisfying the HEAP PROPERTY: at every node, the key is at most the children's keys. Extract-Min swaps the last leaf up to the root and bubbles it down; Insert adds a leaf and bubbles it up. Dijkstra's algorithm also needs to delete an element from the middle of the heap, which bubbles the replacement element up or down as needed, also in $O(\log n)$ time.

The implementation maintains two invariants.

1. The elements in the heap are the vertices of $V \setminus X$.
2. For each $v \notin X$, key$[v]$ is the smallest Dijkstra greedy score $A[u] + \ell_{uv}$ of an edge $(u, v) \in E$ with $u \in X$, or $+\infty$ if no such edge exists.

By the invariants, Extract-Min yields the correct vertex w^* to add to X next, and we set $A[w^*] \leftarrow$ key$[w^*]$: the minimum key is the minimum greedy score over all crossing edges.

When w moves from the heap to X, the edges leaving w may become new crossing edges, so some keys need updating.

Algorithm 11.2: Dijkstra's algorithm with a heap

Input: Directed graph $G = (V, E)$, lengths $\ell_e \geq 0$, source s.
Output: Distances $A[v]$ for every $v \in V$.

```
1  X ← {s}; A[s] ← 0
2  for each v ∈ V \ {s} do
3  |   key[v] ← min{ℓ_sv : (s, v) ∈ E}, or +∞ if there is no such edge
4  |   Insert v into the heap
5  while the heap is nonempty do
6  |   w ← Extract-Min; add w to X; A[w] ← key[w]
7  |   for each edge (w, v) ∈ E do
8  |   |   if v ∈ V \ X (that is, v is in the heap) then
9  |   |   |   Delete v from the heap
10 |   |   |   key[v] ← min{key[v], A[w] + ℓ_wv}        // greedy score of (w, v)
11 |   |   |   Re-insert v into the heap
```

The initial keys satisfy invariant 2 for $X = \{s\}$. After w joins X, the only new crossing edges into a vertex v of the heap are edges (w, v), and the key update accounts for exactly these. So both invariants hold after each iteration.

Theorem 11.3 [Running time]. *Dijkstra's algorithm with a heap runs in $O(m \log n)$ time.*

Proof. The running time is dominated by heap operations, each taking $O(\log n)$ time. There are $n-1$ initial insertions and $n-1$ Extract-Min operations. Each edge (v, w) triggers at most one Delete and Insert pair, when v is added to X first. So the number of heap operations is $O(n+m)$. Every vertex is reachable from s, so the graph is weakly connected and $m \geq n-1$; hence $O(n+m) = O(m)$. The total time is $O(m \log n)$, like sorting. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Dijkstra's Shortest-Path Algorithm.

Lecture 12: Heaps

A data structure organizes data so that it can be accessed quickly and usefully. Lists, stacks, queues, heaps, search trees, hash tables, Bloom filters, and union-find are examples. Different data structures support different sets of operations, so each suits different tasks. A good rule of thumb is to choose the minimal data structure that supports all the operations you need.

12.1 Heap operations

A heap is a container for objects that have keys, such as employee records, network edges, or events. With n the number of objects currently in the heap, it supports:

- *Insert*: add a new object, in $O(\log n)$ time;
- *Extract-Min*: remove an object with a minimum key, ties broken arbitrarily, in $O(\log n)$ time.

A heap can equally well support Extract-Max instead of Extract-Min. Two further operations are *Heapify*, which performs n batched inserts in $O(n)$ time, and *Delete*, which removes an arbitrary object in $O(\log n)$ time. A heap is also called a priority queue.

12.2 Applications

The canonical use of a heap is to perform repeated minimum computations quickly.

Sorting. Selection sort performs $\Theta(n)$ linear scans, each finding the minimum of the remaining elements, so it takes $\Theta(n^2)$ time on an array of length n. Replacing the scans by a heap gives heap sort.

Algorithm 12.1: Heap sort

Input: Array $A[1 : n]$.
Output: The entries of A in increasing order.

```
H ← an empty heap
for i ← 1 to n do
    Insert A[i] into H
for i ← 1 to n do
    B[i] ← ExtractMin(H)
return B
```

The running time is $2n$ heap operations, each $O(\log n)$, so $O(n \log n)$ in total. This is optimal for a comparison-based sorting algorithm.

Event manager. In a simulation, for example of a video game, the objects are event records, each an

action or update scheduled for a given future time. The key of an event is the time it is scheduled to occur, so Extract-Min yields the next scheduled event.

Median maintenance. Numbers $x_1, \ldots, x_n$ arrive one by one. At each step i we must report the median of $\{x_1, \ldots, x_i\}$, here the $\lceil i/2 \rceil$-th smallest, using $O(\log i)$ time per step. Maintain two heaps: H_{low} supporting Extract-Max and H_{high} supporting Extract-Min. The invariant is that H_{low} holds roughly the $i/2$ smallest elements and H_{high} the $i/2$ largest. Precisely, every element of H_{low} is at most every element of H_{high}, and the two sizes differ by at most one.

Algorithm 12.2: Median maintenance, step i

Input: Heaps H_{low} and H_{high} satisfying the invariant for $x_1, \ldots, x_{i-1}$, and the new number x_i.
Output: The median of $x_1, \ldots, x_i$.

```
if H_low is empty or x_i ≤ max H_low then
    Insert x_i into H_low
else
    Insert x_i into H_high
if |H_low| > |H_high| + 1 then
    Insert ExtractMax(H_low) into H_high
if |H_high| > |H_low| + 1 then
    Insert ExtractMin(H_high) into H_low
if |H_low| ≥ |H_high| then return max H_low
else return min H_high
```

The new element joins the correct side of the split, and moving the maximum of H_{low} (or the minimum of H_{high}) across keeps every low element below every high element, so the invariant is restored. If the two heaps have equal size, $\max H_{\text{low}}$ is the $(i/2)$-th smallest element. Otherwise the larger heap has $\lceil i/2 \rceil$ elements and its root is the $\lceil i/2 \rceil$-th smallest. Reading a root can be done by an extract followed by a re-insert. Each step uses a constant number of heap operations on at most i elements, hence $O(\log i)$ time.

Speeding up Dijkstra. For a graph with n vertices and m edges, the naive implementation of Dijkstra's shortest-path algorithm has n loop iterations, each a linear scan through the edges to compute a minimum, for a running time of $\Theta(nm)$. With heaps, the running time is $O(m \log n)$.

12.3 The heap property

Conceptually, a heap is a rooted binary tree that is as complete as possible: every level is full except possibly the last, which is filled from the left.

Definition 12.1 [Heap property]. At every node x, the key of x is at most the keys of the children of x.

Consequently, the object at the root has a minimum key. Indeed, following parent pointers from any node to the root never increases the key, so every key is at least the root's key. A given set of

keys usually has several valid heaps.

The tree needs no pointers. Store level 0 (the root), then level 1, level 2, and so on, each from left to right, in an array $A[1:n]$. Level ℓ occupies positions 2^ℓ through $2^{\ell+1}-1$, so

$$\text{parent}(i) = \lfloor i/2 \rfloor, \qquad \text{children}(i) = 2i,\ 2i+1.$$

That is, the parent of i is $i/2$ if i is even and $(i-1)/2$ if i is odd. Since every level but the last is full, a heap with n objects has $\lfloor \log_2 n \rfloor + 1$ levels, about $\log_2 n$.

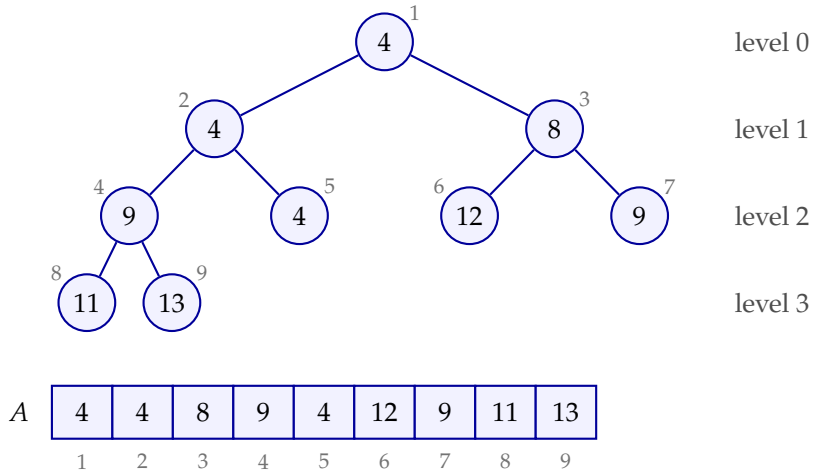


Figure 27: A heap with nine keys, drawn as a tree with each node's array position in gray, and stored as the array A. The children of position i are at positions $2i$ and $2i+1$.

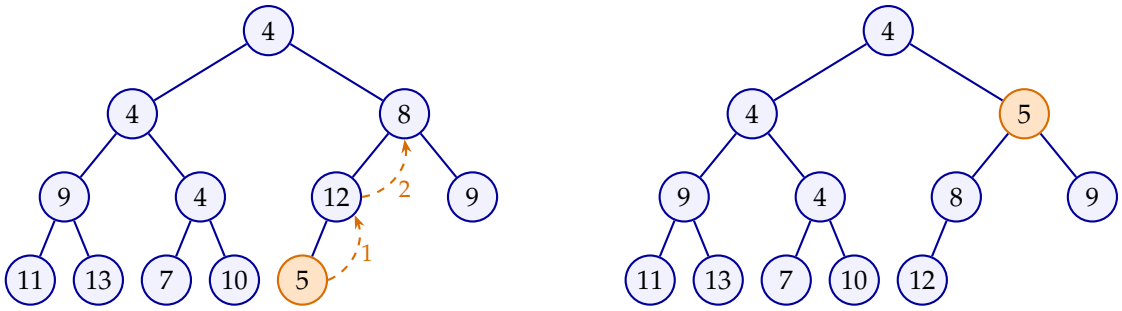
12.4 Insert and bubble-up

To insert a key k, place it at the end of the last level, that is, at position $n+1$ of the array. This can break the heap property only between k and its parent. Bubble-up repeatedly swaps k with its parent until the parent's key is at most k, or k reaches the root.

Algorithm 12.3: Insert

Input: Heap $A[1:n]$ and a new key k.
Output: Heap $A[1:n+1]$ containing k.

$n \leftarrow n+1$; $A[n] \leftarrow k$; $i \leftarrow n$
while $i > 1$ **and** $A[\lfloor i/2 \rfloor] > A[i]$ **do**
    Swap $A[i]$ and $A[\lfloor i/2 \rfloor]$
    $i \leftarrow \lfloor i/2 \rfloor$



(a) 5 placed at the end

(b) after two swaps

Figure 28: Inserting 5 into the heap above, after 7 and 10 were inserted without any swaps. The new key swaps with 12, then with 8, and stops below the root, since $4 \le 5$.

Bubble-up stops, since k rises one level per swap. To see that it restores the heap property, let $x_0 \le x_1 \le \cdots \le x_{d-1}$ be the keys on the path from the root to the new leaf before the insertion. Bubble-up inserts k into this sorted sequence: each $x_i > k$ shifts down one node, and k fills the node vacated by the smallest of them, x_j. A shifted key x_i takes the place of x_{i+1}, so its new children are x_{i+1} and an off-path key that was already at least $x_{i+1} \ge x_i$. Similarly, k sits below $x_{j-1} \le k$ and above keys at least $x_j > k$. With about $\log_2 n$ levels and constant work per swap, Insert takes $O(\log n)$ time.

12.5 Extract-Min and bubble-down

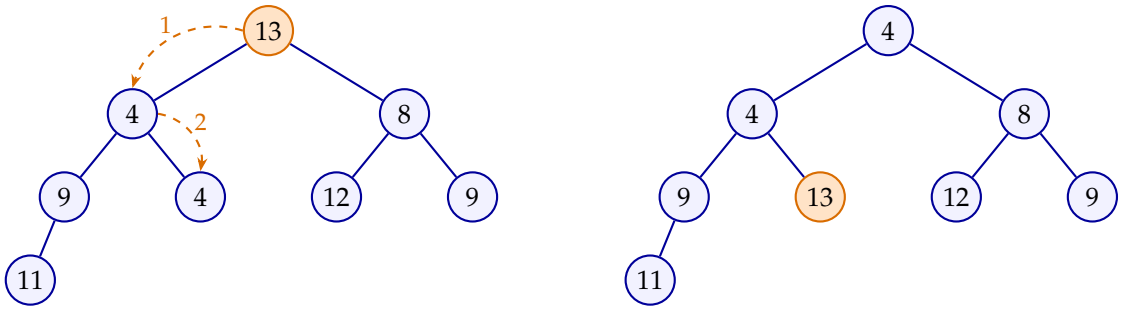
To extract the minimum, delete the root, move the last leaf to be the new root, and iteratively BUBBLE DOWN: swap the moved key with its *smaller* child until the heap property is restored.

Algorithm 12.4: Extract-Min

Input: Nonempty heap $A[1:n]$.
Output: A minimum key; $A[1:n-1]$ remains a heap.

```
m ← A[1]; A[1] ← A[n]; n ← n − 1; i ← 1
while 2i ≤ n do
    c ← 2i
    if 2i + 1 ≤ n and A[2i + 1] < A[2i] then c ← 2i + 1
    if A[i] ≤ A[c] then break
    Swap A[i] and A[c]; i ← c
return m
```

Swapping with the smaller child c is essential. After the swap, c's key sits above the moved key (larger than it) and above its former sibling (at least it), so the heap property holds at that node. Swapping with the larger child would place it above a smaller key. The only possible violation is between the moved key and its new children, one level lower. Hence there is at most one swap per level, the process halts with a heap, and Extract-Min takes $O(\log n)$ time.



(a) last leaf 13 moved to the root

(b) after two swaps

Figure 29: Extract-Min on the nine-key heap. After removing the root 4, the last leaf 13 moves to the root and swaps with its smaller child 4 twice, stopping at a leaf.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Heaps.

Lecture 13: Balanced Binary Search Trees

A sorted array supports many operations quickly, but not changes to the set of keys. A balanced search tree keeps those operations fast while also allowing logarithmic-time insertions and deletions.

13.1 Sorted arrays and balanced search trees

For a sorted array of n keys, *Select* returns the i-th order statistic, *Pred* and *Succ* take a pointer to a key and return its neighbors, and *Rank* returns the number of keys less than or equal to a given value. Inserting or deleting a key, however, takes $\Theta(n)$ time, since entries must shift. A balanced search tree is like a sorted array with fast inserts and deletes:

Operation	Sorted array	Balanced search tree
Search	$\Theta(\log n)$	$O(\log n)$
Select	$O(1)$	$O(\log n)$
Min / Max	$O(1)$	$O(\log n)$
Pred / Succ	$O(1)$	$O(\log n)$
Rank	$O(\log n)$	$O(\log n)$
Output in sorted order	$O(n)$	$O(n)$
Insert / Delete	$\Theta(n)$	$O(\log n)$

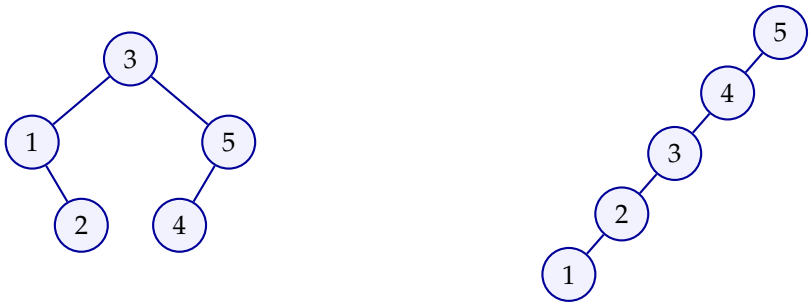
Select, Min/Max, and Pred/Succ become slower, from $O(1)$ to $O(\log n)$, in exchange for fast Insert and Delete. Hash tables also support Search, Insert, and Delete, and heaps support Insert, Delete, and Min (or Max); a search tree supports all of these operations at once.

13.2 Binary search tree structure

A binary search tree has exactly one node per key. In the most basic version, each node has a left child pointer, a right child pointer, and a parent pointer, any of which may be NULL.

Definition 13.1 [Search tree property]. At every node x, all keys in the left subtree of x are less than the key of x, and all keys in the right subtree of x are greater.

A set of keys has many possible search trees. The height, or depth, of a tree is the number of edges on a longest root-leaf path. For n keys it can be anywhere from about $\log_2 n$, when the tree is perfectly balanced, to $n - 1$, when it is a chain.



(a) height 2 (b) height 4

Figure 30: Two search trees on the keys $1, \ldots, 5$. Every key in a left subtree is smaller, and every key in a right subtree larger, than the key above it.

13.3 Search, insert, min, max, and predecessor

To *search* for a key k, start at the root and repeatedly follow the left child pointer if k is less than the current key, or the right child pointer if k is greater. Return the node with key k, or NULL if the search falls off the tree.

To *insert* a new key k, search for k, which fails, and rewire the final NULL pointer to a new node with key k. This preserves the search tree property: at each ancestor, the search went left exactly when k is smaller, so k lies on the correct side of every ancestor.

Each of these operations follows one root-leaf path, so its worst-case running time is $\Theta(\text{height})$. On a chain, this is $\Theta(n)$.

To find the *minimum* (*maximum*) key, start at the root and follow left (right) child pointers until this is no longer possible; the last key found is the answer.

To find the *predecessor* of a key k, the largest key smaller than k:

- if k's left subtree is nonempty, return the maximum key in it;
- otherwise, follow parent pointers until reaching a key less than k, which happens the first time the path turns left (moves up from a right child). If no such key exists, k is the minimum.

In the balanced tree of the figure above, the predecessor of 3 is 2, the maximum of its left subtree, and the predecessor of 4 is 3, reached by moving up through 5. The successor is symmetric.

Proposition 13.2 [Predecessor rule]. *Both cases of this procedure return the predecessor of k.*

Proof. First, a node v outside the subtree of a node u is either smaller than every key in that subtree or larger than all of them: at the lowest common ancestor of u and v, the two lie on

different sides, or v is that ancestor. So in the first case, a key strictly between the maximum of k's left subtree and k would lie in k's subtree; but the left subtree has no larger key and the right subtree no smaller one. In the second case, let a be the first ancestor with a smaller key. Below a, the path to k starts at a's right child and then makes only left moves, and k has no left child, so k is the minimum of a's right subtree. By the mirror image of the first case, that minimum is the successor of a; hence a is the predecessor of k. □

13.4 In-order traversal and deletion

Algorithm 13.1: In-order traversal

Input: The root r of a search tree, with subtrees T_L and T_R.
Output: The keys of the tree, printed in increasing order.

```
1 if r = NULL then return
2 Recurse on T_L
3 Print the key of r
4 Recurse on T_R
```

By induction, the recursive calls print the keys of T_L, all smaller than r's key, and the keys of T_R, all larger, in increasing order. There is $O(1)$ work per call and one call per node (plus NULL children), so the traversal takes $O(n)$ time.

To *delete* a key k, first search for it. Then:

- *Easy case* (k's node has no children): remove the node.
- *Medium case* (k's node has one child): splice out the node; its unique child takes the position previously held by k's node.
- *Difficult case* (k's node has two children): compute k's predecessor ℓ by following k's left child pointer and then right child pointers until no longer possible. Swap k and ℓ. In its new position k has no right child, so delete or splice it out by the easy or medium case.

The result is a valid search tree. In the difficult case, ℓ is the largest key in k's left subtree, so in k's old position it exceeds every remaining key of the left subtree and, being less than k, is below every key of the right subtree. For example, in the balanced tree above, deleting 2 is the easy case, deleting 5 moves 4 into its place, and deleting 3 swaps it with 2 and then removes a leaf. Deletion takes Θ(height) time.

13.5 Select and rank

The idea is to store a little extra information at each node about the tree itself, not about the data. A useful augmentation is size(x), the number of nodes in the subtree rooted at x. If x has children y

and z, then size(x) = size(y) + size(z) + 1, counting the left subtree, the right subtree, and x itself (a missing child has size 0). Sizes are easy to keep up to date during insertion and deletion, since only the nodes on the affected root-leaf path change.

Algorithm 13.2: Select the i-th order statistic

Input: Root x of a size-augmented search tree, and $1 \le i \le$ size(x).
Output: The i-th smallest key in the subtree of x.

```
Let y and z be the left and right children of x
a ← size(y)                                   // a = 0 if x has no left child
if a = i − 1 then return the key of x
if a ≥ i then return Select(y, i)
return Select(z, i − a − 1)
```

The left subtree holds the a smallest keys and the right subtree the larger ones, so the answer is x, in the left subtree, or in the right subtree with $a + 1$ smaller keys skipped. For example, in the balanced tree above, the 4th smallest key goes right at 3 (with $a = 2$), asks for the 1st smallest under 5, goes left (with $a = 1$), and returns 4. Select takes Θ(height) time.

Rank is similar. Search for the given value; each time the search moves right from a node x, or finds the value at x, add size(left child of x) + 1: these are x and the keys below it that are smaller. The total counts exactly the keys at most the value, also in Θ(height) time.

13.6 Red-black trees

A BALANCED SEARCH TREE ensures that its height is always $O(\log n)$, which is best possible. Then Search, Insert, Delete, Min, Max, Pred, and Succ all run in $O(\log n)$ time. Red-black trees (Bayer 1972; Guibas and Sedgewick 1978) are one example; AVL trees, splay trees, and B-trees are others.

Definition 13.3 [RED-BLACK TREE]. A RED-BLACK TREE is a binary search tree satisfying four invariants:

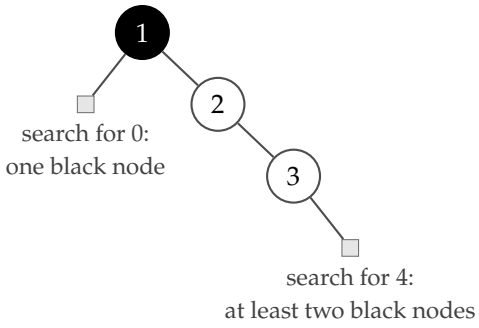
(1) each node is red or black;

(2) the root is black;

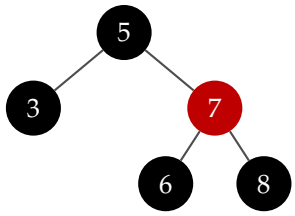
(3) there are no two reds in a row: a red node has only black children;

(4) every root-NULL path, as in an unsuccessful search, has the same number of black nodes.

Example: A chain of three keys 1, 2, 3 cannot be a red-black tree. The root 1 is black, so an unsuccessful search for 0 passes exactly one black node. An unsuccessful search for 4 passes 1, 2, and 3; since 2 and 3 cannot both be red, it passes at least two black nodes, violating invariant (4). By contrast, the tree with root 5, children 3 and 7, and 7's children 6 and 8 is a red-black tree when 7 is red and the rest are black: every root-NULL path has two black nodes.



(a) a chain of three keys



(b) a red-black tree

Figure 31: (a) The two unsuccessful searches, ending at the gray NULL pointers, pass different numbers of black nodes however 2 and 3 are colored. (b) A valid coloring: every root-NULL path has two black nodes.

Theorem 13.4 [HEIGHT GUARANTEE]. *Every red-black tree with n nodes has height at most $2\log_2(n+1)$.*

Proof. Let k be the minimum number of nodes on a root-NULL path. Then the top k levels of the tree are full: it contains a perfectly balanced search tree of depth $k-1$. Hence

$$n \geq 2^k - 1, \qquad \text{so} \qquad k \leq \log_2(n+1).$$

Thus some root-NULL path has at most $\log_2(n+1)$ nodes, and in particular at most $\log_2(n+1)$ black nodes. By invariant (4), every root-NULL path has at most $\log_2(n+1)$ black nodes. By invariants (2) and (3), each red node on a path has a black parent on the same path, so reds are at most as many as blacks. Every root-NULL path therefore has at most $2\log_2(n+1)$ nodes, which bounds the height. □

Search, Min, Max, Pred, Succ, Select, and Rank do not modify the tree, so they run unchanged in $O(\log n)$ time. Insert and Delete must be reimplemented so that the invariants are maintained.

13.7 Rotations

The key primitive, common to all balanced search tree implementations (red-black, AVL, B+, and others), is the ROTATION: it locally rebalances the subtrees at a node in $O(1)$ time. A left rotation of a parent x and its right child y (so $x < y$) makes y the child of x's former parent p and makes x the left child of y. The left subtree A of x stays with x, the right subtree C of y stays with y, and the subtree B of keys between x and y moves from y's left to x's right.

A right rotation of a parent and its left child is the mirror image. Both maintain the search tree property, since the in-order sequence A, x, B, y, C is unchanged, and both rewire a constant number of pointers, so they take $O(1)$ time.

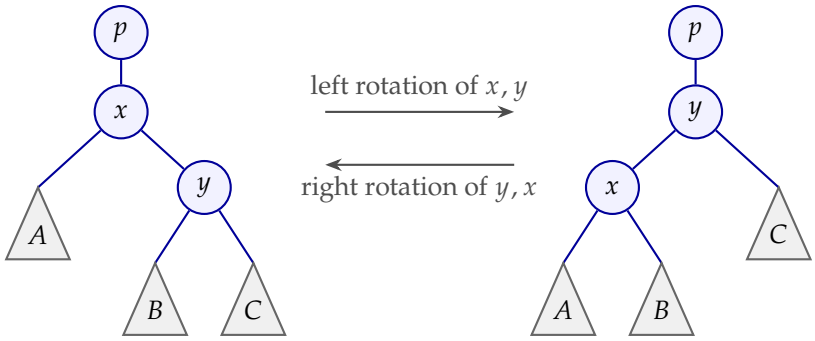


Figure 32: A left rotation and its inverse right rotation. Subtree A holds the keys less than x, B the keys between x and y, and C the keys greater than y.

13.8 Insertion into a red-black tree

The idea for Insert and Delete is to proceed as in an ordinary binary search tree, then recolor and perform rotations until the invariants are restored. To insert x:

1. Insert x as usual, which makes x a leaf.
2. Color x red. This preserves invariant (4), since it adds no black node to any root-NULL path.
3. If x's parent y is black, all invariants hold; done.
4. Otherwise y is red, a double red violating invariant (3). Then y is not the root, and its parent w is black.

In general, let x, y be the current double red with x the deeper node, and let w be x's grandparent, which is black. Let z be w's other child, if any.

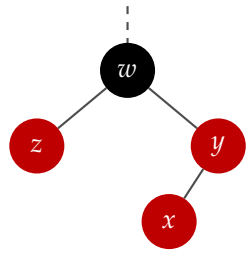
Case 1: z is red. Recolor y and z black and w red. Every root-NULL path through w loses the black w and gains exactly one of y and z, so invariant (4) still holds. Either invariant (3) is restored, or w and its parent form a new double red, two levels higher. This can happen only $O(\log n)$ times. If the double red reaches the root, recolor the root black, which adds one black node to every root-NULL path and so preserves invariant (4).

Case 2: z is black or NULL. The double red can be eliminated in $O(1)$ time by rotations and recolorings, after which all invariants are satisfied. Let B, C, and D be the remaining subtrees below x and y, as in the figure. Rearrange w, y, x so that the middle key of the three is on top, colored black, with the other two as its red children:

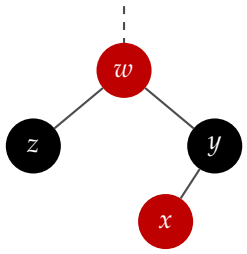
- if x and y are both left or both right children, rotate w and y, putting y on top;
- otherwise, as in the figure, first rotate y and x, which reduces to the previous situation with x in place of y, then rotate w and x.

The new top is black and its children are red, while every subtree that hangs below them has a

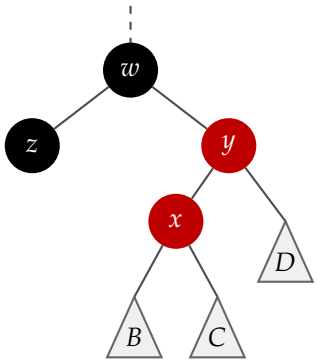
black root or is empty: z is black or NULL, and the children of the red nodes x and y were black or NULL. So there is no double red, and every root-NULL path passes the same number of black nodes as before, one at the top. If w was the root, the new top is a black root.



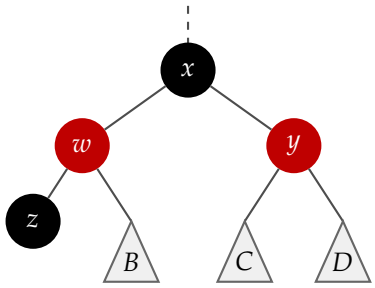
(a) Case 1: z red



(b) after recoloring y, z, w



(c) Case 2: z black or NULL



(d) after two rotations and recoloring

Figure 33: Fixing a double red x, y with grandparent w and uncle z. Case 1 recolors and may push the double red up to w and its parent. Case 2 restructures with x in the middle on top, keeping the in-order sequence z, w, B, x, C, y, D.

Since Case 1 repeats at most $O(\log n)$ times and Case 2 occurs at most once, insertion takes $O(\log n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Balanced Binary Search Trees.

Lecture 14: Hashing: The Basics

A hash table maintains a possibly evolving set of records, such as transactions, people with associated data, or IP addresses. Each record is identified by a key, and the table supports three operations:

- *Insert*: add a new record;
- *Delete*: remove an existing record;
- *Lookup*: check for a particular record.

A structure supporting these operations is a dictionary. A hash table performs all three in $O(1)$ time, provided it is properly implemented and the data are not pathological. Both caveats are explained below and in Lecture 15.

14.1 Applications

Hash tables are useful whenever an algorithm repeatedly asks whether it has seen something before.

Example: De-duplication. Objects arrive in a stream, from a linear scan through a huge file or in real time. The goal is to keep track of the unique objects, for example to report unique visitors to a web site or to avoid duplicates in search results.

Algorithm 14.1: De-duplication with a hash table

Input: A stream of objects.
Output: The distinct objects of the stream, stored in H.

```
H ← an empty hash table
for each arriving object x do
    if Lookup(H, x) fails then
        Insert(H, x)
```

Each arrival costs one Lookup and at most one Insert, so the work per object is constant.

Definition 14.1 [2-SUM]. Given an unsorted array A of n integers and a target sum t, determine whether A contains two numbers x, y with $x + y = t$.

Exhaustive search over all pairs takes $\Theta(n^2)$ time. Sorting A in $\Theta(n \log n)$ time and then binary searching for $t - x$ for each x in A takes another $\Theta(n \log n)$ time. A hash table removes the logarithmic factors.

Algorithm 14.2: 2-SUM with a hash table

Input: Array $A[1:n]$ of integers and target t.
Output: Whether some x, y in A satisfy $x + y = t$.

```
H ← an empty hash table
for i ← 1 to n do
    Insert(H, A[i])
for i ← 1 to n do
    if Lookup(H, t − A[i]) succeeds then
        return true
return false
```

Both loops take $\Theta(n)$ time. If x and y must occupy different positions, a match with $t - x = x$ also requires x to occur twice in A, which a count stored with each key detects.

Further applications include symbol tables in compilers (the historical application), blocking network traffic, and search algorithms such as game tree exploration. In the last case, a hash table of visited configurations, such as arrangements of chess pieces, prevents exploring any configuration more than once.

14.2 Buckets and hash functions

Let U be a UNIVERSE of possible keys, such as all IP addresses, all names, or all chessboard configurations. It is generally very large. The goal is to maintain an evolving set $S \subseteq U$, which is generally of reasonable size.

There are two naive solutions. An array indexed by the elements of U supports $O(1)$ operations but uses $\Theta(|U|)$ space. A list of the elements of S uses $\Theta(|S|)$ space but takes $\Theta(|S|)$ time per Lookup. A hash table achieves the space of the list and the speed of the array:

1. pick a number n of BUCKETS with n comparable to $|S|$, assuming for simplicity that $|S|$ does not vary much;
2. choose a HASH FUNCTION $h : U \to \{0, 1, \ldots, n-1\}$;
3. use an array A of length n, and store x in $A[h(x)]$.

Definition 14.2 [COLLISION]. A COLLISION is a pair of distinct keys $x, y \in U$ with $h(x) = h(y)$.

Collisions are unavoidable in practice, even when keys are assigned to buckets uniformly at random. The BIRTHDAY PARADOX shows this with days of the year as buckets and people with uniformly random birthdays.

The probability that two of them share a birthday is

Number of people	Probability of a shared birthday
23	at least 50%
57	about 99%
184	99.99 . . . %
367	100%

The last row is the pigeonhole principle, allowing for February 29. Thus a collision is likely long before the number of objects approaches the number of buckets.

14.3 Resolving collisions

There are two standard ways to store colliding keys.

Definition 14.3 [Chaining]. In separate chaining, each bucket holds a linked list. Insert, Delete, and Lookup for a key x are performed in the list stored in $A[h(x)]$.

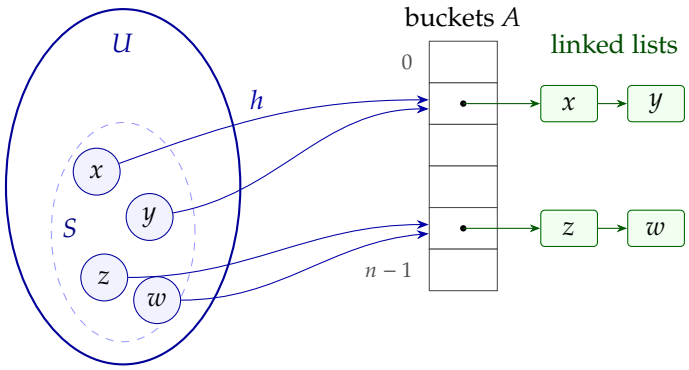


Figure 34: Separate chaining. The hash function sends each key of S to a bucket of A; colliding keys share the linked list of their bucket.

Definition 14.4 [Open addressing]. In open addressing, each bucket holds at most one object. The hash function specifies a probe sequence $h_1(x), h_2(x), \ldots$ for each key, and an operation tries these buckets in order until it finds an open slot (for Insert) or the key (for Lookup).

Linear probing looks at consecutive buckets after the first probe. Double hashing uses two hash functions, one to choose the first bucket and one to choose the step between probes.

14.4 What makes a good hash function

With chaining, Insert takes $\Theta(1)$ time, since a new object goes at the front of the list in $A[h(x)]$. Lookup and Delete take time proportional to the length of that list. For m objects, the longest list

could be anywhere from m/n, when all lists have equal length, to m, when all objects land in the same bucket. Performance therefore depends on the choice of hash function; open addressing behaves analogously.

A good hash function should

1. lead to good performance, that is, spread the data out; the gold standard is completely random hashing;
2. be easy to store and very fast to evaluate.

Example: BAD HASH FUNCTIONS. Take ten-digit phone numbers as keys, so $|U| = 10^{10}$, and choose $n = 10^3$ buckets. Using the first three digits (the area code) is terrible, since many keys share an area code. Using the last three digits is mediocre: it is still vulnerable to patterns in those digits. For memory locations as keys, which are multiples of a power of 2, the function $h(x) = x \bmod 1000$ (again $n = 10^3$) leaves every odd bucket empty, because an even x has an even remainder modulo 1000.

A quick-and-dirty hash function composes two maps. A HASH CODE converts an object to an integer, for example a subroutine converting strings to integers. A COMPRESSION FUNCTION, such as $z \mapsto z \bmod n$, then maps the integer to a bucket.

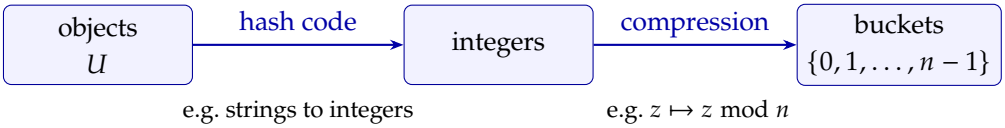


Figure 35: A quick-and-dirty hash function in two stages: a hash code maps objects to integers, and a compression function maps integers to buckets.

The number of buckets n should be chosen

1. prime, and within a constant factor of the number of objects in the table;
2. not too close to a power of 2;
3. not too close to a power of 10.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Hashing: The Basics.

Lecture 15: Universal Hashing

A hash table gives constant-time operations only when its load is controlled and its hash function spreads the data out. No single hash function spreads every data set; choosing the function at random from a suitable family does so for every data set in expectation.

15.1 The load of a hash table

Definition 15.1 [Load factor]. The load factor of a hash table is

$$\alpha = \frac{\text{number of objects in the table}}{\text{number of buckets}}.$$

Only chaining remains feasible for $\alpha > 1$: open addressing stores at most one object per bucket, so it requires $\alpha \leq 1$. Two further observations follow.

1. $\alpha = O(1)$ is necessary for operations to run in constant time. With chaining, some list has length at least α.
2. With open addressing, one needs $\alpha \ll 1$.

Good performance therefore requires controlling the load. Deletion is also easier and more common with chaining than with open addressing.

15.2 Pathological data sets

Good performance also requires a good hash function. Ideally, a single clever hash function would spread every data set out evenly. No such function exists.

Proposition 15.2 [Every hash function has a pathological data set]. *For every hash function $h : U \to \{0, 1, \ldots, n-1\}$, some bucket i receives at least $|U|/n$ elements of U.*

Proof. The n buckets partition U into the preimages $h^{-1}(0), \ldots, h^{-1}(n-1)$, whose sizes sum to $|U|$. By the pigeonhole principle, at least one has size at least $|U|/n$. A data set drawn only from that preimage has every object in the same bucket, so every pair collides. □

Since U is generally much larger than n, such a data set can be as large as desired. Pathological data arise in practice: Crosby and Wallach (USENIX 2003) showed that several real-world systems, such as network intrusion detection, can be paralyzed by exploiting badly designed hash functions.

Their targets were open source and used overly simplistic hash functions, so pathological data sets were easy to reverse engineer.

There are two remedies.

1. Use a cryptographic hash function, such as SHA-2, for which reverse engineering a pathological data set is infeasible.
2. Use randomization: design a family $\mathcal{H}$ of hash functions such that for every data set S, almost all functions $h \in \mathcal{H}$ spread S out pretty evenly.

The second remedy resembles the QuickSort guarantee: the randomness lies in the algorithm, and the guarantee holds for every input. The rest of this lecture develops it.

15.3 Universal families of hash functions

Definition 15.3 [Universal family]. Let $\mathcal{H}$ be a set of hash functions from U to $\{0, 1, \ldots, n-1\}$, where n is the number of buckets. Then $\mathcal{H}$ is universal if for all distinct $x, y \in U$,

$$\mathbb{P}_{h \in \mathcal{H}}\big[h(x) = h(y)\big] \leq \frac{1}{n},$$

where h is chosen uniformly at random from $\mathcal{H}$.

The collision probability of each pair is thus as small as with the gold standard of perfectly random hashing. A weaker per-key condition does not suffice.

Example: Suppose that for every bucket i and key k, exactly a $1/n$ fraction of the functions in $\mathcal{H}$ map k to i. Then $\mathcal{H}$ may or may not be universal. The family of all functions from U to $\{0, 1, \ldots, n-1\}$ has this property and is universal. The family of the n constant functions also has it, but every pair of keys collides with probability 1.

15.4 A universal family for IP addresses

Let U be the set of IP addresses $x = (x_1, x_2, x_3, x_4)$ with each $x_i \in \{0, 1, \ldots, 255\}$. Let n be a prime, for example a small multiple of the number of objects in the table, with $n > 255$. For each $a = (a_1, a_2, a_3, a_4)$ with every $a_i \in \{0, 1, \ldots, n-1\}$, define

$$h_a(x_1, x_2, x_3, x_4) = \big(a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4\big) \bmod n,$$

and let $\mathcal{H} = \{h_a : a \in \{0, 1, \ldots, n-1\}^4\}$. There are n^4 such functions. Each is stored as four numbers and evaluated with four multiplications.

Theorem 15.4 [Universality of the IP-address family]. *The family $\mathcal{H}$ is universal.*

Proof. Let x and y be distinct IP addresses. They differ in some coordinate; by symmetry, assume $x_4 \neq y_4$. Rearranging, x and y collide under h_a if and only if

$$a_4(x_4 - y_4) \equiv \sum_{i=1}^{3} a_i(y_i - x_i) \pmod{n}. \tag{15.1}$$

Condition on arbitrary values of a_1, a_2, a_3, so the right-hand side is some fixed number in $\{0, 1, \ldots, n-1\}$, while a_4 is still uniformly random.
Key claim: the left-hand side is equally likely to be any of $0, 1, \ldots, n-1$. Write $d = x_4 - y_4$. Since $0 < |d| \leq 255 < n$, we have $d \not\equiv 0 \pmod{n}$. If $a_4 d \equiv a_4' d \pmod{n}$, then n divides $(a_4 - a_4')d$; as n is prime and does not divide d, it divides $a_4 - a_4'$, so $a_4 = a_4'$. Hence $a_4 \mapsto a_4 d \bmod n$ is a bijection of $\{0, 1, \ldots, n-1\}$, and a uniform a_4 gives a uniform left-hand side.
Exactly one value of a_4 therefore satisfies equation (15.1), so the conditional collision probability is $1/n$. This holds for every choice of a_1, a_2, a_3, hence $\mathbb{P}[h_a(x) = h_a(y)] = 1/n$. □

Example: With $n = 7$, the multiples $a_4 d \bmod 7$ for $a_4 = 0, 1, \ldots, 6$ are

a_4	0	1	2	3	4	5	6
$2a_4 \bmod 7$	0	2	4	6	1	3	5
$3a_4 \bmod 7$	0	3	6	2	5	1	4

Each residue appears exactly once in each row.

15.5 Chaining with a universal family

Theorem 15.5 [Carter and Wegman, 1979]. *Implement a hash table with chaining, and choose its hash function h uniformly at random from a universal family $\mathcal{H}$. Then for every data set S, all operations run in $O(1)$ expected time, assuming*

1. *the expectation is over the random choice of h;*
2. *$|S| = O(n)$, that is, the load $\alpha = |S|/n$ is $O(1)$;*
3. *evaluating h takes $O(1)$ time.*

Proof. Analyze an unsuccessful Lookup; the other operations are only faster. Let S be an arbitrary data set with $|S| = O(n)$, and consider a Lookup for $x \notin S$. Its running time is $O(1)$ to compute $h(x)$ plus $O(L)$ to traverse the list, where L is the length of the list in $A[h(x)]$. Here L is a random variable depending on h.
Use the decomposition principle: express the random variable as a sum of indicator variables,

then apply linearity of expectation. For each $y \in S$, so $y \neq x$, define

$$Z_y = \begin{cases} 1 & \text{if } h(y) = h(x), \\ 0 & \text{otherwise.} \end{cases}$$

Then $L = \sum_{y \in S} Z_y$, and $\mathbb{E}[Z_y] = 0 \cdot \mathbb{P}[Z_y = 0] + 1 \cdot \mathbb{P}[Z_y = 1] = \mathbb{P}[h(y) = h(x)]$. Since $\mathcal{H}$ is universal, each such probability is at most $1/n$. Therefore

$$\mathbb{E}[L] = \sum_{y \in S} \mathbb{E}[Z_y] = \sum_{y \in S} \mathbb{P}[h(y) = h(x)] \leq \sum_{y \in S} \frac{1}{n} = \frac{|S|}{n} = \alpha = O(1).$$

The expected running time is $O(1) + O(\mathbb{E}[L]) = O(1)$. □

15.6 Open addressing: heuristic analysis

With open addressing, each slot holds one object and the hash function produces a probe sequence for each key. A rigorous analysis is difficult. For a quick idealized analysis, make the HEURISTIC ASSUMPTION that all $n!$ probe sequences are equally likely.

Proposition 15.6 [INSERTION UNDER THE HEURISTIC ASSUMPTION]. *Under the heuristic assumption, the expected insertion time is about $1/(1-\alpha)$, where α is the load.*

Proof. A random probe finds an empty slot with probability about $1 - \alpha$. So the insertion time is about the number N of flips of a coin needed to get heads, where $\mathbb{P}[\text{heads}] = 1 - \alpha$. The first flip always happens; with probability α it is tails, and the expected number of further flips is again $\mathbb{E}[N]$. Thus

$$\mathbb{E}[N] = 1 + \alpha\,\mathbb{E}[N], \qquad \text{so} \qquad \mathbb{E}[N] = \frac{1}{1-\alpha}.$$ □

The bound grows without limit as $\alpha \to 1$, which is why open addressing needs $\alpha \ll 1$.

For linear probing, the heuristic assumption is completely false. A less false assumption is that the initial probes are uniformly random and independent for different keys.

Theorem 15.7 [KNUTH, 1962]. *Under this assumption, the expected insertion time with linear probing is about*

$$\frac{1}{(1-\alpha)^2},$$

where α is the load.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Universal Hashing.

Lecture 16: Bloom Filters

A Bloom filter supports fast Inserts and Lookups for a set of keys. Compared with a hash table, it is more space efficient, but it

1. cannot store an object associated with a key;
2. does not support deletions;
3. has a small false positive probability: Lookup may report that x has been inserted even though it has not.

Bloom filters were originally used in early spellcheckers. A canonical application is a list of forbidden passwords. A modern application is in network routers, which have limited memory and need to be very fast.

16.1 The data structure

A Bloom filter consists of

1. an array A of n bits, all initially 0;
2. k hash functions $h_1, \ldots, h_k$, where k is a small constant.

For a data set S, the quantity $b = n/|S|$ is the number of bits per object.

Algorithm 16.1: Bloom filter operations

```
1 procedure Insert(x)
2     for i ← 1 to k do
3         A[h_i(x)] ← 1                    // whether or not the bit is already 1
4 procedure Lookup(x)
5     return true if A[h_i(x)] = 1 for every i ∈ {1, ..., k}, and false otherwise
```

There are no false negatives: if x was inserted, its k bits were set to 1 and are never reset, so Lookup(x) succeeds. A false positive occurs when x was not inserted but all k bits $A[h_i(x)]$ were already set to 1 by other insertions.

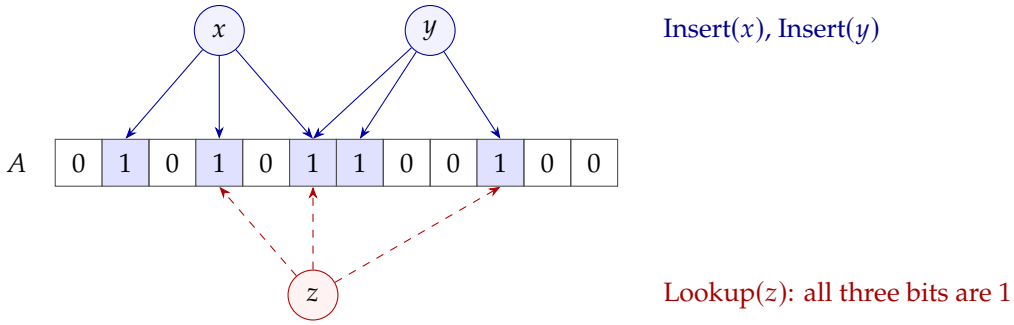


Figure 36: Schematic Bloom filter with $k = 3$. Inserting x and y sets the bits at their hash values. A Lookup of $z \notin S$ whose three bits were all set by these insertions is a false positive.

16.2 Heuristic analysis

There should be a trade-off between space and false positive probability. To quantify it, assume (without justification) that all values $h_i(x)$ are uniformly random and independent, across different i and different x. Insert a data set S into a filter of n bits.

Lemma 16.1 [Probability that a bit is set]. *Under the heuristic assumption, after inserting S, a given bit of A is 1 with probability*

$$1 - \left(1 - \frac{1}{n}\right)^{k|S|} \approx 1 - e^{-k|S|/n} = 1 - e^{-k/b}.$$

Proof. The insertions set $k|S|$ bits, each chosen independently and uniformly. The given bit is missed by each with probability $1 - 1/n$, so it remains 0 with probability $(1 - 1/n)^{k|S|}$. For the approximation, use $1 + z \le e^z$, which is nearly an equality for small $|z|$. With $z = -1/n$,

$$\left(1 - \frac{1}{n}\right)^{k|S|} \le e^{-k|S|/n},$$

and the two sides agree in the limit of large n. □

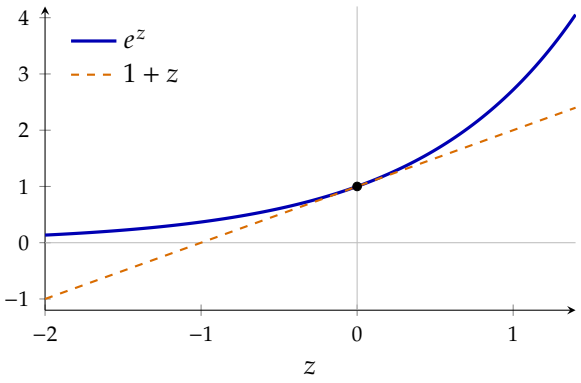


Figure 37: The line $1 + z$ is tangent to e^z at $z = 0$ and lies below it everywhere, so $1 + z \le e^z$ with near-equality for small $|z|$.

Proposition 16.2 [False positive rate]. *Under the heuristic assumption, for $x \notin S$, the false positive probability is approximately*

$$\varepsilon = \left(1 - e^{-k/b}\right)^k,$$

where b is the number of bits per object.

Proof. Lookup(x) is a false positive exactly when all k bits $A[h_1(x)], \ldots, A[h_k(x)]$ are 1. Under the assumption these are k independent uniformly random positions. Treating their values as independent, each is 1 with the probability in Lemma 16.1. □

Proposition 16.3 [Choosing the number of hash functions]. *For fixed b, the error rate ε is minimized by $k \approx (\ln 2)\, b \approx 0.693\, b$. Then*

$$\varepsilon \approx \left(\frac{1}{2}\right)^{(\ln 2) b}, \qquad \textit{equivalently} \qquad b \approx 1.44 \log_2 \frac{1}{\varepsilon}.$$

Proof. Write $p = e^{-k/b}$, so $k = -b \ln p$ and

$$\ln \varepsilon = k \ln(1 - p) = -b \, \ln p \, \ln(1 - p).$$

Minimizing ε means maximizing $\ln p \, \ln(1 - p)$ over $0 < p < 1$. This expression is symmetric under $p \leftrightarrow 1 - p$ and is maximized at $p = 1/2$, that is, at $k = b \ln 2$. Then each bit of x is 1 with probability $1/2$, so $\varepsilon = (1/2)^k = (1/2)^{(\ln 2) b}$. Solving for b gives $b = \log_2(1/\varepsilon)/\ln 2 \approx 1.44 \log_2(1/\varepsilon)$. □

The error rate is exponentially small in the number of bits per object.

Example: With $b = 8$ bits per object, the optimum $k \approx 5.5$, so choose $k = 5$ or $k = 6$. Either gives an error probability of only about 2%.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Bloom Filters.

Lecture 17: Greedy Algorithms

17.1 The greedy paradigm

Informally, a greedy algorithm iteratively makes "myopic" decisions and hopes that everything works out at the end. Dijkstra's shortest-path algorithm is an example: it processes each destination once, irrevocably.

Compared with divide and conquer:

1. It is easy to propose multiple greedy algorithms for many problems.
2. Running-time analysis is usually easy, in contrast with tools such as the master method.
3. Correctness is hard to establish, in contrast with the usually straightforward inductive correctness proofs for divide and conquer.

Most greedy algorithms are *not* correct, even when intuition says otherwise.

Dijkstra's negative-edge counterexample in Lecture 11 shows why a plausible greedy rule can fail.

17.2 Proving greedy algorithms correct

1. Induction, often in the form "greedy stays ahead." The correctness proof for Dijkstra's algorithm in Lecture 11 is an example.
2. Exchange arguments: transform another solution into the greedy one by exchanges that do not make it worse. The scheduling application of Lecture 18 is an example.

17.3 Application: optimal caching

A small fast memory, the cache, sits in front of a big slow memory. A sequence of page requests is processed in order. A request for a page that is not in the cache is a fault, or cache miss; the page must then be brought in, and when the cache is full, something must be evicted to make room. The question is what to evict.

Example: The cache holds a, b, c, d, and the request sequence is c, d, e, f, a, b. Suppose the faults on e and f evict a and b. Then the later requests for a and b also fault, for 4 page faults in total. Two faults, on e and f, were inevitable. The other two are consequences of poor eviction choices: evicting c and d, which are never requested again, would have left only 2 faults.

The FURTHEST-IN-FUTURE algorithm, on each fault with a full cache, evicts the cached page whose next request is furthest in the future (a page never requested again counts as furthest). In the example, it evicts c and d.

Theorem 17.1 [BÉLÁDY, 1960s**].** *The furthest-in-future algorithm is optimal: it minimizes the number of cache misses.*

The algorithm needs the future request sequence, so it is not directly implementable. It is still useful in two ways. First, it guides practical algorithms: for example, evicting the least recently used page (LRU) should do well when the data exhibit locality of reference, since then the recent past predicts the near future. Second, it is an idealized benchmark for caching algorithms.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Introduction to Greedy Algorithms.

Lecture 18: A Scheduling Application

One shared resource, such as a processor, must perform many jobs, such as processes. In what order should the jobs be sequenced? This problem gives a first complete example of designing a greedy algorithm, ruling out a plausible alternative, and proving correctness by an exchange argument.

18.1 Problem definition

Each job j has a WEIGHT w_j, its priority, and a LENGTH $\ell_j > 0$. A schedule is an ordering of the n jobs, processed one after another starting at time 0.

Definition 18.1 [COMPLETION TIME]. The COMPLETION TIME C_j of job j in a schedule is the sum of the lengths of the jobs up to and including j.

The goal is to minimize the WEIGHTED SUM OF COMPLETION TIMES

$$\sum_{j=1}^{n} w_j C_j.$$

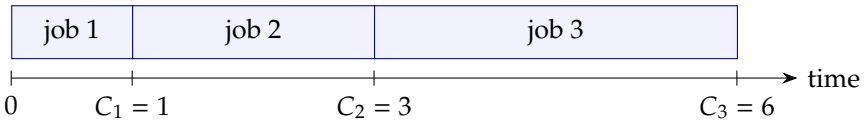


Figure 38: Three jobs with $\ell_1 = 1$, $\ell_2 = 2$, $\ell_3 = 3$, scheduled in the order $1, 2, 3$. The completion times are $C_1 = 1$, $C_2 = 3$, $C_3 = 6$.

Example: With $w_1 = 3$, $w_2 = 2$, $w_3 = 1$, the schedule in the figure has objective $3 \cdot 1 + 2 \cdot 3 + 1 \cdot 6 = 15$.

18.2 Designing the greedy algorithm

Two special cases suggest the right answer. With equal lengths, jobs with larger weight should go earlier. With equal weights, shorter jobs should go earlier. When the advice conflicts, for example $w_i > w_j$ but $\ell_i > \ell_j$, assign each job a score that increases in its weight and decreases in its length, and schedule jobs in order of decreasing score. Two natural guesses are:

1. order jobs by decreasing difference $w_j - \ell_j$;
2. order jobs by decreasing ratio w_j / ℓ_j.

To rule one out, find an instance on which the two orders differ; at least one of them is then incorrect.

Example: Take $\ell_1 = 5$, $w_1 = 3$ and $\ell_2 = 2$, $w_2 = 1$. Job 1 has the larger ratio ($3/5 > 1/2$) and job 2 the larger difference ($-1 > -2$). The difference order $2, 1$ gives $C_2 = 2$, $C_1 = 7$ and objective $1 \cdot 2 + 3 \cdot 7 = 23$. The ratio order $1, 2$ gives $C_1 = 5$, $C_2 = 7$ and objective $3 \cdot 5 + 1 \cdot 7 = 22$.

So the difference rule is not always correct. The ratio rule is.

Algorithm 18.1: Greedy scheduling by ratio

Input: Jobs $1, \dots, n$ with weights w_j and lengths $\ell_j > 0$.
Output: A schedule minimizing $\sum_j w_j C_j$.

1 **sort** the jobs by nonincreasing ratio w_j / ℓ_j, breaking ties arbitrarily
2 **return** the jobs in that order

The running time is $O(n \log n)$, since the algorithm only needs to sort.

18.3 Correctness with distinct ratios

The key computation is the effect of swapping two consecutive jobs.

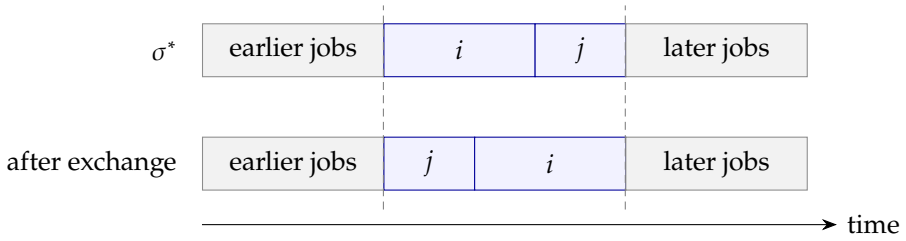


Figure 39: Exchanging consecutive jobs i and j, with i scheduled immediately before j. Jobs before and after the pair keep their completion times.

Lemma 18.2 [Cost of an exchange]. *Let i be scheduled immediately before j, and exchange them, leaving the other jobs unchanged. Then every other job's completion time is unaffected, C_i goes up by ℓ_j, and C_j goes down by ℓ_i. The objective changes by*

$$w_i \ell_j - w_j \ell_i.$$

Proof. A job before the pair is unaffected. A job after the pair still completes after the same set of jobs, including both i and j, so its completion time is unchanged. Job i now also waits for j, and job j no longer waits for i. The cost of the exchange is $w_i \ell_j$ and the benefit is $w_j \ell_i$. □

Theorem 18.3 [Greedy by ratio, distinct ratios]. *If all ratios w_j/ℓ_j are distinct, ordering the jobs by decreasing ratio minimizes $\sum_j w_j C_j$.*

Proof. The proof is by an exchange argument, by contradiction. Fix an input of n jobs and rename them so that

$$\frac{w_1}{\ell_1} > \frac{w_2}{\ell_2} > \cdots > \frac{w_n}{\ell_n}.$$

The greedy schedule σ is then $1, 2, \ldots, n$. Suppose an optimal schedule σ^* is strictly better than σ, so $\sigma^* \neq \sigma$. The only schedule in which the indices of consecutive jobs always go up is $1, 2, \ldots, n$, so σ^* has consecutive jobs i, j, with i immediately before j and $i > j$.
Exchange i and j in σ^*. Since $i > j$,

$$\frac{w_i}{\ell_i} < \frac{w_j}{\ell_j} \quad \Longrightarrow \quad w_i \ell_j < w_j \ell_i,$$

so by Lemma 18.2 the cost is less than the benefit. The exchange strictly improves σ^*, contradicting its optimality. □

18.4 Handling ties

With ties, a strictly improving exchange may not exist, so the proof shows directly that the greedy schedule is at least as good as *every* other schedule.

Theorem 18.4 [Greedy by ratio, with ties]. *Ordering the jobs by nonincreasing ratio w_j/ℓ_j, with ties broken arbitrarily, minimizes $\sum_j w_j C_j$.*

Proof. Fix an input and rename the jobs so that the greedy schedule σ is $1, 2, \ldots, n$; then $w_1/\ell_1 \geq w_2/\ell_2 \geq \cdots \geq w_n/\ell_n$. Let σ^* be an arbitrary schedule. Call a pair of jobs $i > j$ with i scheduled earlier an inverted pair.
If $\sigma^* = \sigma$, there is nothing to prove. Otherwise, as before, σ^* has consecutive jobs i, j with i immediately before j and $i > j$, an adjacent inversion. Since $i > j$,

$$\frac{w_i}{\ell_i} \leq \frac{w_j}{\ell_j} \quad \Longrightarrow \quad w_i \ell_j \leq w_j \ell_i,$$

so by Lemma 18.2 exchanging i and j has net benefit $w_j \ell_i - w_i \ell_j \geq 0$. The exchange changes the relative order of only the pair i, j, so it reduces the number of inverted pairs by exactly one. Repeat. There are at most $\binom{n}{2}$ inverted pairs, so after at most $\binom{n}{2}$ such exchanges no inversion remains, and the schedule is σ. No exchange increased the objective, so σ is at least as good as σ^*. Since σ^* was arbitrary, the greedy schedule is optimal. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), A Scheduling Application.

Lecture 19: Prim's Minimum Spanning Tree Algorithm

The informal goal is to connect a set of points as cheaply as possible. Applications include networking and clustering (Lecture 21). Two greedy algorithms solve the problem in $O(m \log n)$ time with suitable data structures, where m is the number of edges and n the number of vertices: Prim's algorithm (1957; also Dijkstra 1959 and Jarník 1930) and Kruskal's algorithm (1956, Lecture 20).

19.1 The minimum spanning tree problem

Definition 19.1 [Minimum spanning tree]. The input is an undirected graph $G = (V, E)$, in adjacency-list representation, with a cost c_e for each edge $e \in E$. Negative costs are allowed. A spanning tree is a set $T \subseteq E$ such that

1. T has no cycles, and
2. the subgraph (V, T) is connected: it contains a path between each pair of vertices.

The output is a minimum spanning tree (MST): a spanning tree minimizing the total cost $\sum_{e \in T} c_e$.

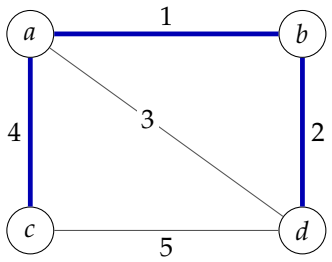


Figure 40: The running example. The highlighted edges ab, bd, ac form the minimum spanning tree, of cost 7. The set $\{ab, ad\}$ has no cycle but misses c, so it is not spanning; a set containing a cycle is disallowed.

Two standing assumptions hold throughout.

1. *G is connected.* Otherwise there is no spanning tree. This is easy to check in preprocessing, for example by depth-first search.
2. *Edge costs are distinct.* Prim's and Kruskal's algorithms remain correct with ties broken arbitrarily, but the correctness proof is more annoying, so it is omitted.

Since G is connected, $m \geq n - 1$.

19.2 Prim's algorithm

Prim's algorithm grows a single tree from an arbitrary start vertex. Each iteration increases the number of spanned vertices in the cheapest way possible, much as Dijkstra's shortest-path algorithm grows its set of processed vertices.

Algorithm 19.1: Prim's algorithm

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs c_e.
Output: A spanning tree T of G.

$X \leftarrow \{s\}$ for an arbitrary $s \in V$
$T \leftarrow \emptyset$ *// invariant: X is the vertex set spanned by T*
while $X \neq V$ **do**
  Let $e = (u, v)$ be the cheapest edge of G with $u \in X$, $v \notin X$
  $T \leftarrow T \cup \{e\}$; $X \leftarrow X \cup \{v\}$
return T

Example: In Figure 40, start at $s = a$. The edges leaving $\{a\}$ cost $1, 3, 4$, so ab is added. The edges leaving $\{a, b\}$ cost $2, 3, 4$, so bd is added. The edges leaving $\{a, b, d\}$ are ac and cd, so ac is added. The result is the minimum spanning tree $\{ab, bd, ac\}$.

Theorem 19.2 [Correctness of Prim's algorithm]. *Prim's algorithm always outputs a minimum spanning tree.*

The proof has two parts. Part I shows that the output T^* is a spanning tree, using basic facts about cuts. Part II shows that T^* is a minimum spanning tree, using the cut property. Both tools are reused for Kruskal's algorithm.

19.3 Cuts and three graph facts

Definition 19.3 [Cut]. A cut of a graph $G = (V, E)$ is a partition of V into two nonempty sets A and B. An edge crosses the cut (A, B) if it has one endpoint in each set.

A graph with n vertices has roughly 2^n cuts: each vertex chooses whether it lies in A or in B. (Exactly $2^n - 2$ ordered choices leave both sides nonempty.)

Lemma 19.4 [Empty cut lemma]. *A graph is not connected if and only if there is a cut (A, B) with no crossing edges.*

Proof. ($\Leftarrow$) Pick $u \in A$ and $v \in B$. A u–v path starts in A and ends in B, so one of its edges crosses (A, B). No edge crosses, so there is no u–v path and G is not connected.
($\Rightarrow$) Suppose G has no u–v path. Let A be the set of vertices reachable from u (its connected component) and let $B = V \setminus A$ (all other components). Both are nonempty, since $u \in A$ and

$v \in B$. No edge crosses (A, B); otherwise its endpoint in B would also be reachable from u, and A would be bigger. □

Lemma 19.5 [Double-crossing lemma]. *If a cycle $C \subseteq E$ has an edge crossing the cut (A, B), then some other edge of C also crosses (A, B).*

Proof. Traverse C starting in A at an endpoint of the crossing edge, beginning along that edge. The traversal moves to B and must return to its starting vertex in A, which requires another crossing edge of C. □

Corollary 19.6 [Lonely cut corollary]. *If e is the only edge crossing some cut (A, B), then e lies in no cycle.*

Proof. If e lay in a cycle, Lemma 19.5 would give another edge of that cycle crossing (A, B). □

19.4 Part I: Prim's algorithm outputs a spanning tree

Proposition 19.7. *Prim's algorithm outputs a spanning tree T^*.*

Proof. We check three properties.

1. *T spans X.* This holds initially, and each iteration adds one edge joining the old tree to the one new vertex, so it holds by induction.
2. *The algorithm cannot get stuck with $X \neq V$.* Otherwise no edge crosses the cut $(X, V \setminus X)$, and Lemma 19.4 says G is disconnected, contrary to the standing assumption.
3. *No cycle is ever created in T.* Consider an iteration with current sets X and T, in which e is added. Every edge of T has both endpoints in X, so e is the first edge crossing $(X, V \setminus X)$ to be added to T. It is lonely in this cut within $T \cup \{e\}$, so by Corollary 19.6 its addition cannot create a cycle.

At termination $X = V$, so T^* connects all vertices and has no cycles. □

19.5 Part II: the cut property

The key question is when it is *safe* to include an edge in the tree so far.

Theorem 19.8 [Cut property]. *Assume distinct edge costs. Let e be an edge of G, and suppose there is a cut (A, B) such that e is the cheapest edge of G crossing it. Then e belongs to every minimum spanning tree of G.*

For example, take a 4-cycle whose edges cost $1, 2, 3, 4$ in order around the cycle. The cut isolating the vertex shared by edges 1 and 2 justifies edge 1. The cut isolating the vertex shared by edges 2 and 3 justifies edge 2. The cut isolating the vertex shared by edges 3 and 4 justifies edge 3. These three edges form the minimum spanning tree.

Proof that the cut property implies Theorem 19.2. By Proposition 19.7, Prim's algorithm outputs a spanning tree T^*. Every edge it adds is the cheapest edge crossing the current cut $(X, V \setminus X)$, so each edge of T^* is justified by the cut property. Hence T^* is contained in every minimum spanning tree T. Adding an edge $e \notin T^*$ to the spanning tree T^* creates a cycle with the path in T^* joining its endpoints, so a spanning tree cannot strictly contain T^*. Thus $T = T^*$: the minimum spanning tree is unique, and Prim's algorithm finds it. □

19.6 Proof of the cut property

The proof is by contradiction, through an exchange argument similar to the scheduling application of Lecture 18. Suppose e is the cheapest edge crossing (A, B), but e is not in a minimum spanning tree T^*. The idea is to exchange e with an edge of T^* and obtain a cheaper spanning tree.

Since T^* is connected, some edge $f \neq e$ of T^* crosses (A, B), and $c_e < c_f$. Exchanging an arbitrary such f can fail, however: for a spanning tree T^* with $e \notin T^*$ and $f \in T^*$, the set $T^* \cup \{e\} \setminus \{f\}$ may or may not be a spanning tree, depending on the choice of e and f, even when e is the cheapest edge crossing a cut.

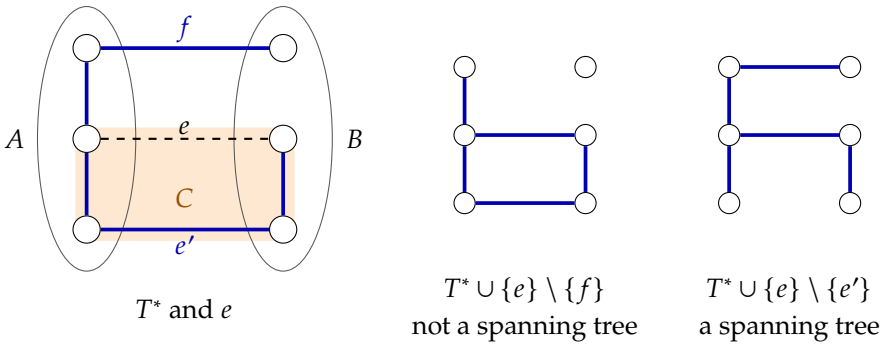


Figure 41: Left: a spanning tree T^* (solid), a non-tree edge e crossing (A, B), and the cycle C (shaded) that e creates. Both f and e' cross the cut. Middle: exchanging e with f isolates a vertex and leaves a cycle. Right: exchanging e with $e' \in C$ gives a spanning tree.

Proof of Theorem 19.8. Since T^* is a spanning tree, it contains a path between the endpoints of e; adding e closes this path into a cycle $C \subseteq T^* \cup \{e\}$. Because e crosses (A, B), Lemma 19.5 gives another edge $e' \neq e$ of C crossing (A, B), and $e' \in T^*$. Let

$$T = T^* \cup \{e\} \setminus \{e'\}.$$

T is a spanning tree. It is connected, because a path in $T^* \cup \{e\}$ that uses e' can be rerouted around the rest of C. It is acyclic, because every cycle of $T^* \cup \{e\}$ uses e and the path in T^* between its endpoints, which is unique; so C is the only cycle, and removing e' breaks it. Since e is the cheapest edge crossing the cut, $c_e < c_{e'}$, and therefore

$$\sum_{g \in T} c_g = \sum_{g \in T^*} c_g + c_e - c_{e'} < \sum_{g \in T^*} c_g.$$

This contradicts the minimality of T^*. □

19.7 A fast implementation with heaps

A straightforward implementation performs $n - 1$ iterations, each scanning all m edges for the cheapest crossing edge in $O(m)$ time, for $O(mn)$ total. As with Dijkstra's algorithm, a heap does better. Maintain:

1. the heap contains exactly the vertices of $V \setminus X$;
2. for $v \in V \setminus X$, key$[v]$ is the cost of the cheapest edge (u, v) with $u \in X$, or $+\infty$ if there is no such edge.

Computing the initial keys takes $O(m)$ time, and the $n - 1$ inserts take $O(n \log n)$, so initialization costs $O(m + n \log n) = O(m \log n)$. Given the invariants, ExtractMin returns the next vertex $v \notin X$, and the edge achieving its key is the edge (u, v) crossing $(X, V \setminus X)$ to add to T.

When v moves into X, keys may become stale. Only vertices $w \in V \setminus X$ adjacent to v can change, and for them the new edge (v, w) is a new candidate.

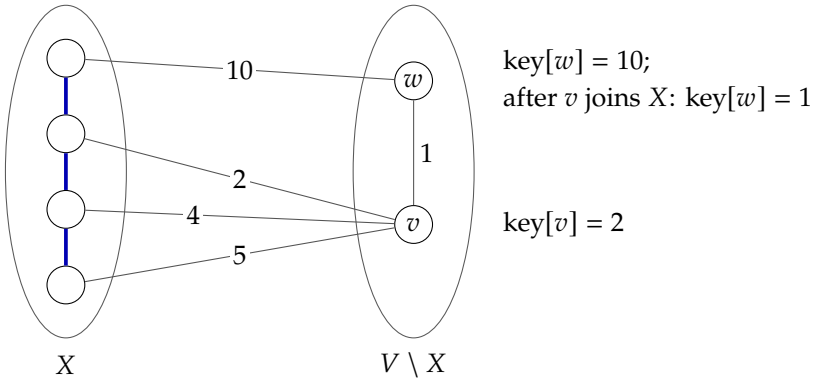


Figure 42: Maintaining the keys. Before the iteration, key$[v] = 2$ and key$[w] = 10$. Prim's algorithm extracts v and adds the edge of cost 2. The edge (v, w) now crosses the new cut, so key$[w]$ drops to 1.

Algorithm 19.2: Prim's algorithm with a heap

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs c_e.
Output: A minimum spanning tree T of G.

```
X ← {s}; T ← ∅
for each v ∈ V \ {s} do
    key[v] ← min({c_sv : (s, v) ∈ E} ∪ {+∞}); insert v into the heap
while X ≠ V do
    v ← ExtractMin; add v to X and its key-achieving edge to T
    for each edge (v, w) ∈ E with w ∈ V \ X do
        Delete w from the heap
        key[w] ← min{key[w], c_vw}
        Reinsert w into the heap
return T
```

Deleting w from the middle of the heap requires knowing where w sits, and the implementation must also remember which edge achieves each key; this bookkeeping is the subtle part of the implementation.

Theorem 19.9 [Running time with heaps]. *Prim's algorithm with a heap runs in $O(m \log n)$ time.*

Proof. The running time is dominated by the heap operations. There are $n - 1$ inserts during preprocessing and $n - 1$ extractions, one per iteration. Each edge (v, w) triggers at most one delete and reinsert, when its first endpoint is added to X. This is $O(m)$ heap operations, using $m \geq n - 1$, each taking $O(\log n)$ time, for $O(m \log n)$ total, as fast as sorting. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Prim's Minimum Spanning Tree Algorithm.

Lecture 20: Kruskal's Minimum Spanning Tree Algorithm

The problem is the same as in Lecture 19: given a connected undirected graph $G = (V, E)$ with distinct edge costs c_e, find a minimum-cost spanning tree (no cycles, connected). The main tool is again the cut property: if e is the cheapest edge crossing some cut (A, B), then e belongs to the minimum spanning tree. Throughout, $n = |V|$ and $m = |E|$.

20.1 Kruskal's algorithm

Prim's algorithm grows one tree. Kruskal's algorithm instead scans all edges from cheapest to most expensive and keeps an edge whenever it does not close a cycle, so the chosen edges may form several trees until the end.

Algorithm 20.1: Kruskal's algorithm

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs.
Output: A spanning tree T of G.

```
sort edges by increasing cost; rename them 1, 2, ..., m so that c_1 < c_2 < ... < c_m
T ← ∅
for i ← 1 to m do
    if T ∪ {i} has no cycles then
        T ← T ∪ {i}
return T
```

Example: In Figure 43, edges 1, 2, and 3 are added. Edge 4 joins two vertices already connected by edges 2 and 3, so it is skipped. Edge 5 reaches the last vertex. Edges 6 and 7 would create cycles. The minimum spanning tree $\{1, 2, 3, 5\}$ has cost 11.

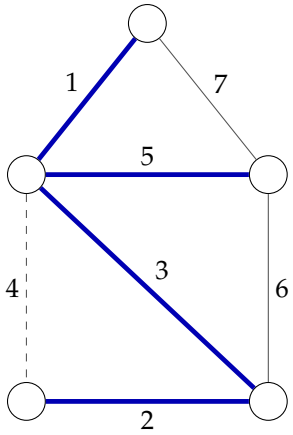


Figure 43: Kruskal's algorithm on a five-vertex graph. Edges 1, 2, 3, 5 are added (highlighted). Edge 4 (dashed) is rejected because it would close the cycle through edges 2 and 3; edges 6 and 7 are rejected for the same reason.

20.2 Correctness

The proof uses three facts from Lecture 19: the *empty cut lemma* (a graph is disconnected if and only if some cut has no crossing edges), the *lonely cut corollary* (an edge that is the only one crossing some cut lies in no cycle), and the cut property.

Theorem 20.1 [Correctness of Kruskal's algorithm]. *Kruskal's algorithm outputs the minimum spanning tree of G.*

Proof. Let T^* be the output.

1. *T^* has no cycles*, since the algorithm never adds an edge that creates one.
2. *T^* is connected.* By the empty cut lemma, it suffices to show that T^* has an edge crossing every cut (A, B). Since G is connected, at least one of its edges crosses (A, B). The key point is that Kruskal's algorithm includes the *first* edge crossing (A, B) that it considers: when that edge is examined, it is the only edge of $T \cup \{i\}$ crossing (A, B), so by the lonely cut corollary it cannot create a cycle.
3. *Every edge of T^* is justified by the cut property.* Consider the iteration in which the edge (u, v) is added to the current set T. Since $T \cup \{(u, v)\}$ has no cycle, T has no u–v path. As in the proof of the empty cut lemma, let A be the set of vertices reachable from u in (V, T) and $B = V \setminus A$; then $v \in B$ and no edge of T crosses (A, B). By the key point of (2), no edge crossing (A, B) was considered earlier: the first one considered would have been added to T, and an edge of T would cross (A, B). Hence (u, v) is the first, and therefore the cheapest, edge of G crossing (A, B), and the cut property justifies it.

By (1) and (2), T^* is a spanning tree. By (3) and the cut property, every edge of T^* lies in the minimum spanning tree; a spanning tree contained in a spanning tree equals it, so T^* is the minimum spanning tree. □

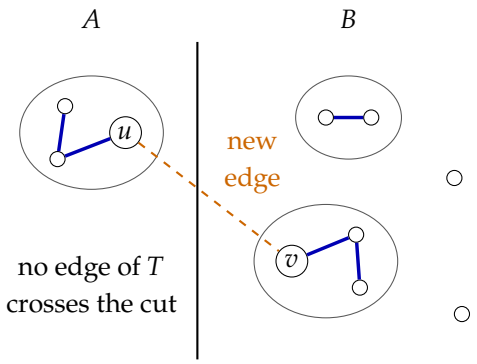


Figure 44: Step (3) of the proof. The blobs are the connected components of the current (V, T). Taking A to be the component of u gives a cut that T does not cross, and the new edge (u, v) is the first edge of G crossing it.

20.3 Running time of the straightforward implementation

- Sorting takes $O(m \log m) = O(m \log n)$ time, because $m = O(n^2)$ when there are no parallel edges, so $\log m \le 2 \log n + O(1)$.
- There are m iterations. Checking whether $T \cup \{i\}$ has a cycle amounts to checking whether the endpoints of i are already connected in (V, T), which breadth-first or depth-first search does in $O(n)$ time, since (V, T) has at most $n - 1$ edges.

The total is $O(m \log n) + O(mn) = O(mn)$. A data structure supporting $O(1)$-time cycle checks reduces this to $O(m \log n)$.

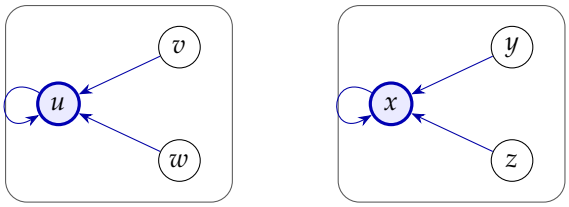
20.4 The union-find data structure

A UNION-FIND data structure maintains a partition of a set of objects into groups, supporting two operations:

- Find(x): return the name of the group containing x;
- Union(C_i, C_j): fuse the groups C_i and C_j into a single group.

For Kruskal's algorithm, the objects are the vertices and the groups are the connected components of (V, T) for the chosen edges T. Adding a new edge (u, v) to T corresponds to fusing the components of u and v, and $T \cup \{(u, v)\}$ has a cycle exactly when u and v are already in the same component.

Idea 1. Keep one linked structure per connected component of (V, T), and give each component an arbitrary LEADER vertex. Maintain the invariant that each vertex points to the leader of its component; the component's name is inherited from its leader.



leaders u and x (shaded) point to themselves

Figure 45: Two components, each with a leader. Every vertex points directly to the leader of its component, so Find is one pointer lookup.

Given an edge (u, v), the vertices u and v lie in the same component if and only if their leader pointers match, that is, Find(u) = Find(v). This is an $O(1)$-time cycle check.

When (u, v) is added to T, the components of u and v merge, and leader pointers must be updated to restore the invariant. In the worst case this takes $\Theta(n)$ updates, for example when two components

with $n/2$ vertices each merge.

Idea 2. When two components merge, the smaller one inherits the leader of the larger one. A size field in each component makes this easy. A single merge can still take $\Theta(n)$ updates, for the same reason as before. The gain appears when counting per vertex.

Lemma 20.2 [Leader updates per vertex]. *With Idea 2, each vertex has its leader pointer updated at most $\log_2 n$ times over the course of Kruskal's algorithm.*

Proof. A vertex v is updated only when its component is the smaller one in a merge, so the size of v's component at least doubles each time. Its size starts at 1 and never exceeds n, so this happens at most $\log_2 n$ times. □

Algorithm 20.2: Kruskal's algorithm with union-find

Input: Connected undirected graph $G = (V, E)$ with distinct edge costs.
Output: The minimum spanning tree T of G.

```
sort edges by increasing cost
T ← ∅; make each vertex its own component and its own leader
for each edge (u, v) in sorted order do
    if Find(u) ≠ Find(v) then
        T ← T ∪ {(u, v)}
        Union the two components, the smaller adopting the larger one's leader
return T
```

Theorem 20.3 [Running time with union-find]. *Kruskal's algorithm with union-find runs in $O(m \log n)$ time.*

Proof. Sorting takes $O(m \log n)$ time. The m cycle checks take $O(1)$ time each, or $O(m)$ in total. By Lemma 20.2, all leader pointer updates together take $O(n \log n)$ time. Since $m \geq n - 1$, the total is $O(m \log n)$, matching Prim's algorithm. □

20.5 State of the art and open questions

Faster minimum spanning tree algorithms than $O(m \log n)$ are known:

- an $O(m)$-time randomized algorithm (Karger, Klein, Tarjan, 1995);
- an $O(m\,\alpha(n))$-time deterministic algorithm (Chazelle, 2000).

Here α is the inverse Ackermann function. It grows much more slowly than $\log^* n$, the number of times $\log_2$ can be applied to n before the result drops below 1; $\log^*$ is in turn the inverse of the tower function, whose value at k is an exponent tower of k twos.

Pettie and Ramachandran (2002) gave a deterministic minimum spanning tree algorithm that is provably optimal, yet its precise asymptotic running time is unknown: it lies between $\Theta(m)$ and $\Theta(m\,\alpha(n))$. Open questions include a *simple* randomized $O(m)$-time algorithm (it suffices to find one for the *MST verification* problem) and whether there is a deterministic $O(m)$-time algorithm. Eisner (1997) is suggested for further reading.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Kruskal's Minimum Spanning Tree Algorithm.

Lecture 21: Clustering

Clustering, also called unsupervised learning, aims to classify n "points" (web pages, images, genome fragments, and so on) into coherent groups. Kruskal's algorithm, stopped early, solves a natural formalization.

21.1 Max-spacing k-clusterings

The input is a dissimilarity measure: a distance $d(p, q)$ for each pair of points, assumed symmetric, $d(p, q) = d(q, p)$. Examples include Euclidean distance and genome similarity. The goal is that points lie in the same cluster if and only if they are nearby.

Assume the number k of desired clusters is known; in practice one can experiment with a range of values. A k-clustering partitions the points into k nonempty clusters. Points p and q are separated if they are assigned to different clusters.

Definition 21.1 [Spacing]. The spacing of a k-clustering is

$$\min_{p,q \text{ separated}} d(p, q).$$

Bigger spacing is better. The max-spacing k-clustering problem asks, given d and k, for a k-clustering of maximum spacing.

21.2 Single-link clustering

Algorithm 21.1: Greedy (single-link) clustering

Input: Points with symmetric distances $d(p, q)$; an integer $2 \le k \le n$.
Output: A k-clustering.

```
Put each point in a separate cluster
while there are more than k clusters do
    Let p, q be the closest pair of separated points        // determines the current spacing
    Merge the clusters containing p and q
return the clusters
```

This is Kruskal's algorithm stopped early, under the correspondence

$$\text{points} \leftrightarrow \text{vertices}, \qquad \text{distances} \leftrightarrow \text{edge costs}, \qquad \text{point pairs} \leftrightarrow \text{edges}.$$

A pair inside one cluster is not separated and is skipped, just as Kruskal's algorithm skips an edge that would create a cycle. The method is called single-link clustering. We say x and y were

DIRECTLY MERGED if the pair x, y was chosen in some iteration.

21.3 Correctness

Theorem 21.2 [SINGLE-LINK CLUSTERING IS OPTIMAL]. *Single-link clustering computes a max-spacing k-clustering.*

Let $C_1, \ldots, C_k$ be the greedy clustering and S its spacing, and let $\hat{C}_1, \ldots, \hat{C}_k$ be an arbitrary k-clustering. We must show that the spacing of $\hat{C}_1, \ldots, \hat{C}_k$ is at most S.

Lemma 21.3 [MERGE DISTANCES]. *If x and y were directly merged at some point, then $d(x, y) \le S$.*

Proof. Merging only removes pairs from the set of separated pairs, so the distance of the closest separated pair never decreases. The pair x, y was the closest separated pair when it was merged, and S is the distance of the closest separated pair at the end, so $d(x, y) \le S$. □

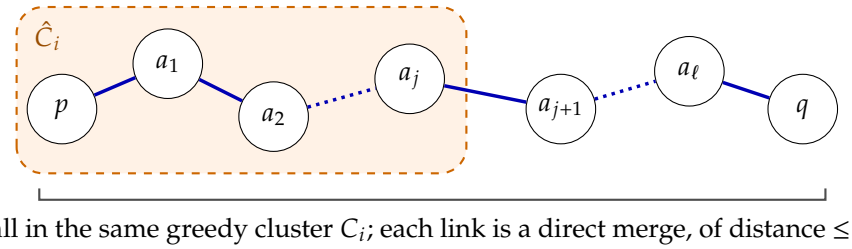


Figure 46: The tricky case. All points shown lie in one greedy cluster C_i, linked by direct merges of distance at most S. The path leaves $\hat{C}_i$ at some step $a_j \to a_{j+1}$, and that separated pair has distance at most S.

Proof of Theorem 21.2. Case 1: the $\hat{C}_i$ are the C_i, possibly after renaming. Then the spacing is the same, S.
Case 2: otherwise. Then there are points p, q in the same greedy cluster C_i but in different clusters $\hat{C}_i$ and $\hat{C}_j$. Indeed, if no such pair existed, each greedy cluster would lie inside a single $\hat{C}$-cluster; each $\hat{C}$-cluster would then be a union of greedy clusters and, being nonempty, would contain at least one, so with k clusters on each side the two clusterings would coincide, which is Case 1.
Easy case: p and q were directly merged. Since p, q are separated in $\hat{C}_1, \ldots, \hat{C}_k$, Lemma 21.3 gives

$$S \ge d(p, q) \ge \text{spacing of } \hat{C}_1, \ldots, \hat{C}_k.$$

Tricky case: p and q were merged indirectly, through several direct merges. The direct merges inside C_i connect all of its points, so there is a path $p = a_0, a_1, \ldots, a_\ell, a_{\ell+1} = q$ of points of C_i in which each consecutive pair was directly merged. Since $a_0 = p \in \hat{C}_i$ and $a_{\ell+1} = q \notin \hat{C}_i$, some

consecutive pair satisfies $a_j \in \hat{C}_i$ and $a_{j+1} \notin \hat{C}_i$. Then

$$S \;\geq\; d(a_j, a_{j+1}) \;\geq\; \text{spacing of } \hat{C}_1, \ldots, \hat{C}_k,$$

where the first inequality holds by Lemma 21.3 because a_j, a_{j+1} were directly merged, and the second because a_j, a_{j+1} are separated in $\hat{C}_1, \ldots, \hat{C}_k$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Clustering.

Lecture 22: Advanced Union-Find

A union-find data structure maintains a partition of a set X of n objects into groups and supports two operations:

- Find(x): return the name of the group containing x;
- Union(x, y): merge the groups containing x and y.

In the implementation used for Kruskal's algorithm, each object points directly to the leader of its group. Then Find takes $O(1)$ time. When two groups merge, the smaller group inherits the leader of the larger one, so each object's pointer changes only when its group at least doubles in size, and n unions take $O(n \log n)$ total work. This lecture trades slower Finds for much cheaper Unions and then shows that two simple optimizations make both operations almost constant time on average.

22.1 Lazy unions

Instead of updating every pointer in the smaller group, a lazy union updates only one: the leader (root) of one group becomes a child of the leader of the other.

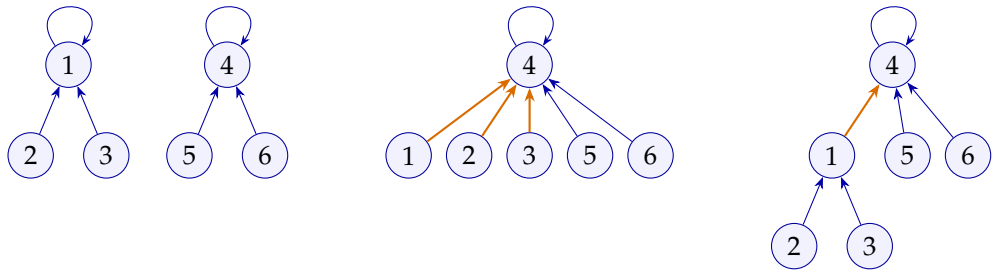


Figure 47: Merging the groups led by 1 and 4. Arrows are parent pointers; a loop marks a root. With leader pointers, objects 1, 2, 3 are all redirected to 4. A lazy union redirects only object 1; objects 2 and 3 now reach the leader in two hops.

In array form, with $A[i]$ the parent of object i:

Object i	1	2	3	4	5	6
Before the union	1	1	1	4	4	4
Leader pointers	4	4	4	4	4	4
Lazy union	4	1	1	4	4	4

Each object x now has a field parent$[x]$. The parent pointers induce a collection of directed trees on X, one per group, and x is a root if and only if parent$[x] = x$. The root serves as the group's name. Initially parent$[x] = x$ for every x, so each object is its own group.

Algorithm 22.1: Find(x) **with lazy unions**

Input: Object $x \in X$.
Output: The root of the tree containing x.

```
while parent[x] ≠ x do
    x ← parent[x]
return x
```

Union(x, y) computes $s_1 =$ Find(x) and $s_2 =$ Find(y) and resets the parent of one of s_1, s_2 to be the other. So a union costs two Finds plus $O(1)$ work. The price is that recovering a leader requires following a path of parent pointers, not just one, so Find may no longer take constant time.

If the new root is chosen arbitrarily, both operations take $\Theta(n)$ time in the worst case. Repeatedly making the root of a path a child of a new singleton object lengthens the path by one each time, so $n - 1$ unions can build a single path through all n objects, and a Find from its bottom visits all n of them.

22.2 Union by rank

To avoid such long, thin trees, each object x keeps a field rank$[x]$, initially 0. For now, ranks maintain the following invariant.

Definition 22.1 [Rank]. rank$[x]$ is the maximum number of hops from a leaf of x's subtree to x. Equivalently, a leaf has rank 0 and every other object has rank $1 + \max\{\text{rank}[c] : c \text{ a child of } x\}$.

Union by rank makes the root with the smaller rank a child of the root with the larger rank. With a tie, the new root is chosen arbitrarily and its rank increases by one; otherwise no rank changes. This is exactly what restores the invariant: hanging a tree of height r_1 below a root of height $r_2 > r_1$ creates paths of length at most $r_1 + 1 \le r_2$, while with $r_1 = r_2 = r$ it creates a path of length $r + 1$.

The following properties are immediate from the invariant and the update rule.

(1) For every object x, rank$[x]$ only goes up over time.

(2) Only ranks of roots go up; once x is a non-root, its rank is frozen forever.

(3) Ranks strictly increase along every path toward the root.

The maximum rank is roughly the worst-case running time of Find, since a root of rank r has a leaf r hops below it.

Algorithm 22.2: Union(x, y) **with union by rank**

```
Input: Objects x, y ∈ X.
s1 ← Find(x); s2 ← Find(y)
if s1 = s2 then return
if rank[s1] > rank[s2] then
    parent[s2] ← s1
else
    parent[s1] ← s2
    if rank[s1] = rank[s2] then
        rank[s2] ← rank[s2] + 1
```

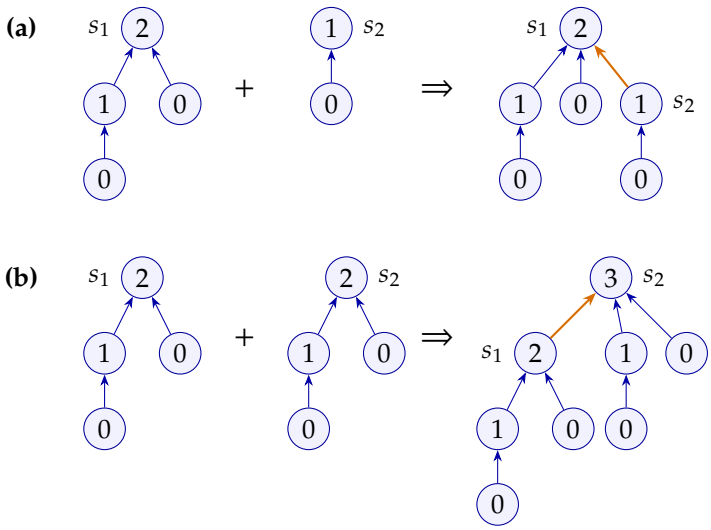


Figure 48: Union by rank; numbers inside nodes are ranks. (a) With unequal ranks, the smaller-rank root s_2 is hung below s_1 and no rank changes. (b) With equal ranks $r = 2$, the new root s_2 gets rank $r + 1 = 3$.

Theorem 22.2 [Rank Lemma]. *Consider an arbitrary sequence of* Union *and* Find *operations. At every time and for every $r \in \{0, 1, 2, \ldots\}$, at most $n/2^r$ objects have rank r.*

The subtree of an object x consists of the objects from which x can be reached by following parent pointers, including x.

Proof. The lemma follows from two claims: objects of rank r have disjoint subtrees, each with at least 2^r of the n objects.
Claim 1: if $x \neq y$ have the same rank r, their subtrees are disjoint. Suppose both subtrees contain z. Following parent pointers from z gives a unique path to its root, and both x and y lie on it. So one of x, y is an ancestor of the other, and by property (3) the ancestor has strictly larger rank, a contradiction.
Claim 2: the subtree of a rank-r object has size at least 2^r. Induct on the number of operations. Initially all ranks are 0 and all subtree sizes are 1. Subtree sizes never decrease, so there is nothing to prove unless some rank changes. That happens only in Union(x, y) with

rank$[s_1]$ = rank$[s_2]$ = r, when s_2 gets rank $r+1$. Its new subtree is the union of the old subtrees of s_1 and s_2, each of size at least 2^r by the inductive hypothesis, so it has size at least 2^{r+1}. □

Corollary 22.3. *The maximum rank is always at most* $\log_2 n$*. Hence, with union by rank,* Find *and* Union *take* $O(\log n)$ *time in the worst case.*

Proof. If some object has rank r, then $1 \le n/2^r$, so $r \le \log_2 n$. Ranks strictly increase along a path to the root, so a Find visits at most $1 + \log_2 n$ objects, and a Union is two Finds plus $O(1)$ work. □

22.3 Path compression

A Find may traverse the same leaf-to-root path many times. Path compression avoids this: after Find(x), reset the parent pointer of every object on the path from x to its root directly to the root.

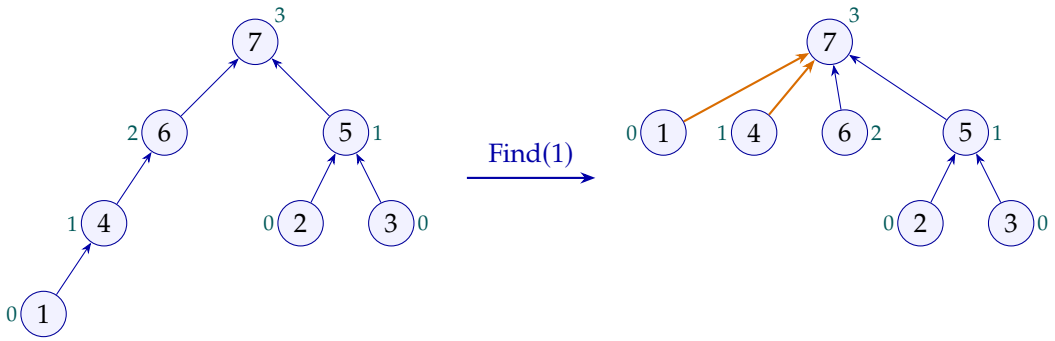


Figure 49: Find(1) with path compression. Numbers inside nodes are objects; small numbers beside them are ranks, which do not change. Objects 1 and 4 on the path now point directly to the root 7 (highlighted); 6 already did.

Algorithm 22.3: Find(x) **with path compression**

Input: Object $x \in X$.
Output: The root of the tree containing x.

```
r ← x
while parent[r] ≠ r do
    r ← parent[r]
while x ≠ r do
    q ← parent[x]; parent[x] ← r; x ← q
return r
```

The parent array before and after Find(1) in the figure:

Object i	1	2	3	4	5	6	7
parent[i] before	4	5	5	6	7	7	7
parent[i] after	7	5	5	7	7	7	7

The extra pass costs only a constant factor per Find and speeds up later ones. Ranks are maintained *exactly* as without path compression: they start at 0, the new root in a union is the old root of larger rank, and merging two roots of common rank r resets the new root's rank to $r + 1$. Compression never changes a rank.

Consequently, rank[x] is now only an upper bound on the maximum number of hops from a leaf to x, and it may be much larger. In the figure, object 6 keeps rank 2 but becomes a leaf. Two facts survive.

- The Rank Lemma still holds. Compression changes neither ranks nor which objects are roots, and each Find returns the same root as before. So every union links the same two roots and makes the same rank change as in the run without compression, ranks agree with that run at all times, and that run satisfies the lemma.
- rank[parent[x]] $>$ rank[x] for every non-root x. Compression only replaces a parent by a proper ancestor, whose rank is larger still.

22.4 The Hopcroft-Ullman bound

Definition 22.4 [Iterated logarithm]. $\log^* n$ is the number of times $\log_2$ must be applied to n before the result is at most 1.

For example, $\log^*(2^{65536}) = 5$, via $2^{65536} \mapsto 65536 \mapsto 16 \mapsto 4 \mapsto 2 \mapsto 1$. In general, if $T_1 = 2$ and $T_{t+1} = 2^{T_t}$, so that T_t is a tower of t twos, then $\log^* T_t = t$.

Theorem 22.5 [Hopcroft-Ullman 1973]. *With union by rank and path compression, m* Union *and* Find *operations take $O((m + n) \log^* n)$ time, which is $O(m \log^* n)$ in the interesting case $m = \Omega(n)$.*

Since a Union is two Finds plus $O(1)$ work, it suffices to count the objects visited by Finds. To measure progress, consider a non-root object x, whose rank is frozen, and the gap rank[parent[x]] $-$ rank[x]. Path compression increases this gap: if x's parent changes from p to $p' \neq p$, then p' is a proper ancestor of p, so rank[p'] $>$ rank[p].

Group ranks into rank blocks

$$\{0\},\ \{1\},\ \{2,3,4\},\ \{5,\ldots,2^4\},\ \{17,\ldots,2^{16}\},\ \{65537,\ldots,2^{65536}\},\ \ldots,$$

where after the first three, each block has the form $\{k+1,\ldots,2^k\}$ with k the largest rank of the

previous block. The block endpoints form a tower of twos, and ranks are at most $\log_2 n$, so only $O(\log^* n)$ blocks are needed. A step $x \to \text{parent}[x]$ makes fast progress when it moves to a larger block.

Definition 22.6 [Good and bad objects]. At a given time, an object x is good if x or its parent is a root, or if rank[parent[x]] lies in a larger rank block than rank[x]. Otherwise x is bad.

Proof of Theorem 22.5. Good objects. Along a path to the root ranks strictly increase, so the block index can increase at most once per block. Hence each Find visits at most 2 + (number of blocks) = $O(\log^* n)$ good objects: the root, its child, and the objects whose parent lies in a larger block. Over m operations this is $O(m \log^* n)$.
Bad objects. Objects of rank 0 or 1 are never bad, since a non-root's parent has larger rank and so lies in a larger block. Fix a block $\{k+1, \ldots, 2^k\}$ and an object x whose rank lies in it; while x is bad, it is a non-root, so this is its final rank. When a Find visits a bad x, its parent is not a root, so compression changes the parent to the root, which has strictly larger rank. The parent's rank starts above $k+1$, and once it exceeds 2^k, x is good forevermore. So x is visited at most 2^k times while bad. By the Rank Lemma, the number of objects with final rank in this block is at most

$$\sum_{i=k+1}^{2^k} \frac{n}{2^i} \le \frac{n}{2^k}.$$

Hence there are at most $2^k \cdot n/2^k = n$ visits to bad objects in this block. The block $\{2, 3, 4\}$ is similar: each of its fewer than $n/2$ objects is visited at most twice while bad, as its parent's rank rises from at least 3 past 4. Over all blocks there are $O(n \log^* n)$ such visits. The total work is $O((m+n) \log^* n)$. □

22.5 The Ackermann function

Many definitions of the Ackermann function exist, all more or less equivalent. Here it is defined recursively for integers $k \ge 0$ and $r \ge 1$.

Definition 22.7 [Ackermann function]. Let $A_0(r) = r + 1$. For $k \ge 1$, let $A_k(r)$ apply A_{k-1} to r, r times:

$$A_k(r) = \underbrace{(A_{k-1} \circ A_{k-1} \circ \cdots \circ A_{k-1})}_{r\text{-fold composition}}(r).$$

Unwinding the first levels:

- $A_1(r) = 2r$: add 1 to r, r times.
- $A_2(r) = r2^r$: double r, r times.

- $A_3(2) = A_2(A_2(2)) = A_2(8) = 8 \cdot 2^8 = 2^{11} = 2048$. In general, since $A_2(s) \geq 2^s$, $A_3(r)$ is at least a tower of r twos.
- $A_4(2) = A_3(A_3(2)) = A_3(2048)$, which is at least a tower of 2048 twos. In general A_4 is roughly an iterated tower function.

By induction on k, each A_k is increasing with $A_k(r) > r$, and $A_k(r) \geq A_{k-1}(r)$; these monotonicity facts are used below.

Definition 22.8 [Inverse Ackermann function]. For every $n \geq 4$, $\alpha(n)$ is the minimum k such that $A_k(2) \geq n$.

$\alpha(n)$	n	$\log^* n$	n
1	$4 \quad (A_1(2) = 4)$	1	2
2	$5, \ldots, 8 \quad (A_2(2) = 8)$	2	$3, 4$
3	$9, \ldots, 2048$	3	$5, \ldots, 16$
4	up to about a tower of 2048 twos	4	$17, \ldots, 65536$
		5	$65537, \ldots, 2^{65536}$

Thus α grows far more slowly than $\log^*$: when n is a tower of 2048 twos, $\log^* n = 2048$ while $\alpha(n) = 4$.

22.6 Tarjan's bound

Theorem 22.9 [Tarjan 1975]. *With union by rank and path compression, m* Union *and* Find *operations take $O((m+n)\alpha(n))$ time, which is $O(m\alpha(n))$ when $m = \Omega(n)$.*

The Hopcroft-Ullman analysis used two facts: the Rank Lemma, and that a compression raising x's parent from p to p' gives $\text{rank}[p'] \geq \text{rank}[p] + 1$. The new idea is a stronger version of the second: in most cases the new parent's rank is much larger, not just larger by one.

Definition 22.10 [Rank gap]. For a non-root object x with $\text{rank}[x] \geq 1$ (so its rank is fixed forever), let $\delta(x)$ be the maximum k such that

$$\text{rank}[\text{parent}[x]] \geq A_k(\text{rank}[x]).$$

Always $\delta(x) \geq 0$, because the parent's rank is at least $\text{rank}[x] + 1 = A_0(\text{rank}[x])$. Further, $\delta(x) \geq 1$ means $\text{rank}[\text{parent}[x]] \geq 2\,\text{rank}[x]$, and $\delta(x) \geq 2$ means $\text{rank}[\text{parent}[x]] \geq \text{rank}[x]\,2^{\text{rank}[x]}$. Since the parent's rank only goes up, $\delta(x)$ only goes up over time. If $\text{rank}[x] \geq 2$, then

$$\text{rank}[\text{parent}[x]] \leq \log_2 n < n \leq A_{\alpha(n)}(2) \leq A_{\alpha(n)}(\text{rank}[x]),$$

so $\delta(x) \in \{0, 1, \ldots, \alpha(n) - 1\}$.

Definition 22.11 [Good and bad objects, revisited]. An object x is bad if all of the following hold: x is not a root, parent$[x]$ is not a root, rank$[x] \geq 2$, and x has an ancestor y with $\delta(y) = \delta(x)$. Otherwise x is good.

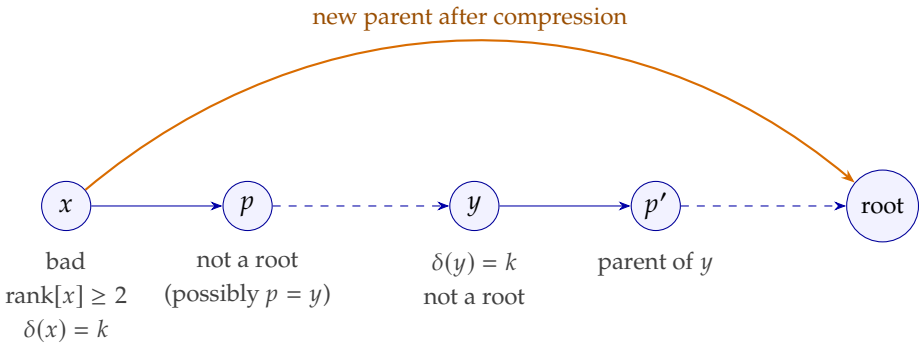


Figure 50: A Find visits a bad object x with $\delta(x) = k$. Its parent p and its ancestor y with $\delta(y) = k$ are not roots ($p = y$ is possible), and p' is the parent of y. Compression makes the root, which is p' or higher, the new parent of x.

Lemma 22.12. *Every object-to-root path contains $O(\alpha(n))$ good objects.*

Proof. Ranks strictly increase along the path, so the good objects are among: the root, the root's child, at most one object of rank 0, at most one of rank 1, and, for each $k \in \{0, \ldots, \alpha(n) - 1\}$, at most one remaining object with $\delta = k$, namely the highest one, since any lower one has that object as an ancestor with the same δ. This is at most $4 + \alpha(n)$ objects. □

Lemma 22.13. *An object x of rank $r \geq 2$ is visited at most $r\alpha(n)$ times while it is bad.*

Proof. Suppose a Find visits x while it is bad, with $\delta(x) = k$, parent p, ancestor y with $\delta(y) = k$, and p' the parent of y. After compression, x's new parent is the root, which is p' or higher, so

$$\text{rank[new parent of } x] \geq \text{rank}[p'] \geq A_k(\text{rank}[y]) \geq A_k(\text{rank}[p]).$$

The first and last inequalities hold because ranks increase toward the root and A_k is increasing; the middle one is $\delta(y) = k$. So each such visit applies (at least) A_k to the rank of x's parent. The parent's rank always exceeds r, so after r bad visits with $\delta(x) = k$ it is at least

$$\underbrace{(A_k \circ \cdots \circ A_k)}_{r \text{ times}}(r) = A_{k+1}(r)$$

by the definition of the Ackermann function, and then $\delta(x) \geq k + 1$. Thus each value of $\delta(x)$ allows at most r visits while x is bad, and $\delta(x)$ takes at most $\alpha(n)$ values. □

Proof of Theorem 22.9. By Lemma 22.12, visits to good objects cost $O(m\alpha(n))$ in total. Bad objects are non-roots, so their ranks are final, and by Lemma 22.13 and the Rank Lemma the visits to bad objects number at most

$$\sum_{\text{objects } x} \text{rank}[x]\,\alpha(n) = \alpha(n) \sum_{r \geq 0} r \cdot (\#\text{objects of rank } r) \leq n\alpha(n) \sum_{r \geq 0} \frac{r}{2^r} = 2n\alpha(n).$$

The total is $O((m+n)\alpha(n))$. □

Tarjan conjectured that no linear-time method exists and that this algorithm is optimal up to a constant factor; Fredman and Saks proved the conjecture in 1989.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Advanced Union-Find.

Lecture 23: Huffman Codes

A binary code maps each character of an alphabet Σ to a binary string. For example, if Σ consists of the letters a–z and some punctuation, 32 characters in all, the obvious encoding uses the 32 binary strings of length 5. This is a fixed-length code. When some characters are much more frequent than others, a variable-length code can do better on average.

23.1 Prefix-free codes

With variable lengths it may be unclear where one character ends and the next begins. For $\Sigma = \{A, B, C, D\}$ with encoding $\{0, 01, 10, 1\}$, the string 001 could encode AB or AAD, so it cannot be decoded without more information. The fixed-length encoding $\{00, 01, 10, 11\}$ has no such problem.

Definition 23.1 [Prefix-free code]. A code f on Σ is prefix-free if for every pair of distinct characters $i, j \in \Sigma$, neither $f(i)$ nor $f(j)$ is a prefix of the other.

For example, $\{0, 10, 110, 111\}$ is prefix-free. Such codes can give shorter encodings when character frequencies are non-uniform:

Character	Frequency	Fixed-length	Prefix-free
A	60%	00	0
B	25%	01	10
C	10%	10	110
D	5%	11	111

The fixed-length code uses 2 bits per character, while the prefix-free code uses on average

$$0.6 \cdot 1 + 0.25 \cdot 2 + (0.1 + 0.05) \cdot 3 = 1.55 \text{ bits per character.}$$

23.2 Codes as trees

Binary codes correspond to binary trees. Left child edges are labeled 0 and right child edges 1; each character $i \in \Sigma$ labels exactly one node, and the encoding of i is the sequence of bits on the path from the root to the node labeled i.

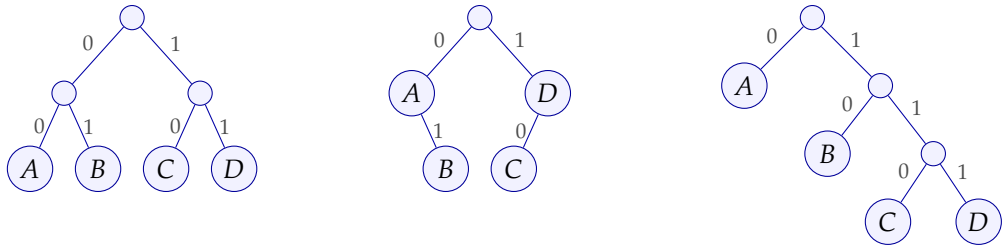


Figure 51: The trees of three codes for $\Sigma = \{A, B, C, D\}$. In the middle tree, labeled nodes A and D are internal, which is why that code is not prefix-free.

One code word is a prefix of another exactly when one labeled node is an ancestor of the other. So a code is prefix-free if and only if the labeled nodes are exactly the leaves. To decode, repeatedly follow the path from the root, according to the bits, until a leaf is reached; output its label and restart at the root. Since only leaves are labeled, this is unambiguous. With $\{0, 10, 110, 111\}$, for example, 0110111 decodes as $0 \mid 110 \mid 111$, that is, ACD. The encoding length of i is the depth of i in the tree.

Definition 23.2 [Optimal prefix-free code]. The input is a probability p_i for each character $i \in \Sigma$. For a binary tree T whose leaves correspond to the symbols of Σ, the average encoding length is

$$L(T) = \sum_{i \in \Sigma} p_i \cdot [\text{depth of } i \text{ in } T].$$

The output is a binary tree T minimizing $L(T)$.

With $p_A = 0.6$, $p_B = 0.25$, $p_C = 0.1$, $p_D = 0.05$, the balanced tree of the fixed-length code has $L = 2$, while the tree of $\{0, 10, 110, 111\}$ has $L = 1.55$.

23.3 Huffman's greedy algorithm

A natural but suboptimal idea is top-down divide and conquer: partition Σ into Σ_1, Σ_2, each with about half of the total frequency, recursively build trees T_1, T_2, and make them the two subtrees of a new root.

Huffman's optimal idea builds the tree bottom-up by successive mergers. Each merger joins two current subtrees under a new root, and increases the encoding length of every symbol in them by one. So the final encoding length of $i \in \Sigma$ is the number of mergers its subtree endures. Symbols that should be deep are those of small frequency, which suggests the greedy rule: first merge the two symbols with the smallest frequencies.

To recurse after merging a and b, replace them by a new meta-symbol ab. Since ab is a proxy for "a or b," its frequency is $p_{ab} = p_a + p_b$.

Algorithm 23.1: Huffman's algorithm

Input: Alphabet Σ with $|\Sigma| \geq 2$ and frequencies p_i.
Output: A binary tree with leaves corresponding to Σ.

if $|\Sigma| = 2$ **then**
  return a root whose two children are the leaves of Σ
Let $a, b \in \Sigma$ have the smallest frequencies
$\Sigma' \leftarrow \Sigma$ with a, b replaced by a new symbol ab
$p_{ab} \leftarrow p_a + p_b$
$T' \leftarrow$ Huffman's algorithm on Σ' with frequencies p
$T \leftarrow T'$ with leaf ab split into an internal node with leaf children a and b
return T

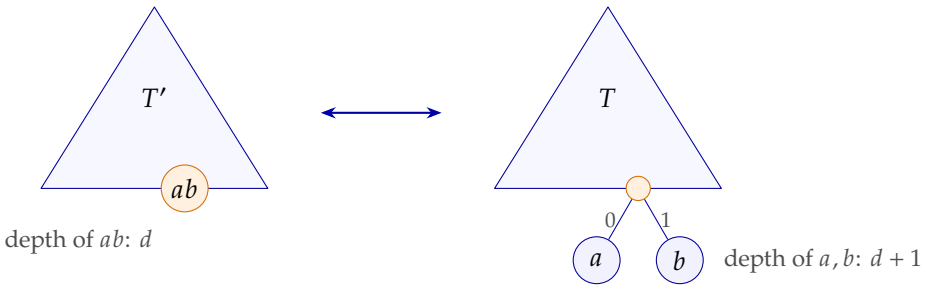


Figure 52: Extending T', whose leaves correspond to Σ', to T, whose leaves correspond to Σ, by splitting the leaf ab.

Example: For $p_A = 60$, $p_B = 25$, $p_C = 10$, $p_D = 5$ (in percent), the algorithm merges C and D into CD with frequency 15, then B and CD into BCD with frequency 40, and the base case pairs A with BCD. Splitting the meta-symbols back gives the tree of $\{0, 10, 110, 111\}$.

Example: Take characters A, B, C, D, E, F with weights $3, 2, 6, 8, 2, 6$. The algorithm merges B and E into BE (weight 4), then A and BE into ABE (weight 7), then C and F into CF (weight 12), then ABE and D into $ABDE$ (weight 15), and the base case pairs $ABDE$ with CF. The resulting code is $A = 000$, $B = 0010$, $E = 0011$, $D = 01$, $C = 10$, $F = 11$.

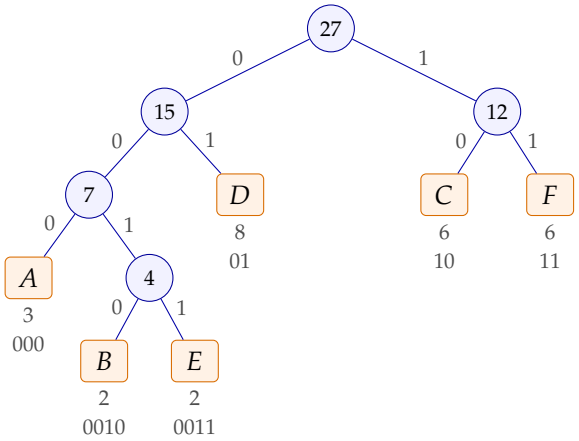


Figure 53: The final tree for the weights $3, 2, 6, 8, 2, 6$. Each internal node shows the total weight of the meta-symbol it represents; below each leaf are its weight and code word.

23.4 Correctness

Theorem 23.3 [Huffman 1952]. *Huffman's algorithm computes a binary tree, with leaves corresponding to the symbols of Σ, that minimizes the average encoding length $L(T) = \sum_{i \in \Sigma} p_i [\text{depth of leaf } i \text{ in } T]$.*

The proof is by induction on $n = |\Sigma| \geq 2$. When $n = 2$, the algorithm uses one bit per symbol, which is optimal since in any tree with at least two leaves every leaf has depth at least 1. For $n > 2$, let a, b be the symbols of smallest frequency, and let Σ' and $p_{ab} = p_a + p_b$ be as in the algorithm. By the inductive hypothesis, the recursive call returns a tree $\hat{T}'$ minimizing L over trees for Σ'.

Let X_{ab} be the set of trees for Σ in which a and b are siblings. Splitting the leaf ab is a bijection from trees T' for Σ' to X_{ab}. If ab has depth d in T', then a and b have depth $d + 1$ in the corresponding T, and all other depths agree, so after cancellation

$$L(T) - L(T') = p_a(d+1) + p_b(d+1) - (p_a + p_b)d = p_a + p_b.$$

This difference is independent of T and T'. Hence the tree $\hat{T}$ corresponding to $\hat{T}'$, which is the algorithm's output, minimizes L over X_{ab}. The theorem therefore follows from the next lemma: merging a and b first is safe.

Lemma 23.4 [Key lemma]. *Some tree minimizing L for Σ lies in X_{ab}.*

The intuition is that an optimal tree can only improve by pushing the least frequent symbols a and b as deep as possible.

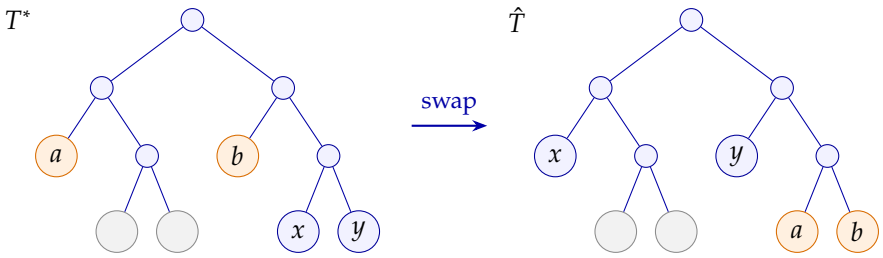


Figure 54: The exchange in the key lemma (schematic). Swapping a with x and b with y moves the two least frequent symbols to the deepest pair of sibling leaves.

Proof. Use an exchange argument. Let T^* minimize L for Σ. If an internal node has only one child, contracting it decreases the depths below it and does not increase L; so we may assume every internal node of T^* has two children. Then a deepest leaf has a sibling, which is also a leaf, since anything below it would be deeper. Let x, y be these two sibling leaves at the deepest level, named so that $x = a$ if a is one of them and $y = b$ if b is one of them.
Obtain $\hat{T}$ from T^* by swapping the labels $a \leftrightarrow x$ and $b \leftrightarrow y$. Then a and b are siblings, so

$\hat{T} \in X_{ab}$. Writing d_i for the depth of i in T^*, only the four swapped symbols change depth, and

$$L(T^*) - L(\hat{T}) = (p_x - p_a)(d_x - d_a) + (p_y - p_b)(d_y - d_b) \geq 0.$$

The frequency differences are nonnegative because a and b have the smallest frequencies, and the depth differences are nonnegative because x and y are at the deepest level. So $L(\hat{T}) \leq L(T^*)$, and $\hat{T}$ is also optimal. □

23.5 Running time

A naive implementation takes $O(n^2)$ time for $n = |\Sigma|$: there are $n-2$ recursive calls, each finding the two smallest frequencies in $O(n)$ time. A heap speeds up these repeated minimum computations. Use the frequencies as keys; after extracting the two smallest-frequency (meta-)symbols, insert the new meta-symbol with key equal to the sum of the two old keys. This gives an iterative $O(n \log n)$ implementation. Even faster, after sorting the frequencies, the remaining work can be done in $O(n)$ time by managing the (meta-)symbols with two queues.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Huffman Codes.

Lecture 24: Introduction to Dynamic Programming

Dynamic programming solves a problem by reasoning about the structure of an optimal solution, identifying a small collection of subproblems, and solving them from smallest to largest. A simple graph problem shows why greedy and divide-and-conquer approaches can fail and how the technique arises.

24.1 Weighted independent sets in path graphs

A path graph has vertices $v_1, \ldots, v_n$ and edges (v_i, v_{i+1}) for $1 \le i < n$. Each vertex v_i carries a nonnegative weight w_i.

Definition 24.1 [Independent set]. An independent set (IS) of a graph is a subset of its vertices, no two of which are adjacent. The weighted independent set problem asks for an independent set of maximum total weight.

The empty set is always independent. Brute-force search over all subsets requires exponential time.

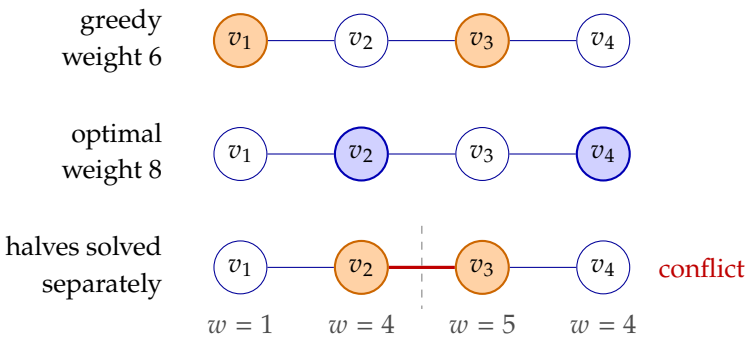


Figure 55: The path with weights $1, 4, 5, 4$. Greedy selection picks v_3, then v_1, for total weight 6; the optimum $\{v_2, v_4\}$ has weight 8. Solving each half separately picks the adjacent vertices v_2 and v_3.

The natural greedy algorithm repeatedly chooses the maximum-weight vertex not adjacent to any previously chosen vertex. On the path in Figure 55, it chooses v_3 and then v_1, for weight 6, while the maximum weight is 8.

A divide-and-conquer approach would compute a maximum-weight independent set of each half recursively and then combine them. The two recursive solutions may conflict across the boundary: in Figure 55, the halves contribute v_2 and v_3, which are adjacent. It is not clear how to repair such a conflict quickly.

24.2 Optimal substructure

The critical step is to reason about the structure of an optimal solution in terms of optimal solutions of smaller subproblems. This narrows the candidates for an optimal solution to a small set that can be searched by brute force.

Let $S \subseteq V$ be a maximum-weight independent set of G. Let G' be G with v_n deleted, and let G'' be G with v_{n-1} and v_n deleted. For $n = 1$, G'' is the empty graph.

Lemma 24.2 [Optimal substructure]. *If $v_n \notin S$, then S is a maximum-weight IS of G'. If $v_n \in S$, then $S - \{v_n\}$ is a maximum-weight IS of G''.*

Proof. *Case $v_n \notin S$.* Then S is an IS of G'. If some IS S^* of G' had larger weight, then S^* would also be an IS of G with larger weight than S, a contradiction.

Case $v_n \in S$. By independence, $v_{n-1} \notin S$, so $S - \{v_n\}$ is an IS of G''. If some IS S^* of G'' had larger weight, then $S^* \cup \{v_n\}$ would be independent in G, because S^* contains neither v_{n-1} nor v_n, and its weight would exceed that of S, a contradiction. □

Hence a maximum-weight IS of G is either a maximum-weight IS of G', or v_n together with a maximum-weight IS of G''. If we knew which case held, one recursive call would finish the job. Since we do not know, we try both possibilities and return the better solution.

24.3 Eliminating redundant recursion

The resulting recursive algorithm computes a maximum-weight IS S_1 of G' and S_2 of G'', then returns the heavier of S_1 and $S_2 \cup \{v_n\}$. It is correct by induction on n using Lemma 24.2. However, its running time satisfies $T(n) = T(n-1) + T(n-2) + O(1)$, which grows exponentially.

The recursion only removes vertices from the right end of the path, so every subproblem is a prefix of the graph. There are only $\Theta(n)$ distinct subproblems, which the exponential recursion solves over and over. One fix is memoization: the first time a subproblem is solved, store its solution in a global table for $O(1)$-time lookup later. Better still, solve the subproblems bottom-up.

Let G_i be the path on the first i vertices, with G_0 empty. Fill an array A from left to right so that $A[i]$ is the value of a maximum-weight IS of G_i.

Algorithm 24.1: Weighted independent set in a path graph

Input: Path graph with vertex weights $w_1, \dots, w_n \geq 0$.
Output: Array A with $A[i]$ the maximum weight of an IS of G_i.

```
A[0] ← 0; A[1] ← w_1
for i ← 2 to n do
    A[i] ← max{A[i − 1], A[i − 2] + w_i}
return A
```

The first argument of the maximum is case 1 (a maximum-weight IS of G_{i-1}); the second is case 2 (a maximum-weight IS of G_{i-2} plus v_i). Each iteration takes constant time, so the algorithm runs in $O(n)$ time.

Proposition 24.3 [Correctness]. *For every $0 \le i \le n$, $A[i]$ is the maximum weight of an independent set of G_i.*

Proof. Induct on i. The cases $i = 0, 1$ hold because $\emptyset$ and $\{v_1\}$ are optimal for G_0 and G_1 (weights are nonnegative). For $i \ge 2$, Lemma 24.2 applied to G_i says that an optimal IS of G_i has weight $A[i-1]$ or $A[i-2] + w_i$, and both values are attained by independent sets of G_i. Taking the maximum gives the optimum. □

24.4 Reconstructing an optimal solution

The algorithm computes the value of a maximum-weight IS, not the set. Storing an optimal set for each G_i alongside its value is correct but wasteful. Instead, trace back through the filled-in array. By the correctness of the algorithm, v_i belongs to a maximum-weight IS of G_i if and only if

$$w_i + A[i-2] \ge A[i-1].$$

For the test at $i = 1$, interpret $A[-1] = 0$.

Algorithm 24.2: Reconstruction

Input: Filled array $A[0:n]$ and weights $w_1, \dots, w_n$.
Output: A maximum-weight IS S of G.

```
S ← ∅; i ← n
while i ≥ 1 do
    if A[i − 1] ≥ A[i − 2] + w_i then          // case 1 wins
        i ← i − 1
    else                                       // case 2 wins
        S ← S ∪ {v_i}; i ← i − 2
return S
```

Proposition 24.4. *The reconstruction returns a maximum-weight IS of G in $O(n)$ time.*

Proof. Show by induction on i that the remaining loop, started at i, adds an IS of G_i of weight $A[i]$. If case 1 wins, then $A[i] = A[i-1]$ and the loop continues on G_{i-1}. Otherwise $A[i] = A[i-2] + w_i$; the loop adds v_i and continues on G_{i-2}, which does not contain the neighbor v_{i-1}, so independence is preserved. Each iteration decreases i, so there are at most n iterations. □

24.5 Principles of dynamic programming

The weighted independent set algorithm is a dynamic programming algorithm. Its key ingredients are the following.

1. Identify a small number of subproblems, such as the maximum-weight IS of G_i for $i = 0, 1, \ldots, n$.
2. Quickly and correctly solve larger subproblems given the solutions to smaller ones, usually via a recurrence such as $A[i] = \max\{A[i-1], A[i-2] + w_i\}$.
3. After solving all subproblems, quickly compute the final solution, usually the answer to the largest subproblem.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Introduction to Dynamic Programming.

Lecture 25: The Knapsack Problem

The knapsack problem chooses a subset of items of maximum total value subject to a capacity constraint. The dynamic programming algorithm follows the same three steps as for weighted independent sets: formulate a recurrence from the structure of an optimal solution, identify the subproblems, and solve them systematically.

Definition 25.1 [Knapsack problem]. The input is n items, where item i has a nonnegative value v_i and a nonnegative integral size w_i, together with a nonnegative integer capacity W. The output is a subset $S \subseteq \{1, 2, \ldots, n\}$ that maximizes $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

25.1 Optimal substructure

Let S be a maximum-value solution to a knapsack instance, and consider the last item n.

Lemma 25.2 [Optimal substructure]. *If $n \notin S$, then S is an optimal solution for the first $n-1$ items with the same capacity W. If $n \in S$, then $S - \{n\}$ is an optimal solution for the first $n-1$ items with capacity $W - w_n$.*

Proof. Case $n \notin S$. If some S^* using only the first $n-1$ items were feasible for capacity W with larger value than S, it would be equally feasible and better with respect to all n items, a contradiction.

Case $n \in S$. The set $S - \{n\}$ has total size at most $W - w_n$, so it is feasible for the smaller capacity. If some S^* using the first $n-1$ items had total size at most $W - w_n$ and higher value than $S - \{n\}$, then $S^* \cup \{n\}$ would have size at most W and value greater than S, a contradiction. □

The residual capacity in the second case is $W - w_n$, the capacity left after reserving room for item n; it is not $W - v_n$.

25.2 The recurrence and the subproblems

Let $V_{i,x}$ be the value of the best solution that uses only the first i items and has total size at most x. Applying Lemma 25.2 to this smaller instance gives, for $i \in \{1, \ldots, n\}$ and every x,

$$V_{i,x} = \max\{V_{i-1,x},\ v_i + V_{i-1,x-w_i}\}.$$

The first term is case 1 (item i excluded) and the second is case 2 (item i included). If $w_i > x$, item i cannot fit, so $V_{i,x} = V_{i-1,x}$.

The subproblems are indexed by all prefixes $\{1, 2, \dots, i\}$ of the items, $0 \le i \le n$, and all residual capacities $x \in \{0, 1, \dots, W\}$. Because W and the sizes w_i are integers, every residual capacity reached by the recurrence is one of these $W + 1$ values.

Algorithm 25.1: Knapsack

Input: Values $v_1, \dots, v_n$, integral sizes $w_1, \dots, w_n$, and capacity W.
Output: The maximum total value of a feasible subset.

```
A ← a 2-D array indexed by 0 ≤ i ≤ n and 0 ≤ x ≤ W
for x ← 0 to W do
    A[0, x] ← 0
for i ← 1 to n do
    for x ← 0 to W do
        if w_i > x then
            A[i, x] ← A[i − 1, x]
        else
            A[i, x] ← max{A[i − 1, x], A[i − 1, x − w_i] + v_i}
return A[n, W]
```

Both entries on the right-hand side were computed in the previous iteration of the outer loop, so each is available for $O(1)$-time lookup.

Proposition 25.3 [Correctness and running time]. *The algorithm returns the optimal value $V_{n,W}$ in $\Theta(nW)$ time.*

Proof. By induction on i, $A[i, x] = V_{i,x}$ for every x: the case $i = 0$ holds since no items give value 0, and the inductive step is exactly the recurrence justified by Lemma 25.2. There are $(n + 1)(W + 1) = \Theta(nW)$ subproblems, each solved in $\Theta(1)$ time. □

25.3 An example

Take $n = 4$ items and capacity $W = 6$, with

$$(v_1, w_1) = (3, 4), \quad (v_2, w_2) = (2, 3), \quad (v_3, w_3) = (4, 2), \quad (v_4, w_4) = (4, 3).$$

Filling the table column by column gives Figure 56. The optimal value is $A[4, 6] = 8$.

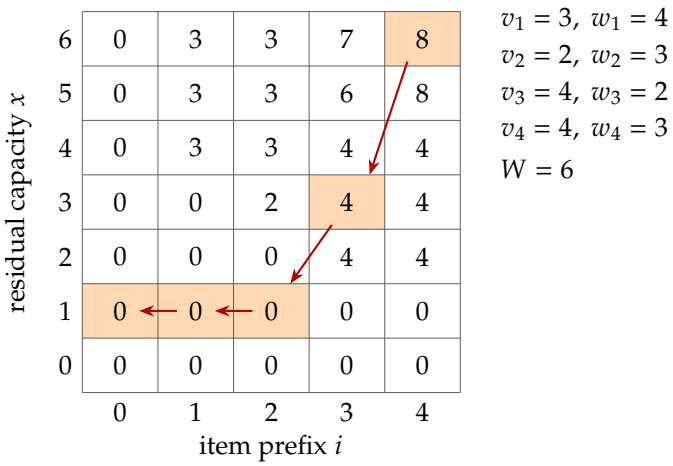


Figure 56: The filled table $A[i, x]$ for the example, with column i and row x. Shaded cells and arrows trace back an optimal solution from $A[4, 6]$.

An optimal subset can be read off by tracing back from $A[n, W]$. At entry $A[i, x]$, if $A[i, x] = A[i-1, x]$ then item i can be excluded and we move to $A[i-1, x]$; otherwise item i is included and we move to $A[i-1, x-w_i]$. Here $A[4, 6] = 8 \neq A[3, 6] = 7$, so item 4 is included and we move to $A[3, 3]$. Then $A[3, 3] = 4 \neq A[2, 3] = 2$, so item 3 is included and we move to $A[2, 1]$. Finally $A[2, 1] = A[1, 1] = A[0, 1] = 0$, so items 2 and 1 are excluded. The optimal solution is $\{3, 4\}$, with total size $5 \leq 6$ and value 8.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Knapsack Problem.

Lecture 26: Sequence Alignment

Sequence alignment compares DNA strings to study functional similarity or evolutionary relationships. It measures their similarity by the Needleman–Wunsch score: the minimum total penalty of an alignment. Dynamic programming computes it by peeling letters off the right ends of the two strings.

26.1 Problem definition

The input is two strings $X = x_1 \cdots x_m$ and $Y = y_1 \cdots y_n$ over an alphabet Σ, such as $\{A, C, G, T\}$, together with a penalty $\alpha_{\text{gap}} \geq 0$ for each inserted gap and a penalty $\alpha_{ab} \geq 0$ for matching letters a and b, presumably with $\alpha_{ab} = 0$ when $a = b$.

Definition 26.1 [Alignment]. An alignment of X and Y inserts gaps into the two strings so that they have equal length, then pairs them position by position. Its penalty is α_{gap} for each gap plus α_{ab} for each position that pairs a letter a of X with a letter b of Y. The goal is an alignment of minimum total penalty.

Example: The alignment

A	G	G	G	C	T
A	G	G	–	C	A

of AGGGCT and AGGCA has total penalty $\alpha_{\text{gap}} + \alpha_{\text{AT}}$.

Even gap-only interleavings give $\binom{m+n}{m}$ alignments, at least 2^n when $m = n$. Enumerating all alignments is therefore exponential.

26.2 Optimal substructure

To identify subproblems, develop a recurrence from the structure of an optimal solution and then read off the subproblems it uses. Consider an optimal alignment of X and Y and its final position. There are three relevant possibilities for that position:

1. x_m and y_n are matched;
2. x_m is matched with a gap;
3. y_n is matched with a gap.

A position with two gaps is pointless: deleting it gives an alignment with no larger penalty.

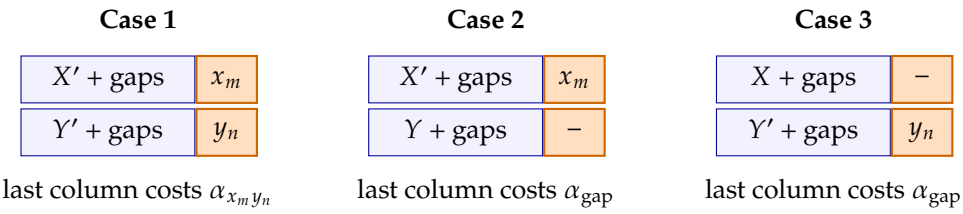


Figure 57: The three possible final positions of an optimal alignment. The remaining positions form an alignment of the indicated smaller strings.

Let $X' = x_1 \cdots x_{m-1}$ and $Y' = y_1 \cdots y_{n-1}$ denote the strings with their last letters removed.

Lemma 26.2 [Optimal substructure]. *In an optimal alignment of X and Y, the alignment induced by the other positions is optimal for X' and Y' in case 1, for X' and Y in case 2, and for X and Y' in case 3.*

Proof. Consider case 1; the other cases are similar. Suppose the induced alignment of X' and Y' has penalty P, while some other alignment of X' and Y' has penalty $P^* < P$. The original alignment has penalty $P + \alpha_{x_m y_n}$. Appending the final position x_m over y_n to the better alignment gives an alignment of X and Y with penalty

$$P^* + \alpha_{x_m y_n} < P + \alpha_{x_m y_n},$$

contradicting the optimality of the original alignment. In cases 2 and 3, the appended final position contributes α_{gap} instead. □

Thus an optimal alignment is one of only three candidates, each built from an optimal alignment of a smaller pair of strings.

26.3 Subproblems and recurrence

Since letters are only removed from the right ends, the relevant subproblems are pairs (X_i, Y_j), where X_i is the first i letters of X and Y_j is the first j letters of Y. Let P_{ij} be the penalty of an optimal alignment of X_i and Y_j. For all $i = 1, \ldots, m$ and $j = 1, \ldots, n$,

$$P_{ij} = \min \begin{cases} \alpha_{x_i y_j} + P_{i-1,j-1} & \text{case 1,} \\ \alpha_{\text{gap}} + P_{i-1,j} & \text{case 2,} \\ \alpha_{\text{gap}} + P_{i,j-1} & \text{case 3.} \end{cases}$$

This is correct because the optimal solution is one of these three candidates by Lemma 26.2, and the recurrence selects the best of them.

The base cases are $P_{i,0} = P_{0,i} = i \cdot \alpha_{\text{gap}}$: the only way to align i letters with the empty string, without pointless double-gap positions, is to match each letter with a gap.

26.4 The algorithm

Algorithm 26.1: Sequence alignment

Input: Strings $X = x_1 \cdots x_m$, $Y = y_1 \cdots y_n$, and penalties α_{gap}, α_{ab}.
Output: The minimum penalty P_{mn} of an alignment.

```
A ← a 2-D array indexed by 0 ≤ i ≤ m and 0 ≤ j ≤ n
for i ← 0 to m do A[i, 0] ← i · α_gap
for j ← 0 to n do A[0, j] ← j · α_gap
for i ← 1 to m do
    for j ← 1 to n do
        A[i, j] ← min{A[i − 1, j − 1] + α_{x_i y_j}, A[i − 1, j] + α_gap, A[i, j − 1] + α_gap}
return A[m, n]
```

All three entries on the right-hand side were filled earlier in the loop order, so each is available for $O(1)$-time lookup. By induction and the correctness of the recurrence, $A[i, j] = P_{ij}$ for all $i, j \geq 0$. The running time is $O(mn)$: $\Theta(1)$ work for each of $\Theta(mn)$ subproblems.

26.5 Reconstructing an optimal alignment

Trace back through the filled-in table, starting at $A[m, n]$. At entry $A[i, j]$:

- if $A[i, j]$ was filled using case 1, match x_i with y_j and go to $A[i - 1, j - 1]$;
- if it was filled using case 2, match x_i with a gap and go to $A[i - 1, j]$;
- if it was filled using case 3, match y_j with a gap and go to $A[i, j - 1]$.

When $i = 0$ or $j = 0$, match the remaining letters of the other string with gaps. The positions are produced from right to left. Each step decreases $i + j$, so the traceback takes only $O(m + n)$ time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Sequence Alignment; Introduction: Two Motivating Applications.

Lecture 27: Optimal Binary Search Trees

A set of keys has many valid binary search trees. When some keys are searched for much more often than others, the best tree is not necessarily balanced, and dynamic programming finds a tree of minimum average search time.

27.1 Search trees and non-uniform access

A binary search tree satisfies the search tree property: for every node with key x, all keys in its left subtree are less than x and all keys in its right subtree are greater than x. For keys $x < y < z$, for example, the tree rooted at y with children x and z, the path $x \to y \to z$ of right children, and the path $z \to y \to x$ of left children are all valid.

A balanced search tree, such as a red-black tree, has worst-case search time $\Theta(\text{height}) = \Theta(\log n)$. With known search frequencies, however, a different tree can be better on average.

Example: Suppose 80% of searches are for x and 10% each are for y and z. Counting the number of nodes examined, the balanced tree has average search time 1.9, while the path rooted at x has average search time 1.3 (Figure 58).

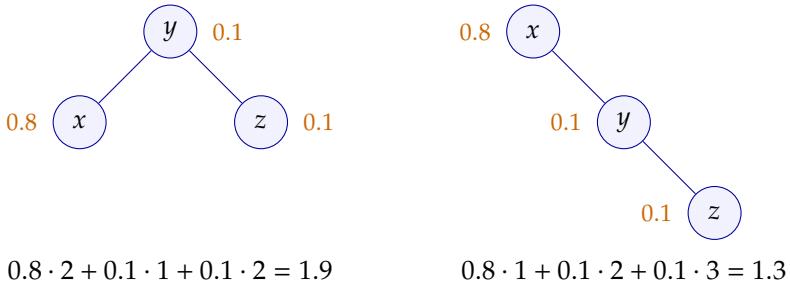


Figure 58: Average search times for keys $x < y < z$ with search frequencies 0.8, 0.1, 0.1, shown beside each node. A search for a key at depth d examines $d + 1$ nodes.

27.2 Problem definition

Definition 27.1 [Optimal binary search tree]. The input is frequencies $p_1, \ldots, p_n$ for items $1, 2, \ldots, n$, where the items are in sorted order $1 < 2 < \cdots < n$. The goal is a valid search tree T that minimizes the weighted search time

$$C(T) = \sum_{i=1}^{n} p_i \cdot [\text{search time for } i \text{ in } T],$$

where the search time for i is its depth in T plus one.

If $\sum_i p_i = 1$, then $C(T)$ is an average search time, and a red-black tree T has $C(T) = O(\log n)$.

The problem resembles Huffman coding: both output a binary tree and essentially minimize average depth with respect to given probabilities. The constraints differ. Huffman codes must be prefix-free, so symbols appear only at leaves; here every node holds a key and the tree must satisfy the search tree property, which seems harder to handle.

27.3 Greedy does not work

Intuitively, the most frequently accessed items should be closest to the root and the least frequently accessed ones furthest from it. This suggests two greedy algorithms:

- bottom-up: populate the lowest level with the least frequently accessed keys;
- top-down: put the most frequently accessed item at the root, then recurse on the two sides.

Neither is optimal in general (Figure 59). In the first instance, the greedy tree keeps the rarely accessed keys 1 and 4 on the lowest levels but forces key 3 (frequency 73) below the root, for cost 179 instead of 129. In the second instance, top-down greedy puts key 2 (frequency 34) at the root and then key 3 (frequency 33) below it, for cost 168; rooting the tree at key 3 instead gives cost 148.

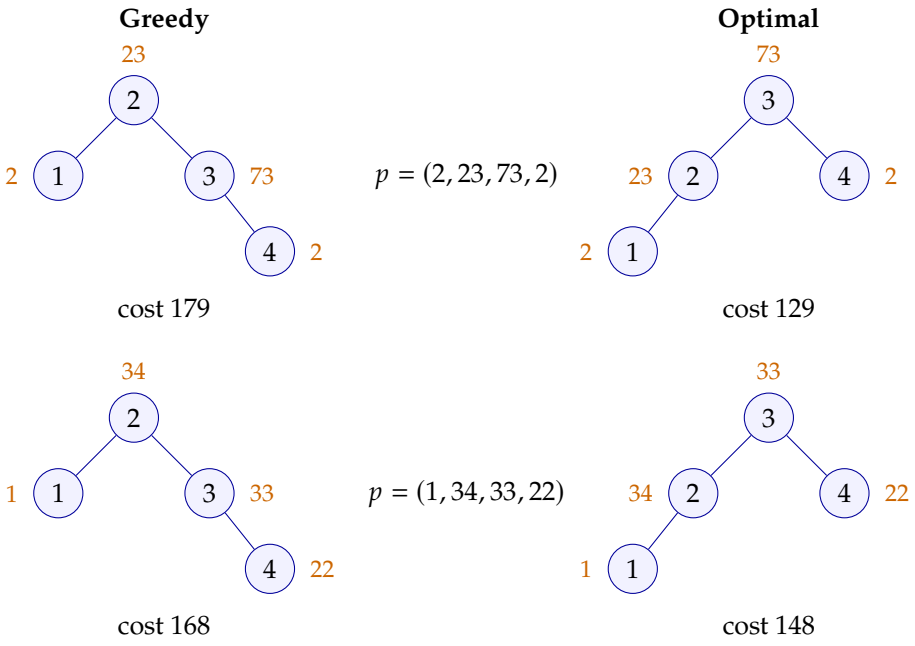


Figure 59: Two four-key instances, with frequencies beside the nodes. In each row, the greedy tree on the left has higher cost than the optimal tree on the right.

With the top-down approach, the choice of root has hard-to-predict repercussions further down the tree; this defeats both greedy and naive divide-and-conquer approaches. But if we knew the root,

the rest of the problem would split into two independent subproblems. A dynamic programming algorithm can try all possible roots.

27.4 Optimal substructure

Lemma 27.2 [Optimal substructure]. *If T is an optimal BST for the keys $\{1, 2, \ldots, n\}$ with root r, then its left subtree T_1 and right subtree T_2 are optimal BSTs for the keys $\{1, \ldots, r-1\}$ and $\{r+1, \ldots, n\}$, respectively.*

The proof rests on a decomposition of the cost. A search for $i < r$ examines the root and then follows the search for i in T_1; similarly for $i > r$ in T_2. Hence

$$\begin{aligned} C(T) &= p_r \cdot 1 + \sum_{i=1}^{r-1} p_i\big(1 + [\text{search time for } i \text{ in } T_1]\big) + \sum_{i=r+1}^{n} p_i\big(1 + [\text{search time for } i \text{ in } T_2]\big) \\ &= \sum_{i=1}^{n} p_i + C(T_1) + C(T_2). \end{aligned}$$

The first term is a constant, independent of the tree.

Proof. Suppose for contradiction that T_1 is not optimal for $\{1, \ldots, r-1\}$, so some BST T_1^* on these keys has $C(T_1^*) < C(T_1)$; the case of T_2 is similar. Obtain T^* from T by cutting out T_1 and pasting in T_1^*. Since every key of T_1^* is less than r, T^* is a valid search tree. By the decomposition,

$$C(T^*) = \sum_{i=1}^{n} p_i + C(T_1^*) + C(T_2) < \sum_{i=1}^{n} p_i + C(T_1) + C(T_2) = C(T),$$

contradicting the optimality of T. □

27.5 Subproblems and recurrence

The items in a subproblem of Lemma 27.2 are a prefix or a suffix of the original items. Applying the lemma again inside such a subproblem produces a prefix or suffix of that range. The relevant subproblems are therefore the contiguous intervals $\{i, i+1, \ldots, j\}$ for $1 \le i \le j \le n$; prefixes alone, or prefixes and suffixes alone, are not enough.

For $1 \le i \le j \le n$, let C_{ij} be the weighted search cost of an optimal BST for the items $\{i, i+1, \ldots, j\}$ with frequencies $p_i, \ldots, p_j$. Interpret $C_{xy} = 0$ if $x > y$. Then, for every $1 \le i \le j \le n$,

$$C_{ij} = \min_{r=i,\ldots,j} \left\{ \sum_{k=i}^{j} p_k + C_{i,r-1} + C_{r+1,j} \right\}.$$

This follows from the decomposition $C(T) = \sum_k p_k + C(T_1) + C(T_2)$: optimal substructure narrows the candidates down to $j - i + 1$ possibilities, one for each root, and the recurrence picks the best by brute force.

27.6 The algorithm

Solve the smallest subproblems, those with the fewest items $j - i + 1$, first. The index $s = j - i$ counts rounds, and $A[i, j]$ holds the optimal value for the items $\{i, \dots, j\}$.

Algorithm 27.1: Optimal binary search tree

Input: Frequencies $p_1, \dots, p_n$ for items $1 < 2 < \cdots < n$.
Output: The minimum weighted search time C_{1n}.

$A \leftarrow$ a 2-D array, with $A[x, y]$ interpreted as 0 whenever $x > y$
for $s \leftarrow 0$ **to** $n - 1$ **do**
  for $i \leftarrow 1$ **to** $n - s$ **do**
    $A[i, i + s] \leftarrow \min_{r=i,\dots,i+s} \left\{ \sum_{k=i}^{i+s} p_k + A[i, r - 1] + A[r + 1, i + s] \right\}$
return $A[1, n]$

Here $i + s$ plays the role of j. The entries $A[i, r - 1]$ and $A[r + 1, i + s]$ concern fewer items, so they were computed in an earlier round and are available for $O(1)$-time lookup. In particular, $A[i, i] = p_i$.

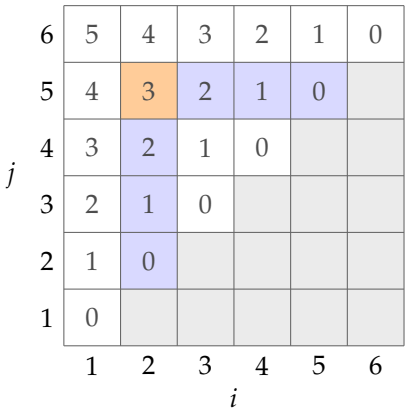


Figure 60: Schematic fill order for $n = 6$, with column i and row j. Each used cell is labelled by its round $s = j - i$; gray cells have $j < i$. The entry $A[2, 5]$ (orange) reads the shaded entries $A[2, r - 1]$ below it and $A[r + 1, 5]$ to its right, all from earlier rounds, together with empty intervals that count as 0.

Proposition 27.3 [Running time]. *The algorithm runs in $\Theta(n^3)$ time.*

Proof. There are $\Theta(n^2)$ subproblems. Computing $A[i, j]$ takes $\Theta(j - i)$ time: one pass for the sum $\sum_{k=i}^{j} p_k$, which is the same for every root, and one pass over the $j - i + 1$ candidate roots. Summing over all intervals gives $\Theta(n^3)$. □

An optimized version of this algorithm, due to Knuth (1971) and Yao (1980), correctly fills the entire

table in only $\Theta(n^2)$ time, that is, $\Theta(1)$ on average per subproblem. It piggybacks on work done in previous subproblems to avoid trying all possible roots.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Optimal Binary Search Trees.

Lecture 28: The Bellman-Ford Algorithm

In the single-source shortest path problem, the input is a directed graph $G = (V, E)$ with n vertices and m edges, a length c_e for each edge $e \in E$, and a source vertex $s \in V$. We may assume there are no parallel edges. The goal is to compute, for every destination $v \in V$, the length (sum of edge lengths) of a shortest s-v path.

Dijkstra's algorithm runs in $O(m \log n)$ time with heaps, but it can fail with negative edge lengths, as when edges represent financial transactions, and it is not naturally distributed, which matters for Internet routing. The Bellman-Ford algorithm addresses both issues.

28.1 Negative cycles

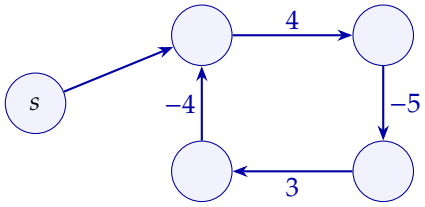


Figure 61: A negative cycle reachable from s. One trip around the cycle has length $4 - 5 + 3 - 4 = -2$, so repeated trips make walks arbitrarily short.

If a negative cycle is reachable from s, as in Figure 61, the notion of a shortest path needs care.

- Allowing cycles makes some distances undefined or $-\infty$, since a walk can keep traversing the negative cycle.
- Requiring cycle-free paths gives an NP-hard problem, so there is no polynomial-time algorithm unless P = NP.
- For now, assume the input graph has no negative cycles. We show later how to check this condition quickly.

Proposition 28.1 [Few edges suffice]. *If G has no negative cycles, then for every v reachable from s there is a shortest s-v path with at most $n - 1$ edges.*

Proof. Take any s-v walk. If it repeats a vertex, it contains a cycle, of nonnegative length, and removing the cycle gives an s-v walk that is no longer. Repeating this yields a cycle-free path that is at least as short. Such a path visits at most n vertices, so it has at most $n - 1$ edges. □

In general, the goal is either (A) to compute the length of a shortest s-v path for every $v \in V$, or (B) to output a negative cycle as the reason for failing to do so.

28.2 Optimal substructure

A subpath of a shortest path should itself be shortest, but in a graph it is not clear which subproblems are smaller than others. The key idea is to restrict the number of edges in a path artificially. The size of a subproblem is the number of permitted edges.

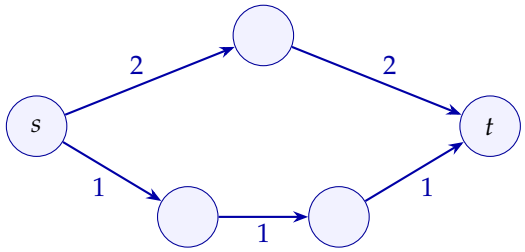


Figure 62: Restricting the number of edges. With at most two edges, the shortest s-t path is the upper one, of length 4. With three edges, the lower path has length 3.

Lemma 28.2 [Optimal substructure]. *Let $G = (V, E)$ be a directed graph with edge lengths c_e and source s, with or without negative cycles. For $v \in V$ and $i \in \{1, 2, \ldots\}$, let P be a shortest s-v path with at most i edges, where cycles are permitted.*

1. *If P has at most $i-1$ edges, then P is a shortest s-v path with at most $i-1$ edges.*
2. *If P has exactly i edges and last hop (w, v), then the prefix P' of P ending at w is a shortest s-w path with at most $i-1$ edges.*

Proof. Case 1: every path with at most $i-1$ edges also has at most i edges, so a shorter one would contradict the optimality of P.
Case 2: suppose an s-w path Q with at most $i-1$ edges is shorter than P'. Then $Q + (w, v)$ is an s-v path with at most i edges, and it is shorter than $P' + (w, v) = P$. This contradicts the optimality of P. □

Thus an optimal solution for destination v has only 1 + in-deg(v) candidates: one from Case 1, and one from Case 2 for each choice of the final hop (w, v).

28.3 The recurrence and the algorithm

Let $L_{i,v}$ be the minimum length of an s-v path with at most i edges, with cycles allowed, and $L_{i,v} = +\infty$ if no such path exists. For every $v \in V$ and $i \geq 1$,

$$L_{i,v} = \min\Big\{ L_{i-1,v},\ \min_{(w,v)\in E} \{L_{i-1,w} + c_{wv}\} \Big\}.$$

This is correct because it is a brute-force search over the $1 + \text{in-deg}(v)$ candidates from Lemma 28.2.

If G has no negative cycles, shortest paths need no cycles, since removing a cycle does not increase the length, and so they have at most $n-1$ edges by Proposition 28.1. It therefore suffices to compute $L_{i,v}$ for $i \in \{0, 1, \ldots, n-1\}$ and all $v \in V$; the answers are the $L_{n-1,v}$.

Algorithm 28.1: Bellman-Ford

Input: Directed graph $G = (V, E)$, edge lengths c_e, source $s \in V$.
Output: Shortest-path distances $A[n-1, v]$ from s, if G has no negative cycle.

```
A ← a two-dimensional array indexed by i and v
A[0, s] ← 0; A[0, v] ← +∞ for all v ≠ s
for i ← 1 to n − 1 do
    for each v ∈ V do
        A[i, v] ← min{A[i − 1, v], min_{(w,v)∈E} {A[i − 1, w] + c_wv}}
return A[n − 1, v] for all v ∈ V
```

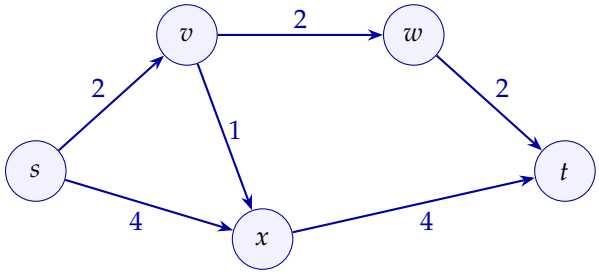


Figure 63: A small input for Bellman-Ford with source s. The rows of $A[i, v]$ appear in the table below.

i	s	v	w	x	t
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	2	$+\infty$	4	$+\infty$
2	0	2	4	3	8
3	0	2	4	3	6
4	0	2	4	3	6

In Figure 63, $A[2, x] = 3$ uses the two-edge path $s \to v \to x$, which beats the direct edge of length 4. At t, the two-edge path through x gives 8, and the three-edge path $s \to v \to w \to t$ gives 6.

Running time. There are $\Theta(n^2)$ subproblems, but one subproblem can take $\Theta(n)$ work, so counting

subproblems gives only $O(n^3)$. Instead, for each i the work is $O\big(\sum_{v \in V} \text{in-deg}(v)\big) = O(m)$, so the total is $O(mn)$.

Stopping early. Row i of A depends only on row $i-1$. If $A[j, v] = A[j-1, v]$ for all v for some $j < n-1$, then all later rows equal row j. In particular $A[n-1, v] = A[j, v]$, and the algorithm can halt. In the table, rows 3 and 4 agree.

28.4 Detecting negative cycles

To report a negative cycle when one exists, run one extra iteration, for $i = n$, still in $O(mn)$ time.

Proposition 28.3 [Negative-cycle check]. *G has no negative cycle reachable from s if and only if $A[n-1, v] = A[n, v]$ for all $v \in V$ in the extended Bellman-Ford algorithm.*

Proof. ($\Rightarrow$) With no reachable negative cycle, removing cycles from an s-v walk never increases its length, so no walk is shorter than a shortest path, which has at most $n-1$ edges. Hence $A[n-1, v]$ and $A[n, v]$ both equal the distance.
($\Leftarrow$) Suppose $A[n-1, v] = A[n, v]$ for all v, and call the common value $d(v)$. Consider a cycle C reachable from s; its vertices have finite values d. By the recurrence,

$$d(v) = A[n, v] \le A[n-1, w] + c_{wv} = d(w) + c_{wv} \qquad \text{for every edge } (w, v) \in E,$$

that is, $c_{wv} \ge d(v) - d(w)$. Summing over the edges of C, the right side telescopes:

$$\sum_{(w,v) \in C} c_{wv} \;\ge\; \sum_{(w,v) \in C} \big(d(v) - d(w)\big) = 0.$$

So no reachable cycle is negative. □

28.5 Space and predecessor pointers

The basic algorithm stores $\Theta(n^2)$ values, $\Theta(1)$ for each of n^2 subproblems. Since row i uses only row $i-1$, it is enough to keep the current and previous rounds, which takes $O(n)$ space, $O(1)$ per destination.

Without the full table, paths are recovered with a second table of predecessor pointers: $B[i, v]$ is the second-to-last vertex on a shortest s-v path with at most i edges, or NULL if no such path exists. Set $B[0, v]$ = NULL for all v. For $i > 0$:

- if Case 1 of the recurrence wins, $B[i, v] = B[i-1, v]$;
- if Case 2 wins, $B[i, v]$ is the vertex w achieving the minimum, that is, the new last hop.

Thus $B[i, v]$ caches the last hop of the winner in the brute-force search for $A[i, v]$.

If G has no negative cycles, $B[n-1, v]$ is the last hop of a shortest s-v path. Tracing the pointers back from v to s yields a shortest s-v path, by optimal substructure. To find a negative cycle instead, check after each round, with depth-first search, whether the predecessor pointers form a cycle; such a cycle must have negative cost.

28.6 From Bellman-Ford to Internet routing

Bellman-Ford is naturally distributed: each vertex updates its value using only the values of its neighbors. Three changes lead to a routing protocol.

Destination-driven. Reverse all edge directions. Each vertex v stores its shortest-path distance to a destination t and the first hop of a shortest path, for every relevant destination t. These are DISTANCE VECTOR PROTOCOLS.

Asynchrony. One cannot assume that all values of round $i-1$ are computed before any of round i. The fix is to switch from pull-based to push-based updates: as soon as the estimate at v decreases, v notifies all of its neighbors. Assuming no negative cycles, this is guaranteed to converge eventually, because updates strictly decrease the sum of the shortest-path estimates. The RIP and RIP2 Internet routing protocols are very close to this algorithm (RFC 1058).

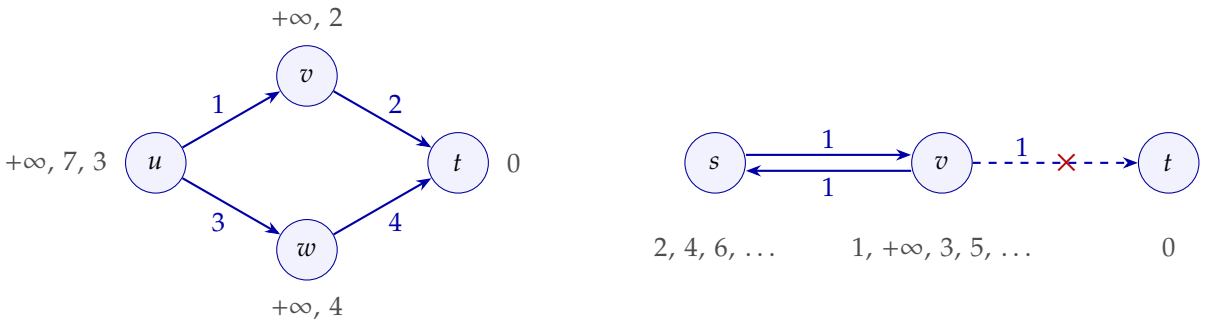


Figure 64: Distance estimates to t. Left: with asynchronous updates, u may hear from w (estimate 7) before v (estimate 3). Right: after (v, t) fails, s and v keep raising each other's estimates.

Failures. Convergence is guaranteed only for static networks. After a link fails, estimates can COUNT TO INFINITY, as in the right panel of Figure 64: when (v, t) fails, v learns from s a route of length 2 (which actually passes through v), sets its estimate to 3, then s updates to 4, and so on without bound. The fix is for each vertex to maintain the entire shortest path to t, not just the next hop. This is a PATH VECTOR PROTOCOL, used by the Border Gateway Protocol (BGP). It needs more space, but it is more robust to failures and permits more sophisticated route selection, such as caring about intermediate stops.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Bellman-Ford Algorithm.

Lecture 29: All-Pairs Shortest Paths

In the all-pairs shortest path problem (APSP), the input is a directed graph $G = (V, E)$ with n vertices, m edges, and an edge cost c_e for each $e \in E$; there is no distinguished source. The goal is either (A) to compute the length of a shortest u-v path for all pairs $u, v \in V$, or (B) to report correctly that G contains a negative cycle.

29.1 Reduction to single-source shortest paths

Running a single-source subroutine once from each vertex, n invocations in all, solves APSP.

- With nonnegative edge costs, n runs of Dijkstra's algorithm take $O(nm \log n)$ time: $O(n^2 \log n)$ if $m = \Theta(n)$ and $O(n^3 \log n)$ if $m = \Theta(n^2)$.
- With general edge costs, n runs of Bellman-Ford take $O(n^2 m)$ time: $O(n^3)$ if $m = \Theta(n)$ and $O(n^4)$ if $m = \Theta(n^2)$.

The Floyd-Warshall algorithm solves APSP in $O(n^3)$ time, even with negative edge lengths. It is therefore at least as good as n runs of Bellman-Ford and better in dense graphs. With nonnegative edge costs, it is competitive with n runs of Dijkstra's algorithm in dense graphs. An important special case is the transitive closure of a binary relation, that is, all-pairs reachability. Whether APSP can be solved significantly faster than $O(n^3)$ in dense graphs is an open question.

29.2 Optimal substructure

Ordering subproblems in a graph problem can be tricky. The key idea is to order the vertices arbitrarily as $V = \{1, 2, \ldots, n\}$ and to let $V^{(k)} = \{1, 2, \ldots, k\}$. Subproblems restrict which vertices may appear in the interior of a path.

Lemma 29.1 [Optimal substructure]. *Suppose G has no negative cycle. Fix a source $i \in V$, a destination $j \in V$, and $k \in \{1, 2, \ldots, n\}$. Let P be a shortest cycle-free i-j path with all internal vertices in $V^{(k)}$.*

1. *If k is not internal to P, then P is a shortest cycle-free i-j path with all internal vertices in $V^{(k-1)}$.*
2. *If k is internal to P, then P splits at k into P_1 and P_2, where P_1 is a shortest cycle-free i-k path and P_2 is a shortest cycle-free k-j path, each with all internal vertices in $V^{(k-1)}$.*

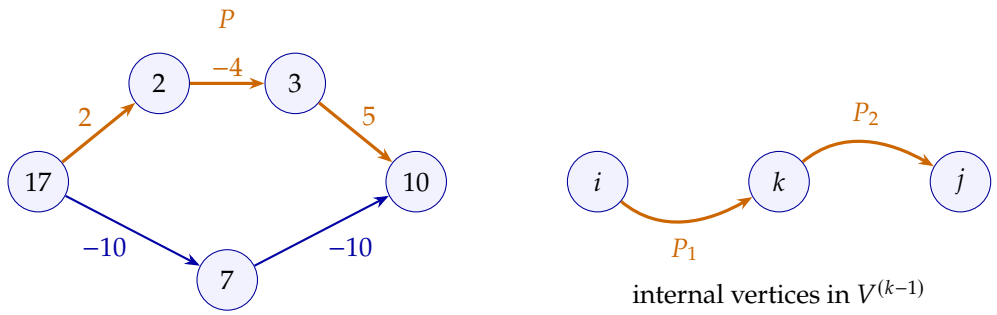


Figure 65: Left: with $i = 17$, $j = 10$, and $k = 5$, the path P through 2 and 3 is a shortest i-j path with internal vertices in $V^{(5)}$; the shorter path through 7 is not allowed. Right: in Case 2, P passes through k once and splits into P_1 and P_2.

Proof. Case 1: P is a candidate for the smaller subproblem, and every candidate for the smaller subproblem is also one for the larger. So P is optimal for both.
Case 2: since P is cycle-free, k appears on it exactly once. So P_1 and P_2 are cycle-free with all internal vertices in $V^{(k-1)}$, and the length of P is the sum of their lengths. Suppose a cycle-free i-k path Q_1 with internal vertices in $V^{(k-1)}$ is shorter than P_1. Then $Q_1 + P_2$ is an i-j walk with internal vertices in $V^{(k)}$ that is shorter than P. It may repeat a vertex, but deleting its cycles, none of which is negative, gives a cycle-free i-j path with internal vertices in $V^{(k)}$ that is still shorter than P, a contradiction. The argument for P_2 is symmetric. □

29.3 The Floyd-Warshall algorithm

Let $A[i, j, k]$ be the length of a shortest i-j path with all internal vertices in $\{1, 2, \ldots, k\}$, or $+\infty$ if no such path exists. For $k = 0$ no internal vertices are allowed, so

$$A[i,j,0] = \begin{cases} 0 & \text{if } i = j, \\ c_{ij} & \text{if } (i,j) \in E, \\ +\infty & \text{if } i \neq j \text{ and } (i,j) \notin E. \end{cases}$$

Algorithm 29.1: Floyd-Warshall

Input: Directed graph $G = (V, E)$ with $V = \{1, \ldots, n\}$ and edge costs c_e.
Output: Shortest-path distances $A[i, j, n]$ for all $i, j \in V$.

```
A ← a three-dimensional array indexed by i, j, k
Set A[i, j, 0] for all i, j ∈ V by the base cases
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            A[i, j, k] ← min{A[i, j, k − 1], A[i, k, k − 1] + A[k, j, k − 1]}
return A[i, j, n] for all i, j ∈ V
```

The two terms are Case 1 and Case 2 of Lemma 29.1, so if G has no negative cycle, correctness follows by induction on k. Each subproblem takes $O(1)$ time, so the total is $O(n^3)$.

Negative cycles. If G has a negative cycle, then $A[i, i, n] < 0$ for at least one $i \in V$ at the end of the algorithm. If G has no negative cycle, every $A[i, i, n]$ equals the length 0 of the empty i-i path, so checking the diagonal detects negative cycles.

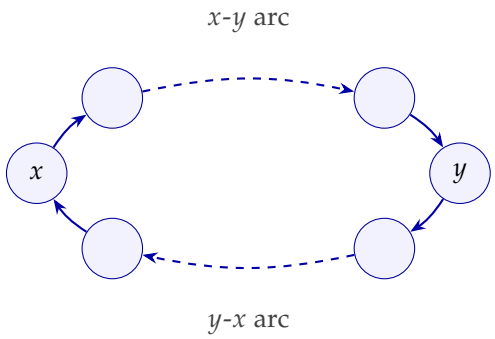


Figure 66: A negative cycle through x, where y has the largest label on the cycle. Both arcs have internal vertices with labels below y.

Proof of the negative-cycle check. By induction on k, $A[i, j, k]$ is at most the length of every cycle-free i-j path with internal vertices in $V^{(k)}$, even if G has negative cycles: if the path avoids k, use $A[i, j, k-1]$; otherwise split it at its single visit to k. Also, $A[i, j, k] \le A[i, j, k-1]$.
Let C be a negative cycle that repeats no vertex and has at least two vertices, let y be the vertex of C with the largest label, and let $x \neq y$ be another vertex of C, as in Figure 66. The cycle consists of an x-y arc and a y-x arc, both cycle-free with internal labels below y. Hence

$$A[x, x, n] \le A[x, x, y] \le A[x, y, y-1] + A[y, x, y-1] \le (\text{length of } C) < 0. \qquad \square$$

Reconstructing paths. Along with A, compute $B[i, j]$, the largest label of an internal vertex on a shortest i-j path. Whenever Case 2 of the recurrence is used to compute $A[i, j, k]$, reset $B[i, j] = k$. A shortest i-j path then consists of a shortest i-$B[i, j]$ path followed by a shortest $B[i, j]$-j path, each reconstructed recursively.

29.4 Reweighting

Johnson's algorithm reduces APSP to one invocation of Bellman-Ford, taking $O(mn)$ time, and n invocations of Dijkstra's algorithm, taking $O(nm \log n)$ time. The total, $O(mn) + O(mn \log n) = O(mn \log n)$, is as good as for nonnegative edge lengths.

Adding the same constant M to every edge length does not preserve shortest paths unless all s-t paths have the same number of edges. For example, if $s \to v \to t$ has edges of length 1 and 1 and the direct edge (s, t) has length 3, the two-edge path is shorter. After adding $M = 2$, the lengths

become 6 and 5, and the direct edge is shorter.

Instead, fix a real number p_v for each vertex v and define the REWEIGHTED LENGTH

$$c'_e = c_e + p_u - p_v \qquad \text{for each edge } e = (u, v).$$

For example, an edge of length 2 with $p_u = -4$ and $p_v = -3$ gets length $2 + (-4) - (-3) = 1$.

Proposition 29.2 [REWEIGHTING PRESERVES SHORTEST PATHS]. *If an s-t path P has length L under the lengths c_e, then its length under the lengths c'_e is $L + p_s - p_t$.*

Proof. The sum telescopes along the path:

$$\sum_{e \in P} c'_e = \sum_{e=(u,v) \in P} (c_e + p_u - p_v) = \left(\sum_{e \in P} c_e\right) + p_s - p_t. \qquad \square$$

Every s-t path shifts by the same amount $p_s - p_t$, so reweighting never changes which s-t paths are shortest. This is useful if G has some negative edge lengths but, after reweighting by suitable $\{p_v\}$, all edge lengths become nonnegative. Such weights always exist when G has no negative cycle, and Bellman-Ford computes them. Bellman-Ford is then needed once, and it enables Dijkstra's algorithm.

29.5 Johnson's algorithm

Add a new vertex s with an edge of length 0 to every vertex of G, and let p_v be the length of a shortest s-v path in this larger graph. Since s has no incoming edges, adding it creates no new u-v paths for u, v in G.

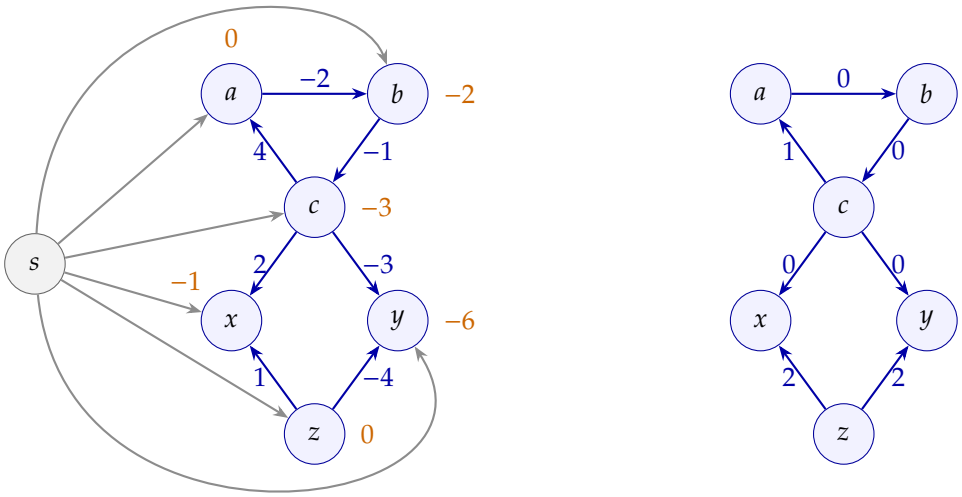


Figure 67: Left: G', formed by adding s with an edge of length 0 (gray) to every vertex; the numbers beside the vertices are the potentials p_v. Right: the reweighted lengths $c'_e = c_e + p_u - p_v$ on G, all nonnegative.

In Figure 67, the potentials are $p_a = 0$, $p_b = -2$, $p_c = -3$, $p_x = -1$, $p_y = -6$, and $p_z = 0$. After reweighting, every edge length is 0, 1, or 2, so all reweighted shortest paths can be computed with n runs of Dijkstra's algorithm, with no further need for Bellman-Ford.

Algorithm 29.2: Johnson's algorithm

Input: Directed graph $G = (V, E)$ with general edge lengths c_e.
Output: Shortest-path distances $d(u, v)$ for all $u, v \in V$, or a negative cycle.

Form G' by adding a new vertex s and an edge (s, v) of length 0 for each $v \in V$
Run Bellman-Ford on G' from s
if Bellman-Ford detects a negative cycle in G' **then**
    Halt and report the negative cycle, which must lie in G
$p_v \leftarrow$ length of a shortest s-v path in G', for each $v \in V$
$c'_e \leftarrow c_e + p_u - p_v$, for each $e = (u, v) \in E$
for each $u \in V$ **do**
    Run Dijkstra's algorithm on G from u with lengths c'_e, giving $d'(u, v)$ for all $v \in V$
return $d(u, v) \leftarrow d'(u, v) - p_u + p_v$ for each pair $u, v \in V$

Running time. Forming G' takes $O(n)$, Bellman-Ford $O(mn)$, forming c' $O(m)$, the n runs of Dijkstra's algorithm $O(nm \log n)$, and the final step $O(1)$ per pair, $O(n^2)$ in all. The total is $O(mn \log n)$, much better than Floyd-Warshall for sparse graphs.

Correctness. Given that every $c'_e \geq 0$, Dijkstra's algorithm correctly computes the reweighted distances. By Proposition 29.2, reweighting does not change the shortest u-v path; it only adds $p_u - p_v$ to its length. So $d(u, v) = d'(u, v) - p_u + p_v$.

Proposition 29.3 [Nonnegative reweighted lengths]. *For every edge $e = (u, v)$ of G, the reweighted length $c'_e = c_e + p_u - p_v$ is nonnegative.*

Proof. By construction, p_u and p_v are the lengths of shortest s-u and s-v paths in G'; they exist because of the edges from s. Let P be a shortest s-u path in G', of length p_u. Then $P + (u, v)$ is an s-v path of length $p_u + c_{uv}$. A shortest s-v path is no longer, so $p_v \leq p_u + c_{uv}$, that is, $c'_{uv} = c_{uv} + p_u - p_v \geq 0$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), All-Pairs Shortest Paths.

Lecture 30: NP-Complete Problems

Many important computational problems seem impossible to solve efficiently. NP-completeness gives a precise way to express this intractability, and its conclusion shapes which algorithmic goals are reasonable.

30.1 Polynomial-time solvability and the class P

Measure the input size n by the input length: the number of keystrokes needed to describe the input.

Definition 30.1 [Polynomial-time solvable]. A problem is polynomial-time solvable if some algorithm correctly solves it in $O(n^k)$ time, for some constant k. The class P is the set of polynomial-time solvable problems.

Any constant exponent qualifies, even $k = 10{,}000$. The definition uses deterministic algorithms; allowing randomization does not change the picture to first order. P is a rough litmus test for computational tractability.

Almost every problem solved so far in the course is in P. Two exceptions are cycle-free shortest paths in graphs with negative cycles, and knapsack. The knapsack dynamic program runs in $\Theta(nW)$ time, but the capacity W is written with about $\log W$ digits, so the input length is proportional to $\log W$ rather than W. Thus nW can be exponential in the input length. Both problems are NP-complete.

Definition 30.2 [Traveling salesman problem]. The input is a complete undirected graph with nonnegative edge costs. The output is a minimum-cost tour: a cycle that visits every vertex exactly once. This is the traveling salesman problem (TSP).

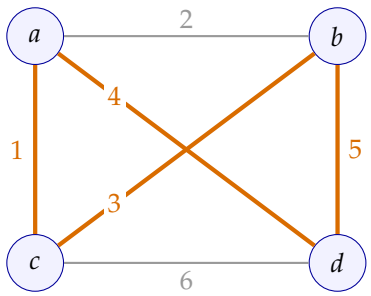


Figure 68: A four-vertex TSP instance. The highlighted tour $a \to c \to b \to d \to a$ costs $1 + 3 + 5 + 4 = 13$, the optimum. The other two tours cost $2 + 5 + 6 + 1 = 14$ and $2 + 3 + 6 + 4 = 15$.

Note. Edmonds' conjecture (1965): there is no polynomial-time algorithm for TSP. Since TSP is NP-complete (see below), this is equivalent to P ≠ NP.

30.2 Reductions and completeness

Instead of proving outright that TSP is hard, amass evidence through relative difficulty: show that TSP is at least as hard as many other problems.

Definition 30.3 [Reduction]. A problem Π_1 reduces to a problem Π_2 if, given a polynomial-time subroutine for Π_2, we can use it to solve Π_1 in polynomial time.

For example, computing the median reduces to sorting (sort, then read the middle entry); detecting a cycle reduces to depth-first search; and all-pairs shortest paths reduces to single-source shortest paths (one call per source vertex).

Proposition 30.4 [Reductions transfer tractability]. *Suppose* Π_1 *reduces to* Π_2. *If* $\Pi_2 \in \mathrm{P}$, *then* $\Pi_1 \in \mathrm{P}$. *Equivalently, if* $\Pi_1 \notin \mathrm{P}$, *then* $\Pi_2 \notin \mathrm{P}$.

Proof. Replace the subroutine in the reduction by a polynomial-time algorithm for Π_2. The reduction makes polynomially many calls, each on an input of polynomial length, and a polynomial of a polynomial is a polynomial. The second statement is the contrapositive. □

So if Π_1 reduces to Π_2, then Π_2 is at least as hard as Π_1.

Definition 30.5 [Completeness]. Let C be a set of problems. A problem Π is C-complete if $\Pi \in C$ and every problem in C reduces to Π.

A C-complete problem is a hardest problem in C. The plan is to show that TSP is C-complete for a very large class C. The class of all problems is too large. In the halting problem, the input is a program and an input for it, and the question is whether the program eventually halts. By Turing's theorem (1936), no algorithm, however slow, solves it. TSP, in contrast, is solvable in finite time by brute-force search, so the halting problem cannot reduce to TSP. The refined goal is to show that TSP is as hard as every problem solvable by brute-force search.

30.3 The class NP and NP-completeness

Definition 30.6 [NP]. A problem is in NP if

1. its solutions always have length polynomial in the input size, and
2. purported solutions can be verified in polynomial time.

Examples are deciding whether a TSP instance has a tour of length at most 1000, and constraint satisfaction problems such as 3SAT. A NP-complete problem is a C-complete problem for C = NP.

Every problem in NP can be solved by brute-force search in exponential time: there are exponentially many candidate solutions of polynomial length, and each can be checked in polynomial time. The vast majority of natural computational problems are in NP; roughly, being in NP means that a solution can be recognized efficiently.

Proposition 30.7 [One NP-complete problem in P suffices]. *If some NP-complete problem* Π *is polynomial-time solvable, then every problem in NP is polynomial-time solvable; that is,* P = NP.

Proof. Every problem in NP reduces to Π, so Proposition 30.4 applies to each of them. □

Hence NP-completeness is strong evidence of intractability. An NP-complete problem simultaneously encodes *all* problems whose solutions can be efficiently recognized; it is a universal problem. Two facts make the theory useful. NP-complete problems exist (Cook 1971, Levin 1973). Moreover, starting with Karp (1972), thousands of natural and important problems, including TSP, have been shown to be NP-complete.

Proposition 30.8 [Proving NP-completeness]. *Let* Π' *be a known NP-complete problem (for instance, from Garey and Johnson,* Computers and Intractability*). If* $\Pi \in$ NP *and* Π' *reduces to* Π*, then* Π *is NP-complete.*

Proof. Reductions compose: a reduction from a problem Π'' to Π' that calls a subroutine for Π' can implement each call through the reduction from Π' to Π, and the result still runs in polynomial time. Every problem in NP reduces to Π', and hence to Π. Together with $\Pi \in$ NP, this is the definition of NP-completeness. □

The reduction shows that Π is at least as hard as Π'; membership in NP is checked separately.

30.4 The P versus NP question

The central question is whether P = NP: is every problem whose solutions can be verified in polynomial time also solvable in polynomial time? It is widely conjectured that P ≠ NP, but this has not been proved; it is one of the Clay Institute's million-dollar problems. Reasons to believe the conjecture are psychological (if P = NP, someone would likely have proved it by now) and philosophical (if P = NP, finding a proof would always be as easy as verifying one). There is no convincing mathematical reason yet.

The name NP stands for *nondeterministic polynomial*, not "not polynomial." The definition by efficient verification of purported solutions is a modern, mathematically equivalent formulation.

30.5 Algorithmic approaches to NP-complete problems

NP-completeness is not a death sentence for a problem, but it calls for appropriate expectations and strategy. Three strategies are useful.

1. **Focus on computationally tractable special cases.** Maximum-weight independent set is NP-complete in general graphs but solvable in path graphs, and also in trees and graphs of bounded tree width. Knapsack is tractable when the capacity is polynomial in size, for example $W = O(n)$. 2SAT is in P, while 3SAT is NP-complete. Vertex cover is tractable when the optimal cover is small.
2. **Heuristics:** fast algorithms that are not always correct. Lecture 32 gives greedy and dynamic-programming heuristics for knapsack.
3. **Exponential time, but faster than brute-force search.** Knapsack takes $O(nW)$ time rather than the 2^n of checking every subset. Lecture 31 solves TSP in roughly 2^n rather than $n!$ time, and vertex cover in roughly $2^{\text{OPT}}n$ rather than n^{OPT} time.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), NP-Complete Problems.

Lecture 31: Faster Exact Algorithms for NP-Complete Problems

An NP-complete problem can still be solved exactly in exponential time that is far smaller than brute-force search. This lecture gives two examples: a search algorithm for small vertex covers and a dynamic program for TSP.

31.1 The vertex cover problem

Definition 31.1 [Vertex cover]. Let $G = (V, E)$ be an undirected graph. A vertex cover is a subset $S \subseteq V$ that contains at least one endpoint of each edge of G. The vertex cover problem asks for a vertex cover of minimum cardinality.

Example: In a star graph with n vertices, the center alone is a vertex cover, so the minimum size is 1. In a clique with n vertices, any two vertices outside a cover would leave the edge between them uncovered. Hence at most one vertex is omitted, and the minimum size is $n - 1$.

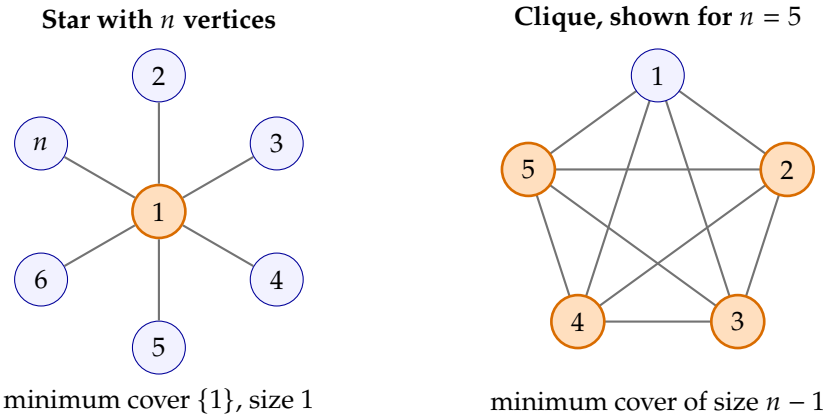


Figure 69: Minimum vertex covers (highlighted) of a star and a clique.

In general, vertex cover is NP-complete. Following the three strategies of Lecture 30:

1. Tractable special cases include trees (by dynamic programming), bipartite graphs (by the maximum flow problem), and graphs whose optimal cover is small, roughly $\log n$ or less.
2. Heuristics, for example suitable greedy algorithms.
3. Exponential time that improves on brute-force search, developed next.

31.2 Smarter search for small vertex covers

Given a positive integer k, thought of as small, decide whether G has a vertex cover of size at most k. Trying all subsets of size k takes about $\binom{n}{k} = \Theta(n^k)$ time. A better search branches on the endpoints of a single edge.

For a vertex u, let G_u be G with u and its incident edges deleted. Write $E = E_u \cup F_u$, where E_u is the edge set of G_u and F_u consists of the edges incident to u.

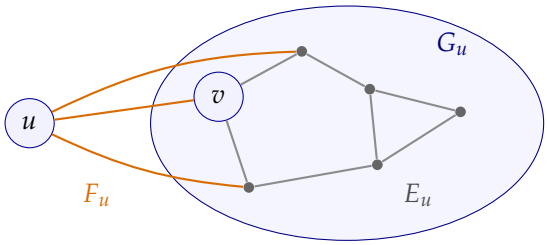


Figure 70: Deleting u leaves the graph G_u with edge set E_u. The deleted edges F_u, including (u, v), are covered by u.

Lemma 31.2 [Substructure lemma]. *Let $(u, v) \in E$ and $k \geq 1$. Then G has a vertex cover of size at most k if and only if G_u or G_v (or both) has a vertex cover of size at most $k - 1$.*

Proof. ($\Leftarrow$) Suppose G_u, say, has a vertex cover S with $|S| \leq k - 1$. Then S contains an endpoint of each edge of E_u, and u is an endpoint of each edge of F_u. So $S \cup \{u\}$ is a vertex cover of G of size at most k.
($\Rightarrow$) Let S be a vertex cover of G with $|S| \leq k$. Since (u, v) is an edge, S contains u or v, say u. No edge of E_u is incident to u, so each edge of E_u has an endpoint in $S - \{u\}$. Thus $S - \{u\}$ is a vertex cover of G_u of size at most $k - 1$. □

Algorithm 31.1: Search for a vertex cover of size at most k

```
Input: Undirected graph G = (V, E) and integer k ≥ 0.
Output: A vertex cover of size at most k, or FAIL if none exists.
if E = ∅ then
    return ∅
if k = 0 then
    return FAIL
Pick an arbitrary edge (u, v) ∈ E
S ← Search(G_u, k − 1)
if S ≠ FAIL then
    return S ∪ {u}
S ← Search(G_v, k − 1)
if S ≠ FAIL then
    return S ∪ {v}
return FAIL
```

Theorem 31.3 [Vertex cover search]. *The search algorithm is correct and runs in $O(2^k m)$ time, where $m = |E|$.*

Proof. Correctness follows by induction on k. If $E = \emptyset$, the empty set is a cover. If $E \neq \emptyset$ and $k = 0$, no cover of size 0 exists. Otherwise, by the induction hypothesis the two recursive calls correctly decide whether G_u or G_v has a cover of size at most $k - 1$, and a returned set extends to a cover of G by the ($\Leftarrow$) direction of Lemma 31.2. If both calls fail, the ($\Rightarrow$) direction shows that G has no cover of size at most k.
For the running time, each call makes at most two recursive calls, each with parameter decreased by one, and calls with $k = 0$ make none. The recursion tree therefore has branching factor at most 2 and depth at most k, so it has at most $2^{k+1} - 1 = O(2^k)$ nodes. Each call does $O(m)$ work outside its recursive subcalls, for choosing an edge and forming G_u and G_v. The total is $O(2^k m)$. □

This bound is polynomial whenever $k = O(\log n)$, remains feasible even when $k \approx 20$, and is far better than $\Theta(n^k)$.

31.3 The traveling salesman problem

Recall TSP from Lecture 30: given a complete undirected graph with vertices $\{1, 2, \ldots, n\}$ and nonnegative edge costs c_{ij}, find a minimum-cost tour. Brute-force search over tours takes about $n!$ time, which is tractable only for $n \approx 12$ or 13. Dynamic programming achieves $O(n^2 2^n)$ time, which is tractable for n close to 30.

A natural first attempt copies the format of the Bellman–Ford algorithm. Each candidate below indexes subproblems by an edge budget $i \in \{0, 1, \ldots, n\}$ and a destination $j \in \{1, 2, \ldots, n\}$, with L_{ij} a shortest-path length from vertex 1 to j.

1. **At most i edges.** These subproblems are polynomial in number and easy to compute, but solving all of them does not solve TSP: nothing forces a path to visit every vertex.
2. **Exactly i edges.** Solving these still does not solve TSP: a path with exactly n edges may repeat vertices and skip others.
3. **Exactly i edges and no repeated vertices.** There are only polynomially many subproblems, and they would solve TSP: the optimal tour cost is $\min_j \{L_{n-1,j} + c_{j1}\}$. The obstacle is that bigger subproblems cannot be computed efficiently from smaller ones.

For the third attempt, one would hope to use the recurrence

$$L_{ij} = \min_{k \neq 1, j} \bigl\{ L_{i-1,k} + c_{kj} \bigr\},$$

combining a shortest path from 1 to k with $i - 1$ edges and no repeated vertices with the final

hop (k, j). But j may already appear on that $1 \to k$ path, and appending (k, j) would then visit j twice, which is not allowed. To enforce that each vertex is visited exactly once, a subproblem must remember the *identities* of the vertices it has visited, though not the order in which they were visited.

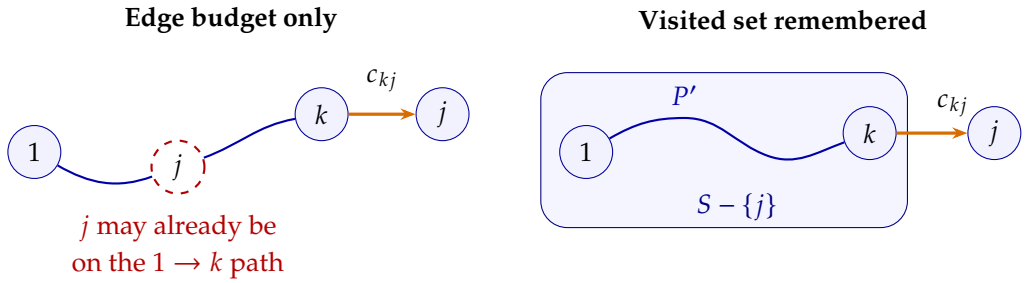


Figure 71: Left: with only an edge budget, the best $1 \to k$ path may already pass through j, so appending (k, j) revisits j. Right: a subproblem that fixes the visited set $S - \{j\}$ excludes j from P'.

31.4 A dynamic programming algorithm for TSP

Definition 31.4 [TSP SUBPROBLEMS]**.** For every destination $j \in \{1, 2, \ldots, n\}$ and every subset $S \subseteq \{1, 2, \ldots, n\}$ that contains 1 and j, let $L_{S,j}$ be the minimum length of a path from 1 to j that visits precisely the vertices of S, each exactly once.

The size of a subproblem is $|S|$.

Lemma 31.5 [OPTIMAL SUBSTRUCTURE]**.** *Let $|S| \geq 2$ and let P be a shortest path from 1 to j that visits each vertex of S exactly once. If the last hop of P is (k, j), then the rest of the path, P', is a shortest path from 1 to k that visits each vertex of $S - \{j\}$ exactly once.*

Proof. Use cut and paste. P' visits exactly the vertices of $S - \{j\}$ and ends at k. If some path Q from 1 to k visiting each vertex of $S - \{j\}$ exactly once were shorter than P', then Q followed by (k, j) would visit each vertex of S exactly once, because $j \notin S - \{j\}$, and would be shorter than P. This contradicts the choice of P. □

Conversely, for every $k \in S - \{j\}$, an optimal path for $(S - \{j\}, k)$ followed by (k, j) is a valid path for (S, j). Hence

$$L_{S,j} = \min_{k \in S,\, k \neq j} \{L_{S-\{j\},k} + c_{kj}\}.$$

A path from 1 to itself cannot visit additional vertices without returning to 1 twice, so $L_{S,1} = 0$ if $S = \{1\}$ and $+\infty$ otherwise.

Algorithm 31.2: Dynamic programming for TSP

Input: Complete undirected graph on $\{1, \dots, n\}$, $n \geq 3$, with costs $c_{ij} \geq 0$.
Output: The minimum cost of a tour.

```
for each S ⊆ {1,...,n} with 1 ∈ S do
    A[S,1] ← 0 if S = {1}, and A[S,1] ← +∞ otherwise
for m ← 2 to n do
    for each S ⊆ {1,...,n} with |S| = m and 1 ∈ S do
        for each j ∈ S with j ≠ 1 do
            A[S,j] ← min_{k∈S, k≠j} {A[S − {j}, k] + c_kj}
return min_{j=2,...,n} {A[{1,...,n}, j] + c_j1}
```

Subsets are processed in order of size, so each $A[S - \{j\}, k]$ is available when needed. In the final line, $A[\{1, \dots, n\}, j]$ is the minimum cost of a path from 1 to j visiting every vertex once, and c_{j1} is the cost of the final hop that closes the tour. Every tour is such a path followed by its return edge, so the minimum is the optimal tour cost.

Theorem 31.6 [TSP RUNNING TIME**].** *The algorithm runs in $O(n^2 2^n)$ time.*

Proof. There are at most n choices of j and 2^{n-1} choices of S, so $O(n2^n)$ subproblems. Each takes $O(n)$ time, a minimum over at most n choices of k. The final line takes $O(n)$ time. The total is $O(n2^n) \cdot O(n) = O(n^2 2^n)$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Faster Exact Algorithms for NP-Complete Problems.

Lecture 32: Approximation Algorithms for NP-Complete Problems

A heuristic for an NP-complete problem should ideally come with a performance guarantee: a proof that its output is almost correct on all, or at least many, instances. For knapsack, this lecture gives a greedy heuristic that is always within a factor of two of optimal, and a dynamic-programming heuristic whose accuracy can be made arbitrarily good.

32.1 Knapsack and a greedy heuristic

Definition 32.1 [Knapsack]. The input is n items, where item i has a positive value v_i and a size w_i, together with a knapsack capacity W. The output is a subset $S \subseteq \{1, 2, \ldots, n\}$ that

$$\text{maximizes } \sum_{i \in S} v_i \quad \text{subject to} \quad \sum_{i \in S} w_i \leq W.$$

Assume every item fits on its own, $w_i \leq W$; any other item can be discarded in advance. Ideal items have big value and small size, which suggests ordering items by their value per unit size, or bang-per-buck, v_i / w_i.

Algorithm 32.1: Three-step greedy knapsack heuristic

Input: Values $v_i > 0$, sizes $0 < w_i \leq W$, and capacity W.
Output: A feasible subset of items.

1 Sort and reindex the items so that $v_1/w_1 \geq v_2/w_2 \geq \cdots \geq v_n/w_n$
2 $S \leftarrow \emptyset$
3 **for** $i \leftarrow 1$ **to** n **do**
4 **if** $\sum_{\ell \in S} w_\ell + w_i > W$ **then break**
5 $S \leftarrow S \cup \{i\}$
6 Let i^* be an item of maximum value
7 **return** the more valuable of S and $\{i^*\}$

Steps 1 and 2 sort, then pack items in order until one does not fit. With $W = 5$ and $(v_i, w_i) = (2, 1), (4, 3), (3, 3)$, they return $\{1, 2\}$, which is optimal. Alone, however, they can be arbitrarily bad: with $W = 1000$ and items $(v_1, w_1) = (2, 1)$, $(v_2, w_2) = (1000, 1000)$, they return item 1 of value 2, whereas the optimum takes item 2 with value 1000. Step 3 fixes this by also considering the single most valuable item.

Theorem 32.2 [Greedy approximation]. *The three-step greedy heuristic runs in $O(n \log n)$ time, and its solution always has value at least 50% of the optimal value. That is, it is a $\frac{1}{2}$-approximation algorithm.*

Sorting dominates the running time; the accuracy guarantee is proved next.

32.2 Analysis of the greedy heuristic

If Step 2 packs every item, its solution is optimal. Otherwise, suppose Step 2 packs items $1, \dots, k$ and item $k+1$ does not fit. As a thought experiment, fill the knapsack completely by also packing the fraction

$$\alpha = \frac{W - \sum_{i \le k} w_i}{w_{k+1}} \in [0, 1)$$

of item $k+1$, with value pro-rated to αv_{k+1}. Call this the GREEDY FRACTIONAL SOLUTION, and let $F = \sum_{i \le k} v_i + \alpha v_{k+1}$ be its value.

Example: With $W = 3$, $v_1 = 3$, $v_2 = 2$, and $w_1 = w_2 = 2$, the greedy fractional solution takes all of item 1 and half of item 2, so $F = 3 + 1 = 4$. Only one item fits in a genuine solution, so the optimal value is $3 < F$.

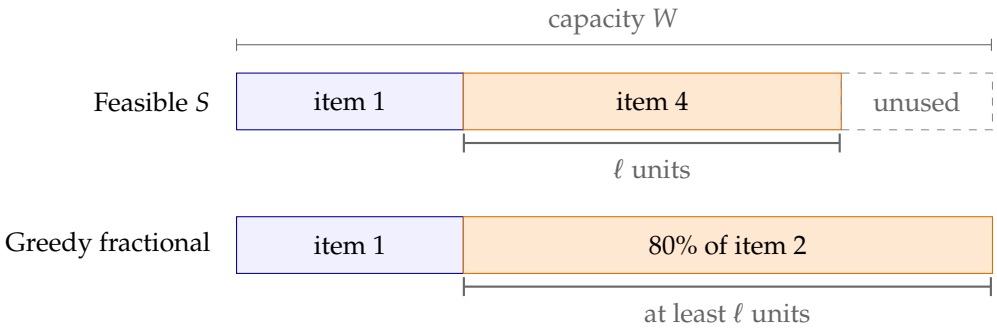


Figure 72: Schematic exchange in the proof of Lemma 32.3. The ℓ units that S spends on item 4 are matched by at least ℓ units that the greedy fractional solution spends on item 2, which has a larger bang-per-buck. Widths are illustrative.

Lemma 32.3 [FRACTIONAL BOUND]. *The greedy fractional solution is at least as good as every feasible non-fractional solution. In particular, $F \ge \mathrm{OPT}$, and the inequality can be strict, as in the example above.*

Proof. The idea is that the greedy fractional solution uses all of the space, and uses it at least as profitably. Let S be any feasible solution. Suppose S fills ℓ units of the knapsack with items not packed by the greedy fractional solution. Since the greedy fractional solution fills the knapsack completely, it fills at least ℓ units with items not packed by S. By the greedy ordering, every item of the second kind has bang-per-buck at least that of every item of the first kind, so the exchange does not lose value.

To make this precise, let x_i be the fraction of item i in the greedy fractional solution, let $y_i = 1$ if $i \in S$ and 0 otherwise, and let $r = v_{k+1}/w_{k+1}$. If $x_i > y_i$, then $i \le k+1$ and $v_i/w_i \ge r$; if $x_i < y_i$, then $i \ge k+1$ and $v_i/w_i \le r$. In both cases $(x_i - y_i)v_i \ge r(x_i - y_i)w_i$. Summing,

$$F - \sum_{i \in S} v_i = \sum_{i=1}^{n} (x_i - y_i) v_i \ge r\Big(W - \sum_{i \in S} w_i\Big) \ge 0. \qquad \square$$

Proof of Theorem 32.2. If Step 2 packs every item, the greedy solution is optimal. Otherwise, by Step 3, the value G of the three-step greedy solution is at least the total value of the first k items, and also at least the value v_{k+1} of item $k+1$. Adding these inequalities and using Lemma 32.3,

$$2G \geq \sum_{i=1}^{k+1} v_i \geq F \geq \text{OPT}.$$

□

Example: The analysis is tight. Let $W = 1000$, $(v_1, w_1) = (502, 501)$, and $(v_2, w_2) = (v_3, w_3) = (500, 500)$. Item 1 has the largest bang-per-buck, after which item 2 no longer fits, so the three-step greedy solution has value 502. The optimal solution $\{2, 3\}$ has value 1000.

The guarantee improves when every item is small relative to the capacity.

Proposition 32.4 [Small items]. *Let $0 < \delta < 1$. If $w_i \leq \delta W$ for every item i, then the solution from Steps 1 and 2 alone has value at least $(1-\delta) \cdot \text{OPT}$. For example, if every item has size at most 10% of W, the greedy value is at least 90% of optimal.*

Proof. If Step 2 packs every item, the solution is optimal. Otherwise item $k+1$ did not fit, so the first k items have total size $W_k > W - w_{k+1} \geq (1-\delta)W$; the knapsack is at least $(1-\delta)$ full. Let V_k be their total value. Each of them has bang-per-buck at least $r = v_{k+1}/w_{k+1}$, so $V_k \geq rW_k$. The fractional part of the greedy fractional solution occupies $W - W_k$ units at rate r. Hence

$$F = V_k + r(W - W_k) \leq V_k + \frac{V_k}{W_k}(W - W_k) = V_k \frac{W}{W_k} \leq \frac{V_k}{1-\delta}.$$

Therefore $V_k \geq (1-\delta)F \geq (1-\delta) \cdot \text{OPT}$ by Lemma 32.3. □

32.3 A dynamic programming heuristic

The goal is now an arbitrarily good approximation: for a user-specified parameter $\epsilon > 0$, such as $\epsilon = 0.01$, guarantee a $(1-\epsilon)$-approximation, a feasible solution whose value is at least $(1-\epsilon) \cdot \text{OPT}$. The catch is that the running time increases as ϵ decreases, so the algorithm exports a trade-off between running time and accuracy. This is the best-case scenario for an NP-complete problem.

The approach is to solve exactly a slightly incorrect, but easier, knapsack instance. If the sizes w_i and capacity W are integers, the earlier dynamic program solves knapsack in $O(nW)$ time. Alternatively, if the values v_i are integers, a second dynamic program (below) solves it in $O(n^2 v_{\max})$ time, where $v_{\max} = \max_i v_i$. Thus if all values are small integers, polynomial in n, knapsack is already solvable in polynomial time. The plan is to throw out the lower-order bits of the values.

Algorithm 32.2: Dynamic programming knapsack heuristic

Input: Values $v_i > 0$, sizes $w_i \le W$, capacity W, and accuracy $\epsilon > 0$.
Output: A feasible subset of items.

$m \leftarrow \epsilon v_{\max} / n$ *// justified in the analysis below*
for each item i **do**
  $\hat{v}_i \leftarrow \lfloor v_i / m \rfloor$
return an optimal solution of the knapsack instance with values $\hat{v}_1, \dots, \hat{v}_n$, sizes $w_1, \dots, w_n$, and capacity W

Step 1 rounds each v_i down to the nearest multiple of m and divides by m, producing integers $\hat{v}_i$. A larger m throws out more information and so loses accuracy; the choice of m, which depends on ϵ, is justified below. Step 2 uses the second dynamic program, in time $O(n^2 \max_i \hat{v}_i)$. Since the sizes and capacity are unchanged, the output is a feasible solution of the original instance.

32.4 Dynamic programming for knapsack, revisited

Assume the values v_i are nonnegative integers. Any solution has value at most $n v_{\max}$, so only targets in that range matter.

Definition 32.5 [Value-indexed subproblems]. For $i = 0, 1, \dots, n$ and $x = 0, 1, \dots, n v_{\max}$, let $S_{i,x}$ be the minimum total size of a subset of the first i items with total value at least x, or $+\infty$ if no such subset exists.

For $i \ge 1$, an optimal subset for (i, x) either omits item i, or contains it and uses the other chosen items to reach the remaining value $x - v_i$. Thus

$$S_{i,x} = \min\bigl\{ S_{i-1,x},\ w_i + S_{i-1,\max\{x - v_i, 0\}} \bigr\},$$

where the second index is 0 when $v_i > x$, since item i then reaches the target alone.

Algorithm 32.3: Knapsack by minimum size for each value

Input: Integer values $v_i \ge 0$, sizes w_i, and capacity W.
Output: The optimal knapsack value.

$A[0,0] \leftarrow 0$; $A[0,x] \leftarrow +\infty$ for $x = 1, \dots, n v_{\max}$
for $i \leftarrow 1$ **to** n **do**
  for $x \leftarrow 0$ **to** $n v_{\max}$ **do**
    $A[i,x] \leftarrow \min\{ A[i-1,x],\ w_i + A[i-1, \max\{x - v_i, 0\}] \}$
return the largest x such that $A[n,x] \le W$

The loops perform $O(n^2 v_{\max})$ iterations with $O(1)$ work each, and the final scan takes $O(n v_{\max})$ time, so the running time is $O(n^2 v_{\max})$. An optimal subset is recovered by tracing back through the table.

32.5 Analysis of the dynamic programming heuristic

The analysis first finds how large m can be while still guaranteeing a $(1-\epsilon)$-approximation, then substitutes this value into the running time. Since v_i is rounded down to the nearest multiple of m,

$$m\hat{v}_i \in [v_i - m,\, v_i] \qquad \text{for each item } i.$$

Theorem 32.6 [Dynamic programming heuristic]. *With $m = \epsilon v_{\max}/n$, the heuristic returns a solution of value at least $(1-\epsilon)\cdot \mathrm{OPT}$ in $O(n^3/\epsilon)$ time.*

Proof. Let S be the heuristic's solution and S^* an optimal solution of the original instance. Use three facts: (1) $v_i \geq m\hat{v}_i$; (2) $m\hat{v}_i \geq v_i - m$; and (3) $\sum_{i\in S}\hat{v}_i \geq \sum_{i\in S^*}\hat{v}_i$, since S is optimal for the rounded values. Then

$$\sum_{i\in S} v_i \overset{(1)}{\geq} m\sum_{i\in S}\hat{v}_i \overset{(3)}{\geq} m\sum_{i\in S^*}\hat{v}_i \overset{(2)}{\geq} \sum_{i\in S^*}(v_i - m) \geq \sum_{i\in S^*} v_i - mn,$$

because S^* contains at most n items. To guarantee $\sum_{i\in S} v_i \geq (1-\epsilon)\sum_{i\in S^*} v_i$, it suffices that $mn \leq \epsilon \sum_{i\in S^*} v_i$. The optimal value is unknown to the algorithm, but it is at least $v_{\max}$, because the most valuable item fits on its own. So $mn = \epsilon v_{\max}$, that is, $m = \epsilon v_{\max}/n$, suffices.
For the running time, every item satisfies

$$\hat{v}_i \leq \frac{v_i}{m} \leq \frac{v_{\max}}{m} = v_{\max}\cdot\frac{n}{\epsilon v_{\max}} = \frac{n}{\epsilon}.$$

The second dynamic program therefore takes $O(n^2 \max_i \hat{v}_i) = O(n^3/\epsilon)$ time, and rounding takes $O(n)$. □

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Approximation Algorithms for NP-Complete Problems.

Lecture 33: Local Search Algorithms

Local search starts from some candidate solution and repeatedly moves to a better nearby solution until none exists. It applies to many NP-complete problems; the price is that the final solution is only locally optimal, so its quality and the number of moves both need analysis.

33.1 The maximum cut problem

Definition 33.1 [Maximum cut]. The input is an undirected graph $G = (V, E)$. A cut (A, B) is a partition of V into two nonempty sets, and an edge crosses the cut if it has one endpoint in each set. The goal is a cut maximizing the number of crossing edges.

The maximum cut problem is NP-complete. Bipartite graphs, which by definition have a cut that every edge crosses, are a tractable special case: breadth-first search finds such a cut in linear time.

Example: In the graph of Figure 73, every one of the 8 edges has one endpoint in $A = \{w, x\}$ and the other in $B = \{u, v, y, z\}$. The graph is bipartite, so the maximum cut has value 8.

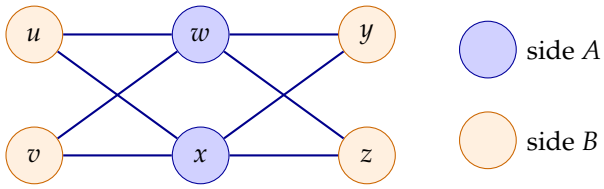


Figure 73: A bipartite graph on six vertices. The cut $A = \{w, x\}$, $B = \{u, v, y, z\}$ is crossed by all 8 edges, so it is a maximum cut.

For a cut (A, B) and a vertex v, let

$$c_v(A, B) = \#\text{ edges incident on } v \text{ that cross } (A, B),$$
$$d_v(A, B) = \#\text{ edges incident on } v \text{ that do not cross } (A, B).$$

Algorithm 33.1: Local search for maximum cut

Input: Undirected graph $G = (V, E)$.
Output: A locally optimal cut (A, B).

```
(A, B) ← an arbitrary cut of G
while some vertex v has d_v(A, B) > c_v(A, B) do
    move v to the other side of the cut
return (A, B)
```

Moving v to the other side changes the status of exactly the edges incident on v: its d_v non-crossing

edges start crossing and its c_v crossing edges stop. All other edges are unaffected. Each iteration therefore increases the number of crossing edges by $d_v(A,B) - c_v(A,B) \geq 1$. A simple graph has at most $\binom{n}{2}$ edges, where $n = |V|$, so there are at most $\binom{n}{2}$ iterations, and the algorithm runs in polynomial time. A cut on which the loop stops, so that $d_v(A,B) \leq c_v(A,B)$ for every vertex v, is LOCALLY OPTIMAL.

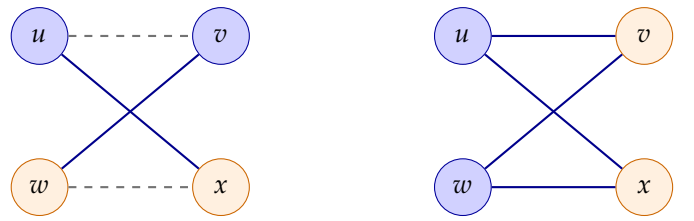
Theorem 33.2 [LOCAL SEARCH IS A $\frac{1}{2}$-APPROXIMATION]. *Every locally optimal cut is crossed by at least $\frac{1}{2}|E|$ edges. In particular, the local search algorithm outputs a cut with at least 50% of the maximum possible number of crossing edges.*

Proof. Let (A,B) be locally optimal, and sum $d_v(A,B) \leq c_v(A,B)$ over all $v \in V$. The left sum counts each non-crossing edge twice, once from each endpoint, and the right sum counts each crossing edge twice. Hence

$$2 \cdot [\#\text{ non-crossing edges}] \leq 2 \cdot [\#\text{ crossing edges}].$$

Adding $2 \cdot [\#\text{ crossing edges}]$ to both sides gives $2|E| \leq 4 \cdot [\#\text{ crossing edges}]$, so at least $\frac{1}{2}|E|$ edges cross. No cut has more than $|E|$ crossing edges. □

The factor $\frac{1}{2}$ is tight. In the four-cycle of Figure 74, the cut $A = \{u,v\}$, $B = \{w,x\}$ is crossed by 2 edges and every vertex has $c_v = d_v = 1$, so no single move helps. The cut $A^* = \{u,w\}$, $B^* = \{v,x\}$ is crossed by all 4 edges.



$A = \{u,v\}$, $B = \{w,x\}$
2 crossing edges

$A^* = \{u,w\}$, $B^* = \{v,x\}$
4 crossing edges

Figure 74: Tight example for Theorem 33.2. Solid edges cross the cut and dashed edges do not. Left: a locally optimal cut with 2 crossing edges. Right: a maximum cut with 4 crossing edges.

Note. The guarantee is no better than that of a random cut. Put each vertex in A or B independently with probability $\frac{1}{2}$, and for $e \in E$ let $X_e = 1$ if e crosses (A,B) and $X_e = 0$ otherwise. The two endpoints of e land on different sides with probability $\frac{1}{2}$, so $\mathbb{E}[X_e] = \mathbb{P}[X_e = 1] = \frac{1}{2}$. By linearity of expectation,

$$\mathbb{E}[\#\text{ crossing edges}] = \mathbb{E}\Big[\sum_{e \in E} X_e\Big] = \sum_{e \in E} \mathbb{E}[X_e] = \tfrac{1}{2}|E|.$$

In the WEIGHTED MAXIMUM CUT problem, each edge e has a nonnegative weight w_e, and the goal is to

maximize the total weight of crossing edges. Local search is still well defined: move v when the total weight of its non-crossing incident edges exceeds that of its crossing incident edges. The 50% guarantee still holds for locally optimal cuts, by the same double-counting argument with every count replaced by a total weight, and it also holds in expectation for a random cut. However, local search is no longer guaranteed to converge in polynomial time.

33.2 Principles of local search

Let X be the set of candidate solutions to a problem, such as the cuts of a graph, the tours of a traveling salesman instance, or the variable assignments of a constraint satisfaction problem. The key design choice is a NEIGHBORHOOD: for each $x \in X$, specify which $y \in X$ are its neighbors. For example:

- two cuts are neighbors if they differ by moving one vertex;
- two variable assignments are neighbors if they differ in the value of a single variable;
- two traveling salesman tours are neighbors if they differ in exactly two edges (Figure 75).

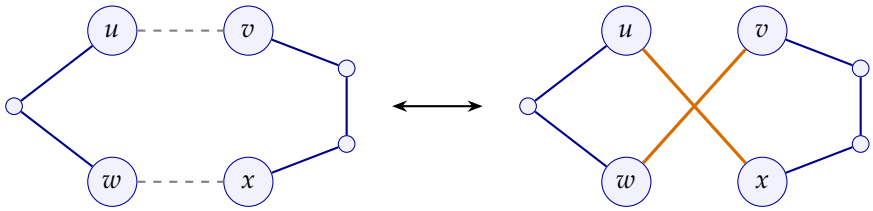


Figure 75: Neighboring tours. Removing the tour edges (u, v) and (w, x) and adding (u, x) and (w, v) gives another tour.

Algorithm 33.2: Generic local search

Input: A set X of candidate solutions with neighborhoods and an objective.
Output: A locally optimal solution.

```
x ← some initial solution
while x has a neighbor y that is superior to x do
    x ← y
return x
```

The generic algorithm leaves three choices open.

- *Initial solution.* Use a random solution, run many independent trials, and return the best local optimum found. Alternatively, start from the output of a good heuristic and use local search as a postprocessing step that can only improve it.
- *Which superior neighbor.* Choose one at random, choose the one with the biggest improvement, or use a more complex heuristic.

- *Neighborhoods.* Bigger neighborhoods make local optimality slower to verify but leave fewer bad local optima. The aim is a balance between solution quality and efficient searchability.

If X is finite and every step strictly improves some objective function, local search terminates, since no solution can repeat. It is usually not guaranteed to converge quickly, although it often does in practice; smoothed analysis is one way to explain this. Locally optimal solutions are generally not good approximations to global optima; running randomized local search many times and keeping the best local optimum mitigates this.

33.3 The 2-SAT problem

Definition 33.3 [2-SAT]. The input is n Boolean variables $x_1, \dots, x_n$ and m clauses, each a disjunction of 2 LITERALS, where a literal is some x_i or $\neg x_i$. The output is "yes" if some assignment of true and false values satisfies every clause simultaneously, and "no" otherwise.

Example: The instance $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee x_4) \wedge (\neg x_2 \vee \neg x_4)$ is a "yes" instance, satisfied for example by $x_1 = x_3 = \text{true}$ and $x_2 = x_4 = \text{false}$.

2-SAT can be solved in polynomial time, for example by a reduction to computing strongly connected components, by backtracking, or by the randomized local search below. In contrast, 3-SAT, where clauses have three literals, is the canonical NP-complete problem. Brute-force search takes about 2^n time, and randomized local search achieves about $(4/3)^n$ time (Schöning, 2002).

Algorithm 33.3: Papadimitriou's 2-SAT algorithm

```
Input: A 2-SAT instance with n variables.
Output: A satisfying assignment, or "unsatisfiable".
for ⌈log_2 n⌉ outer iterations do
    choose a uniformly random initial assignment
    for 2n^2 inner iterations do
        if the current assignment satisfies every clause then return it
        pick an arbitrary unsatisfied clause
        flip the value of one of its two variables, chosen uniformly at random
return "unsatisfiable"
```

The algorithm runs in polynomial time, since it performs $O(n^2 \log n)$ inner iterations, each of which checks the m clauses. It is always correct on unsatisfiable instances, because it reports an assignment only after verifying it. The key question is whether, on a satisfiable instance, it finds a satisfying assignment with probability close to 1. The analysis rests on random walks.

33.4 Random walks on a line

Consider a walk on the nonnegative integers that starts at position 0 at time 0. At each time step, the position goes up or down by 1 with probability $\frac{1}{2}$ each, except that from position 0 it moves to

position 1 with probability 1. For an integer $n \geq 1$, let T_n be the number of steps until the walk first reaches position n. It is a random variable over the coin flips at all time steps.

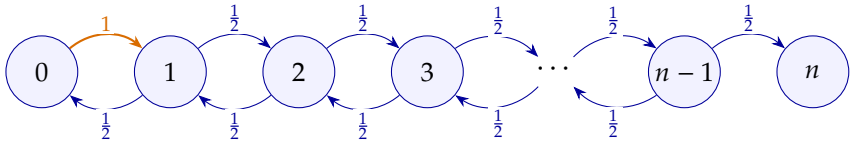


Figure 76: The random walk on the nonnegative integers. Position 0 moves to 1 with probability 1; every other position moves left or right with probability $\frac{1}{2}$ each.

Theorem 33.4 [Expected hitting time]. $\mathbb{E}[T_n] = n^2$.

Proof. For $0 \leq i \leq n$, let Z_i be the number of steps needed to reach n starting from position i, so $Z_0 = T_n$. At the edges,

$$\mathbb{E}[Z_n] = 0, \qquad \mathbb{E}[Z_0] = 1 + \mathbb{E}[Z_1].$$

For $1 \leq i \leq n-1$, condition on the first step. It goes left or right with probability $\frac{1}{2}$ each, after which the number of remaining steps is distributed as Z_{i-1} or Z_{i+1}, respectively:

$$\mathbb{E}[Z_i] = \tfrac{1}{2}\big(1 + \mathbb{E}[Z_{i-1}]\big) + \tfrac{1}{2}\big(1 + \mathbb{E}[Z_{i+1}]\big) = 1 + \tfrac{1}{2}\mathbb{E}[Z_{i+1}] + \tfrac{1}{2}\mathbb{E}[Z_{i-1}].$$

Rearranging,

$$\mathbb{E}[Z_i] - \mathbb{E}[Z_{i+1}] = \mathbb{E}[Z_{i-1}] - \mathbb{E}[Z_i] + 2.$$

The first difference is $\mathbb{E}[Z_0] - \mathbb{E}[Z_1] = 1$, so the consecutive differences are $1, 3, 5, \ldots$, and $\mathbb{E}[Z_i] - \mathbb{E}[Z_{i+1}] = 2i + 1$. Summing the differences for $i = 0, \ldots, n-1$ telescopes:

$$\mathbb{E}[Z_0] - \mathbb{E}[Z_n] = \sum_{i=0}^{n-1} (2i + 1) = n^2,$$

since pairing the term $2i + 1$ with $2(n - 1 - i) + 1$ gives pairs summing to $2n$, and there are $n/2$ such pairs. Since $\mathbb{E}[Z_n] = 0$, this gives $\mathbb{E}[T_n] = \mathbb{E}[Z_0] = n^2$. □

Corollary 33.5. $\mathbb{P}[T_n > 2n^2] \leq \frac{1}{2}$.

Proof. This is a special case of Markov's inequality. Let $p = \mathbb{P}[T_n > 2n^2]$. Split the expectation at $2n^2$; the first part is nonnegative, and every k in the second part is at least $2n^2$:

$$n^2 = \mathbb{E}[T_n] = \sum_{k=0}^{2n^2} k\,\mathbb{P}[T_n = k] + \sum_{k=2n^2+1}^{\infty} k\,\mathbb{P}[T_n = k] \geq 2n^2\,\mathbb{P}[T_n > 2n^2] = 2n^2 p.$$

Therefore $p \leq \frac{1}{2}$. □

33.5 Analysis of Papadimitriou's algorithm

Theorem 33.6 [Papadimitriou's algorithm on satisfiable instances]. *For a satisfiable 2-SAT instance with n variables, Papadimitriou's algorithm produces a satisfying assignment with probability at least $1 - \frac{1}{n}$.*

Proof. First consider a single outer iteration. Fix an arbitrary satisfying assignment a^*. Let a_t be the algorithm's assignment after inner iteration t, for $t = 0, 1, \ldots, 2n^2$, and let $X_t \in \{0, 1, \ldots, n\}$ be the number of variables on which a_t and a^* agree. Both are random variables. If $X_t = n$, then $a_t = a^*$ and the algorithm halts with a satisfying assignment.

Suppose a_t is not satisfying and the algorithm picks an unsatisfied clause on variables x_i and x_j. Since a_t makes both literals of the clause false while a^* makes at least one true, a^* differs from a_t on x_i, on x_j, or on both. Flipping a random one of the two gives:

1. if a^* and a_t differ on both x_i and x_j, then $X_{t+1} = X_t + 1$ with probability 1;
2. if they differ on exactly one of x_i, x_j, then $X_{t+1} = X_t + 1$ or $X_t - 1$ with probability $\frac{1}{2}$ each.

In particular, if $X_t = 0$ then a_t disagrees with a^* everywhere and case 1 applies, matching the walk's forced move from 0.

So $X_0, X_1, \ldots, X_{2n^2}$ behave like the random walk on the nonnegative integers, with three differences, all of which only help: some steps move right with probability 1 instead of $\frac{1}{2}$; X_0 may exceed 0; and the process may stop early at a satisfying assignment other than a^*, before $X_t = n$. Hence a single outer iteration finds a satisfying assignment with probability at least

$$\mathbb{P}[T_n \le 2n^2] \ge \tfrac{1}{2},$$

by Corollary 33.5. The outer iterations use fresh random choices, so they are independent, and

$$\mathbb{P}[\text{algorithm fails}] \le \mathbb{P}[\text{all } \lceil \log_2 n \rceil \text{ outer iterations fail}] \le \left(\tfrac{1}{2}\right)^{\lceil \log_2 n \rceil} \le \frac{1}{n}. \qquad \square$$

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), Local Search Algorithms.

Lecture 34: Matching, Flows, and Linear Programming

34.1 Stable matchings

Consider two node sets U and V, thought of as "men" and "women", with $|U| = |V| = n$ for simplicity. Each node ranks all nodes on the other side, and different nodes may have different rankings. Applications include matching hospitals with residents and colleges with applicants.

Definition 34.1 [Stable matching]. A perfect matching matches each node of U to a distinct node of V. It is a stable matching if, whenever $u \in U$ and $v \in V$ are not matched to each other, either u prefers his mate v' to v, or v prefers her mate u' to u. That is, no unmatched pair would both rather be with each other than with their mates.

Algorithm 34.1: Gale–Shapley proposal algorithm

Input: Sets U, V of size n, each node with a ranking of the other side.
Output: A stable matching.

```
while some man u is unattached do
    u proposes to the top woman v on his list who has not yet rejected him
    v keeps the best proposal received so far and rejects the other
return the current engagements
```

Each woman holds at most one proposal and each man is engaged to at most one woman, so throughout the algorithm the current engagements form a matching. A woman who has received a proposal stays engaged from then on, and each change of her partner is to a man she prefers.

Example: Let every man A, B, C rank the women D, E, F in that order, while D ranks A, B, C, E ranks B, C, A, and F ranks C, A, B. All three men first propose to D, who keeps A. Then B proposes to E and is accepted; C proposes to E, who prefers B; finally C proposes to F. The result is $\{(A, D), (B, E), (C, F)\}$ (Figure 77).

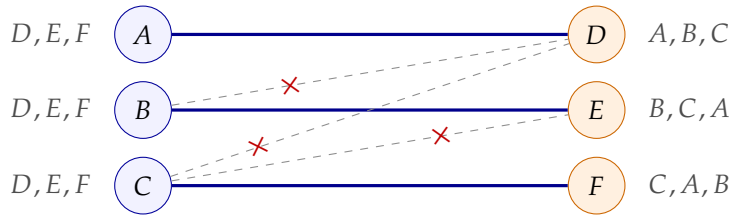


Figure 77: The proposal algorithm on the example. Preference lists are written beside each node. Rejected proposals are crossed out; the remaining proposals form the stable matching.

Theorem 34.2 [Gale–Shapley]. *The proposal algorithm terminates with a stable matching after at most n^2 iterations. In particular, a stable matching always exists.*

Proof. *Running time.* Each iteration is one proposal, and each man proposes to each woman at most once, so each man makes at most n proposals and there are at most n^2 iterations.
Perfect matching. Suppose some man is rejected by all n women. Then every woman has received a proposal, so all n women are engaged at the end. Since the engagements form a matching, all n men are engaged as well, a contradiction. So the algorithm ends only when no man is unattached, and the final matching is perfect.
Stability. Consider $u \in U$ and $v \in V$ not matched to each other. If u never proposed to v, then, because u proposes in order of preference, he is matched to someone he prefers to v. If u did propose to v, then v rejected him for a better offer at some point; since women only trade up, v ends matched to someone she prefers to u. □

34.2 Bipartite matching and maximum flow

Definition 34.3 [Bipartite matching]. The input is a bipartite graph $G = (U, V, E)$, so each edge has one endpoint in U and one in V. A matching is a set $M \subseteq E$ of pairwise disjoint edges. The goal is a matching of maximum size.

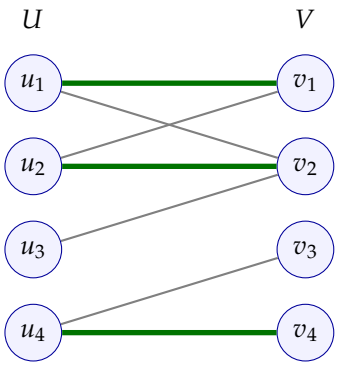


Figure 78: A bipartite graph whose maximum matching (thick edges) has size 3. The left nodes u_1, u_2, u_3 have only the two neighbors v_1, v_2, so at most two of them can be matched.

There is a straightforward reduction from bipartite matching to the maximum flow problem.

Definition 34.4 [Maximum flow]. The input is a directed graph $G = (V, E)$ with a source vertex s, a sink vertex t, and a capacity u_e on each edge e. An s-t flow assigns each edge an amount between 0 and its capacity such that, at every vertex other than s and t, the flow in equals the flow out. The goal is to send as much flow as possible from s to t.

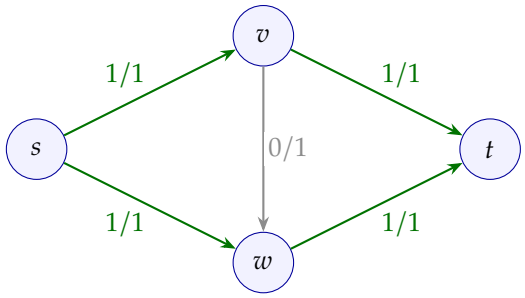


Figure 79: A flow network with unit capacities. Edge labels give flow/capacity for a maximum flow of value 2, which uses the paths $s \to v \to t$ and $s \to w \to t$.

The maximum flow problem is solvable in polynomial time, for example by nontrivial greedy algorithms based on "augmenting paths". They are nontrivial because a naive greedy choice can get stuck: in Figure 79, sending one unit along $s \to v \to w \to t$ leaves no s-t path with spare capacity, although the maximum flow value is 2.

34.3 Selfish flow and Braess's paradox

In a SELFISH FLOW model, a flow network carries one unit of selfish traffic, and each edge has a DELAY FUNCTION giving its travel time as a function of the load on the edge. Each part of the traffic uses a path of minimum total delay.

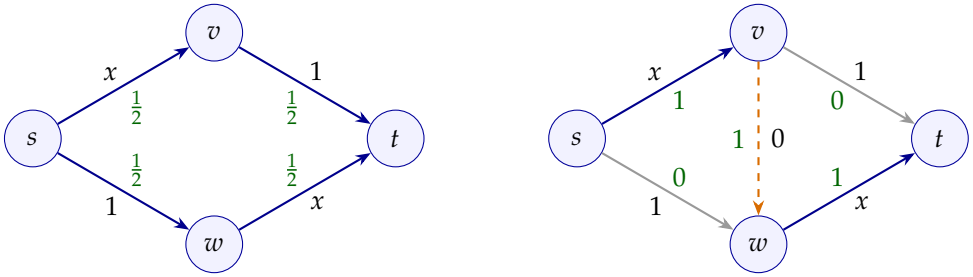


Figure 80: Braess's paradox. Black labels give each edge's delay as a function of its load x; green labels give the load. Left: the two routes have equal delay, and traffic splits evenly, so each route takes $\frac{1}{2} + 1 = 1.5$. Right: after adding a zero-delay edge from v to w, all traffic takes $s \to v \to w \to t$, which takes $1 + 0 + 1 = 2$.

In the network of Figure 80, the edges (s, v) and (w, t) have delay x under load x, while (v, t) and (s, w) have constant delay 1. In the steady state, traffic splits 50/50, and the commute time is 1.5 hours on either route. BRAESS'S PARADOX (1968): after adding a zero-delay "teleporter" from v to w, the commute time of selfish traffic degrades to 2 hours. Indeed, when all traffic uses $s \to v \to w \to t$, each of (s, v) and (w, t) carries load 1; every route then has delay 2, so no traffic can improve by switching.

34.4 Linear programming

The general LINEAR PROGRAMMING problem is to optimize a linear function over the intersection of halfspaces. It generalizes maximum flow and many other problems. Linear programs can be solved efficiently, both in theory and in practice, which makes linear programming a very powerful "black-box" subroutine.

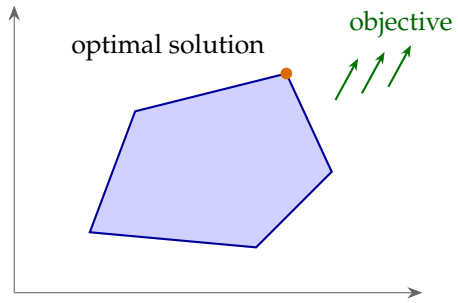


Figure 81: Schematic linear program in two dimensions. The feasible region is an intersection of halfplanes; the optimum is a point of the region that is furthest in the direction of the objective function.

Two extensions are CONVEX PROGRAMMING, which is polynomial-time solvable under mild conditions, and INTEGER PROGRAMMING, which is NP-hard in general.

34.5 Other topics and models

- Deeper study of data structures, graph algorithms, approximation algorithms, and other course topics.
- Geometric algorithms, both low-dimensional (for example, convex hull) and high-dimensional (for example, nearest neighbors in information retrieval).
- Algorithms that run forever, usually in real time, such as caching and routing.
- Algorithms with bounded memory, or STREAMING ALGORITHMS, for example to maintain statistics at a network router.
- Algorithms that exploit parallelism, for example via MapReduce and Hadoop.

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part II), The Wider World of Algorithms.